

Recitation 1

Introduction to Operating Systems

Daniel Karalnik Liam Tal

Spring Semester 2026

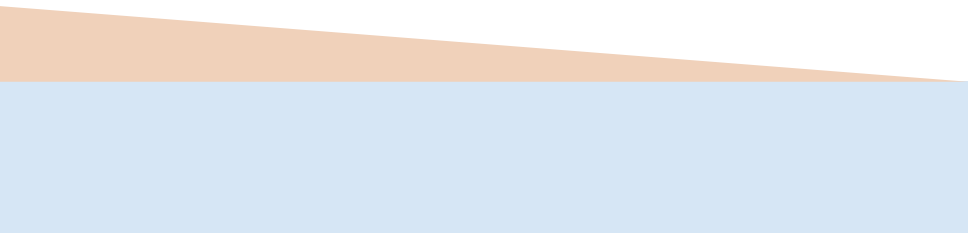


Table of Contents

1 Intro

- Course Logistics

2 The Modern Operating System

- Basic Concepts
- Threads and Processes
- Time Accounting



Intro

The Way of the OS



Contact Information

Teaching Assistants:

- Daniel Karalnik: daniel.karalnik@post.runi.ac.il
- Liam Tal: liam.tal@post.runi.ac.il

Grader:

- Maor Mizrachi: maor.mizrachi@post.runi.ac.il

Office Hours: By appointment only

Homework Overview

3 hybrid assignments (theoretical + programming) – **40% of final grade**

- All assignments are **mandatory**
- Submit **before the deadline** – follow all submission instructions
- **Theoretical answers:** Must be fully explained; partial answers may lose marks
- **Code requirements:** Correct, readable, well-organized, and efficient
- Deviations from instructions may result in point deductions

Extension requests: Email both the lecturer and grader (CC both)

Homework Appeal Policy

- Send appeals **directly to the grader**
- Non-administrative appeals trigger a **full re-check** of the assignment
 - This may result in grade increases *or decreases*

Note: TAs and lecturer will not pre-evaluate appeals. If you believe your case is valid, submit it directly to the grader.

The Modern Operating System

Terminology

Key Terminology

- >  **Latency (time unit)** – The time to complete a job.
- >  **Throughput (amount/time)** – The amount of data processed in a time unit.
- >  **CPU Utilization** – The amount of time the CPU is executing non-OS code.
- >  **Overhead** – Time spent on non-program tasks (memory access, OS computation, etc.).
- >  **CPU Usage** – Utilization + Overhead.

Exercise 1: Terminology

Problem

A program takes 10 seconds to complete a task. During this time:

- The CPU spends 6 seconds executing program code.
- It spends 2 seconds accessing memory.
- It spends 2 seconds performing OS-related tasks.
- The system processes 500GB in these 10 seconds.

Exercise 1: Terminology

Calculate

- 1 The CPU utilization (time executing non-OS code)
- 2 The overhead (time not running program code)
- 3 The total CPU usage (utilization + overhead)
- 4 The throughput (amount of data per time unit)
- 5 The latency (time to complete a job)

Hint

Use the terminology definitions from the previous slide. Remember: Utilization $\neq$ Usage!

Exercise 1: Terminology - Solution

Given

Total: 10s | CPU exec: 6s | Memory: 2s | OS tasks: 2s

Solution

- 1 **CPU Utilization** (time executing non-OS code):
CPU Utilization = 6s
- 2 **Overhead:** Overhead = Memory + OS tasks = $2 + 2 = 4$ seconds

Exercise 1: Terminology - Solution

Given

Total: 10s | CPU exec: 6s | Data processed: 500GB

Solution

3 **CPU Usage** (utilization + overhead):

$$\text{CPU Usage} = \text{Utilization} + \text{Overhead} = 6 + 4 = 10 \text{ s}$$

4 **Throughput:** $\text{Throughput} = \frac{\text{Data processed}}{\text{Total time}} = \frac{500}{10} = 50 \text{ GB/s}$

Exercise 1: Terminology - Solution

Given

Total time = 10 seconds

Solution

- 5 **Latency:** Latency is the time taken to complete the task:
 $\text{Latency} = \text{Total time} = 10 \text{ seconds/task}$

Exercise 1: Key Takeaways

💡 Key Insight

CPU Utilization measures only program code execution, while **CPU Usage** includes all CPU activity (utilization + overhead).

☰ Summary

- **CPU Utilization:** 6s (time executing non-OS code)
- **Overhead:** 4s (time not running program code)
- **CPU Usage:** 10s (utilization + overhead)
- **Throughput:** 50 GB/s (amount per time unit)
- **Latency:** 10s/task (time to complete a job)

Exercise 1: Key Takeaways

Percentage Breakdown

In this example:

- **60%** of CPU time → program code execution
- **40%** of CPU time → overhead (memory access, OS tasks)

Real-World Implication

In practice, high overhead percentages can significantly impact application performance. Optimizing memory access patterns and reducing OS calls can improve CPU utilization.

Exercise 1b: Idle CPU Time

Problem

A job runs on the system for 12 seconds. During this time:

- The CPU spends 5 seconds executing non-OS program code
- The CPU spends 2 seconds performing OS computations
- The CPU spends 1 second accessing memory
- For the remaining time, the CPU is idle (not executing the job and waiting for IO, i.e network request, user input etc)
- During the execution of the job, the system processes 360 GB of data

Exercise 1b: Idle CPU Time



Calculate

- 1 CPU Utilization (time executing non-OS code)
- 2 Overhead (time not running program code)
- 3 CPU Usage (utilization + overhead)
- 4 CPU Idle Time:
 - In seconds
 - As a percentage of total time
- 5 Throughput (amount of data processed per time unit)
- 6 Latency (time to complete a job)

Exercise 1b: Idle CPU Time

Hint

$\text{CPU Idle Time} = \text{Total Time} - \text{CPU Usage}$. The CPU is idle when waiting for I/O!

Exercise 1b: Idle CPU Time - Solution

Given

Total: 12s | Non-OS execution: 5s | OS computation: 2s | Memory access: 1s

Solution

- 1 **CPU Utilization** (time executing non-OS code):
CPU Utilization = 5s
- 2 **Overhead** (time not running program code):
Overhead = OS + Memory = 2 + 1 = 3s

Exercise 1b: Idle CPU Time - Solution

Given

Total: 12s | Utilization: 5s | Overhead: 3s | Data: 360 GB

Solution

- 3 CPU Usage** (utilization + overhead):
 $\text{CPU Usage} = 5 + 3 = 8 \text{ s}$
- 4 CPU Idle Time:** Idle = Total – Usage = $12 - 8 = 4 \text{ s} \approx 33.3\%$
- 5 Throughput** (amount per time unit):
 $\text{Throughput} = \frac{360 \text{ GB}}{12 \text{ s}} = 30 \text{ GB/s}$
- 6 Latency** (time to complete a job): Latency = 12 s/job

Exercise 1b: Key Takeaways

Key Insight

CPU Usage does not have to be 100%! Idle CPU time is normal and occurs when the CPU is waiting or has no work to do.

When does CPU idle?

- Waiting for I/O operations (disk, network)
- Waiting for user input
- Waiting for external events or signals
- No ready tasks in the scheduler queue

Exercise 1b: Key Takeaways

☰ Summary

- **CPU Utilization:** 5s (time executing non-OS code)
- **Overhead:** 3s (OS computation + memory access)
- **CPU Usage:** 8s (utilization + overhead)
- **CPU Idle:** 4s
- **Throughput:** 30 GB/s
- **Latency:** 12s per job

Exercise 1b: Key Takeaways

Percentage Breakdown

- **Utilization:** $\frac{5}{12} \approx 41.7\%$ (program code)
- **Overhead:** $\frac{3}{12} = 25\%$ (OS + memory)
- **Usage:** $\frac{8}{12} \approx 66.7\%$ (active CPU)
- **Idle:** $\frac{4}{12} \approx 33.3\%$ (waiting)

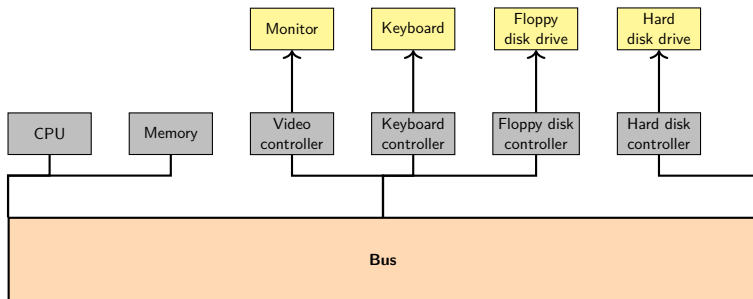
Visual Summary

$$\text{Utilization (41.7\%)} + \text{Overhead (25\%)} + \text{Idle (33.3\%)} = 100\%$$

PC Structure

Hardware Architecture

The **bus** connects all hardware components, allowing CPU, memory, and controllers to communicate.



OS Abstractions

Core OS Abstractions

-  **Thread** – Abstract the CPU execution of the program
-  **Address Space** – Abstracts the memory
-  **File** – Abstract the inner structure of storage
-  **Process** – Ties all entities of a running program together

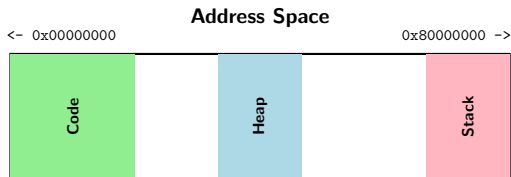
? Why abstractions? They hide hardware complexity and provide a consistent interface for programs to interact with system resources.

OS Abstractions - Address Space



Virtual Memory

Each process has its own **virtual address space** – an illusion of having the entire memory to itself.



Code: Program instructions

Heap: Dynamic allocations
calls & locals

Stack: Function

OS Abstractions - Files

File Abstraction

A **file** abstracts storage details – programs see a simple path, not disk sectors or blocks.

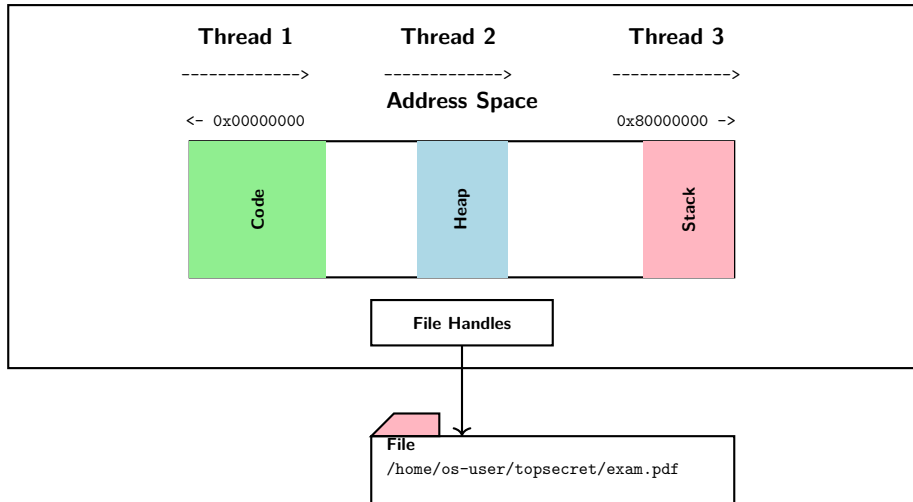
File

`/home/os-user/topsecret/exam.pdf`

Benefits: Uniform interface for different storage types (HDD, SSD, network drives), access control, and organization via directories.

OS Abstractions - Process

Process



≡ What is a Thread?

A thread is the basic unit of CPU execution within a process.

-  **Old single-process OS:** The thread running the OS is also used to run applications.
-  **Modern multi-process OS:** For each new process, a thread is spawned (the "main thread"). The process executes as long as its main thread runs.
- Multiple processes with threads running concurrently $\Rightarrow$ multiple threads running concurrently.

Threads - Memory Caveats

Memory Sharing

Each process can have multiple threads which **partially share memory** (e.g., the heap). More on this in later lectures.

✓ Shared

- Code section
- Heap memory
- Global variables
- File handles

✗ Not Shared

- Stack (each thread has its own)
- Registers
- Program counter

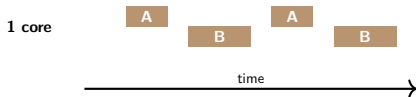
Threads - Creation Context

i Note: In our context, all processes and threads are created and managed by the OS.

Concurrency vs. Parallelism

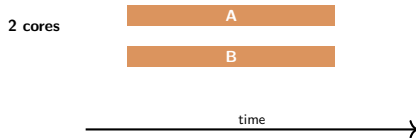
⌘ Concurrency

Run multiple tasks in *overlapping* time periods (interleaved execution)



⚡ Parallelism

Run multiple tasks *simultaneously* (true parallel execution)



💡 **Key Difference:** Concurrency is about *dealing* with multiple things at once. Parallelism is about *doing* multiple things at once.

Threads - Parallelism

Parallel Execution

- If we have enough CPU cores to serve all our processes we can run two processes in parallel.
- However in most cases there are more processes running on our PCs than we have cores.

The Scheduling Problem

To solve this problem, the Operating System will try to schedule each thread to run on the CPU on its own turn.

Threads - Concurrency

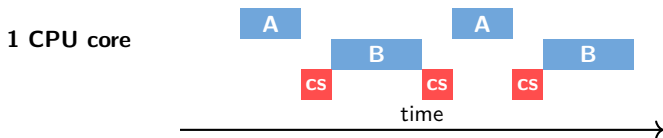
OS Scheduling

To solve the problem of limited cores vs. number of processes/threads, the OS will schedule each thread to run on some core according to a certain policy (more on this later).

Context Switch

The process where the OS interrupts the execution of a thread to execute another is called **Context Switch**.

Context Switch



⚠ Performance Warning

Context switching is a **"heavy" action**. Over time, it can consume a significant portion of CPU usage, especially in scenarios with many short-lived tasks.

Exercise 2: Context Switching

Problem

A system is running three processes (P1, P2, P3) concurrently over 15 seconds:

- CPU execution: 4s (P1) + 3s (P2) + 5s (P3)
- Context switches: 1s, Memory access: 1s, OS tasks: 1s
- Total data processed: 600GB

Exercise 2: Context Switching



Calculate

- 1 The CPU utilization (time executing non-OS code)
- 2 The overhead (time spent on other things, e.g. memory access, OS computation)
- 3 The CPU usage (utilization + overhead)
- 4 The throughput (amount of data processed per time unit)
- 5 The latency (time to complete a job)



Hint

Context switch time is part of the overhead – it's time the CPU spends **not** running your program code.

Exercise 2: Context Switching - Solution

Given Data

Total: 15s | Execution (non-OS code): $P1(4s) + P2(3s) + P3(5s) = 12s$

Solution

- 1 **CPU Utilization** (time executing non-OS code):
 $CPU\ Utilization = 4 + 3 + 5 = 12s$

Note

CPU Utilization sums the execution time of **all three processes** – the actual time spent running program code (not OS operations).

Exercise 2: Context Switching - Solution

Given Data

Total: 15s | Context switch: 1s | Memory access: 1s | OS computation: 1s

Solution

- 2 **Overhead** (time not running program code):
 $\text{Overhead} = \text{CS} + \text{Memory} + \text{OS} = 1 + 1 + 1 = 3 \text{ s}$
- 3 **CPU Usage** (utilization + overhead):
 $\text{CPU Usage} = \text{Utilization} + \text{Overhead} = 12 + 3 = 15 \text{ s}$

Exercise 2: Context Switching - Solution

Given Data

Total: 15s | 3 jobs (processes) | Data processed: 600GB

Solution

4 **Throughput** (amount per time unit):
$$\text{Throughput} = \frac{\text{Data}}{\text{Time}} = \frac{600 \text{ GB}}{15 \text{ s}} = 40 \text{ GB/s}$$

5 **Latency** (time to complete a job):
$$\text{Latency} = \frac{\text{Total Time}}{\text{Jobs}} = \frac{15}{3} = 5 \text{ s/job}$$

Note: In this context, a job is essentially a process.

Exercise 2: Key Takeaways

Key Insight

Context switching adds overhead but enables concurrency. The trade-off is essential for multi-tasking systems.

Summary

- **CPU Utilization:** 12s (time executing non-OS code)
- **Overhead:** 3s (time not running program code)
- **CPU Usage:** 15s (utilization + overhead)
- **Throughput:** 40 GB/s (amount per time unit)
- **Latency:** 5s per job (time to complete a job)

Time Accounting: Time Metrics

Wall Time

Total elapsed time measured by a stopwatch.

“How long did it take?”

System Time

CPU time executing **kernel code** (OS code) on behalf of the program (scheduling, context switching).

User Time

CPU time executing the program's code in **user mode** (non-OS).

Wait Time

Time a thread is alive but **not executing** (preempted, blocked, sleeping).

Time Accounting: Formulas

Key Formulas

- **CPU Time** = User Time + System Time
- **Wait Time (per thread)** = Wall Time – Execution Time
- **Total Wait Time** = $\sum$ (Wait Time per thread)

Single Core

User + System $\leq$ Wall Time
CPU does one thing at a time

Multi-Core

User + System **may exceed** Wall Time
Multiple cores work in parallel

Wait Time: Single vs Multi-Threaded

Single Thread

Wait time occurs when:

- Blocked on I/O (disk, network)
- Sleeping (e.g., `sleep()`)
- Waiting for user input

No competition for CPU – only external waits

Multi-Threaded

Wait time **also** includes:

- Preempted by scheduler
- Waiting for a CPU core
- Other threads running

Threads compete for limited CPU time

Wait Time in Multi-Threading

Key Insight

In multi-threaded programs, **total wait time grows** because threads must share CPU resources. Even without I/O, threads wait while others execute!

Exercise 3: Time Accounting

Scenario

A program creates **5 threads**. Each thread:

- Needs **40 ms** of pure user-mode CPU execution
- Performs no I/O operations
- Does not voluntarily sleep
- Terminates immediately after finishing
- All threads terminate at approximately the same time.

Total Work

$5 \text{ threads} \times 40 \text{ ms} = \mathbf{200 \text{ ms}}$ of user-mode work

Exercise 3: Part A – No Overhead

Question

Assume fair time slicing and **negligible context-switch overhead**.

For both **single-core** and **4-core** CPUs, calculate:

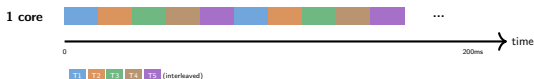
- 1 Minimum wall time
- 2 Total user time
- 3 Total system time
- 4 Total wait time (summed over all threads)
- 5 Is this concurrency, parallelism, or both?

Exercise 3: Part A – No Overhead



Wait Time (per thread) = Wall Time – Execution Time of that thread

Exercise 3: Part A – Solution (Single Core)



✓ Solution

- **Wall Time:** $5 \times 40 = 200$ ms
- **User Time:** 200 ms **System Time:** 0 ms
- **Wait Time:** Each thread waits while others run:
 $(200 - 40) \times 5 = 800$ ms

Concurrency without parallelism – threads overlap in time but not in execution

Exercise 3: Wait Time Calculation (Single Core)

How Wait Time is Calculated

With **fair time slicing** (round-robin) under our naive assumption:

- All threads start at time 0
- Threads take turns in small time slices
- Each thread **terminates** when it completes its 40ms of work
- With equal progress, all finish **approximately** around time 200ms

Note

This is an **estimate** – threads cannot all finish at the exact same instant. In practice, they finish in quick succession as each completes its final time slice.

Exercise 3: Wait Time Calculation (Single Core)

📊 Per-Thread Breakdown (Estimated)

Thread	Lifetime	Exec Time	Wait Time
T1	0 → ~200 ms	40 ms	~160 ms
T2	0 → ~200 ms	40 ms	~160 ms
T3	0 → ~200 ms	40 ms	~160 ms
T4	0 → ~200 ms	40 ms	~160 ms
T5	0 → ~200 ms	40 ms	~160 ms
Total	—	200 ms	~800 ms

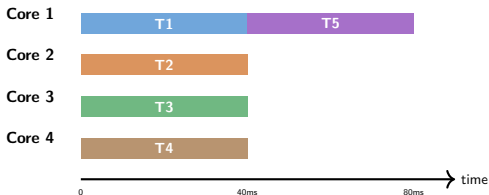
📐 Formula

$$\text{Wait Time} \approx \text{Lifetime} - \text{Execution Time} = 200 - 40 = 160 \text{ ms/thread}$$

Exercise 3: Part A – (4 Cores)

- Assume that we now have 4 cores.
- Scheduling is now by FIFO order and once scheduled, threads run to completion.

Exercise 3: Part A – Solution (4 Cores)



✓ Solution

- **Wall Time:** $\lceil 5/4 \rceil \times 40 = 80$ ms
- **User Time:** 200 ms **System Time:** 0 ms
- **Wait Time:** Only T5 waits 40ms $\Rightarrow$ total = 40 ms

Concurrency with parallelism – User time (200ms) > Wall time (80ms)!

Exercise 3: Part B – Context Switching

Additional Assumptions (Single Core)

- Context switch every **5 ms** of execution
- Each context switch costs **0.2 ms**
- Context switch also occurs after the final chunk

Exercise 3: Part B – Context Switching

Calculate

- 1 How many execution chunks per thread?
- 2 How many context switches total?
- 3 Total system time?
- 4 New wall time?
- 5 Total user time?
- 6 Total wait time?

Exercise 3: Part B – Solution (Step 1)

✓ Basic Calculations

- **Chunks per thread:** $40/5 = 8$ chunks
- **Total chunks:** $5 \times 8 = 40$ chunks
- **Context switches:** 40 (one after each chunk, including final)
- **System time:** $40 \times 0.2 = 8$ ms
- **User time:** 200 ms (unchanged)

🕒 Round Duration & Per-Thread Overhead

- One full round = all 5 threads run once: $5 \times (5 + 0.2) = \mathbf{26\ ms}$
- CPU burst per thread (user + CS): $40 + 8 \times 0.2 = \mathbf{41.6\ ms}$

Exercise 3: Part B – Solution (Important Note!)



Assumption

A context switch occurs **after every chunk**, including the final chunk of each thread. The thread is not considered complete until its final context switch finishes.

Exercise 3: Part B – Solution (Step 2)

✓ Staggered Finish & Wait Times

After **7 full rounds** ($7 \times 26 = 182$ ms), each thread has 1 chunk left.

In round 8, threads finish **one by one**, staggered by 5.2 ms.

Each turnaround **includes** the thread's own final context switch:

Thread	Exec done	+ final CS	Turnaround	Wait ($T_i - 41.6$)
T_1	187.0 ms	187.2 ms	187.2 ms	145.6 ms
T_2	192.2 ms	192.4 ms	192.4 ms	150.8 ms
T_3	197.4 ms	197.6 ms	197.6 ms	156.0 ms
T_4	202.6 ms	202.8 ms	202.8 ms	161.2 ms
T_5	207.8 ms	208.0 ms	208.0 ms	166.4 ms

Exercise 3: Part B – Solution (Step 3)

Results

➤ **Wall time:** 208 ms

➤ **Total wait:** $145.6 + 150.8 + 156.0 + 161.2 + 166.4 = 780$ ms

Exercise 3: Part B – Insight

Insight

- Context switching adds **system time** (8 ms). User work stays unchanged.
- Not all threads finish at the same time – in the final round, each thread waits for the ones scheduled before it.
- T_5 waits **20.8 ms more** than T_1 (4 threads $\times$ 5.2 ms gap).
- **Wait time** = Turnaround – CPU burst (per thread, not a single multiplication).
- CPU burst includes **both** user time and the thread's own CS overhead: $40 + 1.6 = 41.6$ ms.

Exercise 3: Key Takeaways

Comparison Table

Metric	1 Core	4 Cores	1 Core + CS
Wall Time	200 ms	80 ms	208 ms
User Time	200 ms	200 ms	200 ms
System Time	0 ms	0 ms	8 ms
Wait Time	800 ms	40 ms	780 ms

Key Insights

- User time stays constant – it's the **actual work**
- Parallelism reduces wall time and wait time dramatically
- Context switching adds system time **and** staggers thread completion, but total wait (780 ms) is actually *less* than sequential (800 ms) because threads overlap their execution via time-sharing

Exercise 3: Key Takeaways

“ One-Line Intuition

Wall time = how long it took

User + system time = how much CPU you used

Wait time = how long you existed but weren't running

✓ Remember

- Wait time $\neq$ system time (common misconception!)
- Preemption without I/O still creates wait time
- With parallelism: User time $>$ Wall time is normal

See You Next Week!

Questions?

Recitation 2

Multi-Threading and Synchronization

Daniel Karalnik Liam Tal

Spring Semester 2026

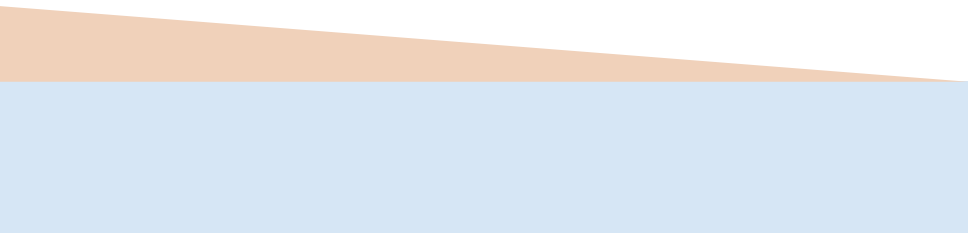


Table of Contents

1 Process and Thread Fundamentals

- › Process Structure
- › Thread Basics
- › Lifecycle Management

2 Process and Thread Communication

- › Communication Models
- › Shared Memory in Practice

3 The Synchronization Problem

- › Overview

Table of Contents

4 Low-Level Synchronization

- › Atomic Operations
- › Critical Sections
- › Spinlocks
- › C Atomics Library
- › Memory Ordering

5 Synchronization Primitives

- › Semaphores
- › Read-Write Locks
- › Condition Variables

6 Classic Synchronization Problems

- › Producer-Consumer Pattern

Process and Thread Fundamentals

Structure of a Process

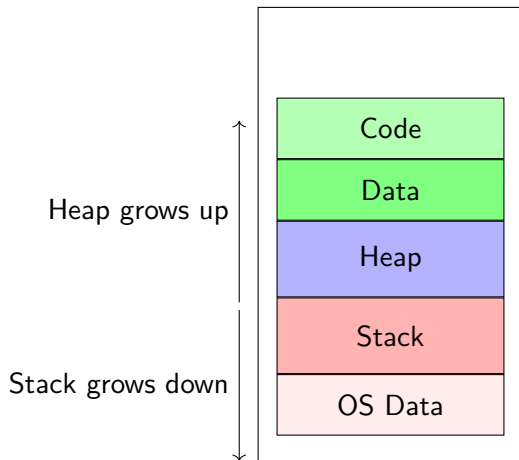
Process Memory Layout

The memory of a **single-threaded process** is (usually) split into 5 sections:

- **Code** – Instructions to execute
- **Data** – Static data compiled into executable file (hard-coded strings...)
- **Heap** – Manually managed memory (malloc, new...)
- **Stack** – Automatically managed memory
 - Local variables
 - Function call stack
- **Metadata** – OS data about the process

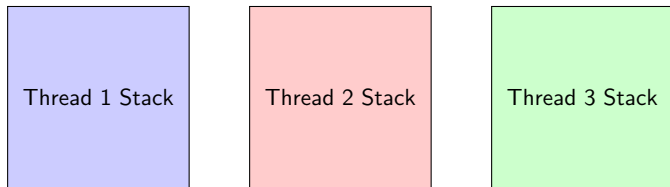
Memory Layout and Stack

Process P_1



Thread Stack

- Each thread has its own stack for function calls and local variables
- The stack pointer (SP) and base pointer (BP) manage the stack
- The stack operates in a Last In, First Out (LIFO) manner
- Stores:
 - Local variables
 - Function call information (stack frames)
 - Return addresses
- Thread stacks are allocated separately to avoid conflicts



Each thread has its own stack in memory.

Stack Growth

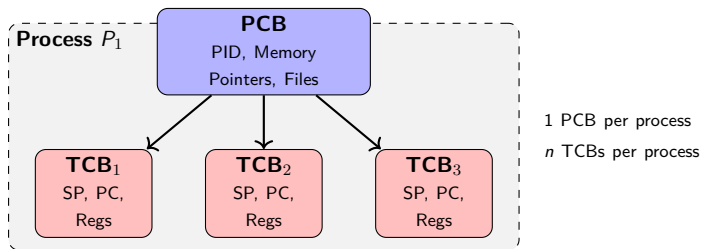
↑ How the Stack Grows

The stack is **dynamic** – it grows and shrinks during execution:

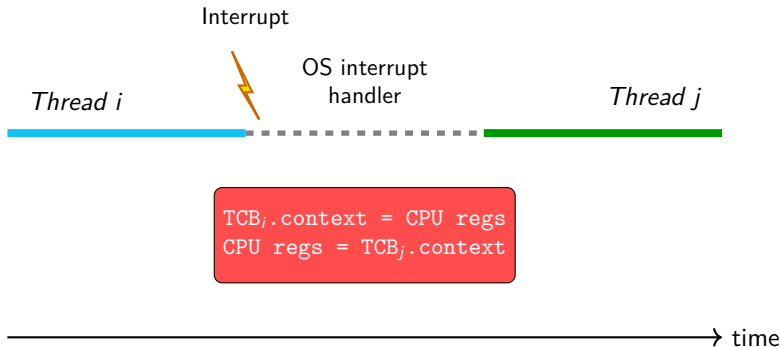
- **Grows** when a function is called (new stack frame pushed)
 - **Shrinks** when a function returns (stack frame popped)
-
- Stack typically grows **downward** (toward lower addresses)
 - The OS sets a **maximum stack size** for each thread
 - Stack can grow dynamically **up to this limit**
 - Exceeding the limit causes **stack overflow**

Process and Thread Control Blocks

- **Process Control Block (PCB):** Stores process-level information like PID, memory allocation, and open files.
- **Thread Control Block (TCB):** Contains thread-specific data such as stack pointer, registers, and thread state.
- Each process has **one PCB** but can have **multiple TCBs** (one per thread).

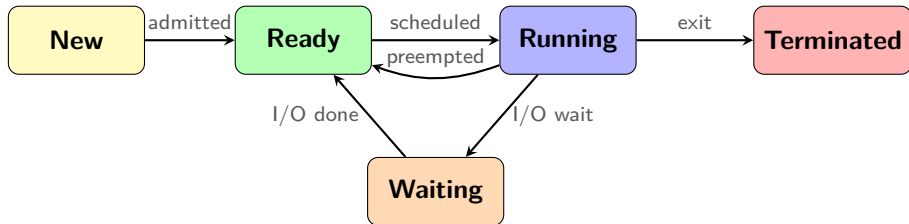


Context Switching



- **Context:** Stack pointer (SP), Program counter (PC), CPU registers
- **Requires:** Updating the PCB and TCB
- **Overhead:** Saving/loading states, managing memory mappings

Thread States



➤ **New:** Being created

➤ **Ready:** Waiting for CPU

➤ **Running:** Executing

➤ **Waiting:** Blocked on I/O

➤ **Terminated:** Finished

Process Creation with fork()

- fork() creates a **copy** of the calling process
- Parent and child continue execution from the same point
- **Return value** distinguishes parent from child:
 - **Parent**: receives child's PID (positive)
 - **Child**: receives 0
 - **Error**: returns -1

What Does `fork()` Copy?

Child receives a copy of:

- Address space (code, data, heap, stack)
- Open file descriptors (more on that later)
- Signal handlers (more on that later)
- Environment variables
- Current working directory

Not copied (unique to child):

- PID (child gets new PID)
- Parent PID (set to original process)
- Pending signals (cleared, more on that later)
- **Other threads** – only the calling thread is copied

Copy-on-Write (COW): Pages (memory sections) are shared until one process writes, then copied. Efficient for `fork()` + `exec()` pattern.

fork() Example in C

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
    } else if (pid == 0) {
        printf("Child PID: %d\n", getpid());
    } else {
        printf("Parent, child: %d\n", pid);
        wait(NULL);
    }
    return 0;
}
```

Output:

```
Parent, child: 1234
Child PID: 1234
```

Order may vary

The `exec()` Family

- `exec()` **replaces** the current process memory and code with a new program
- The PID remains the same – only the code/data/stack change
- Does **not return** on success (the old program is gone)

Common variants:

- `execl(path, arg0, arg1, ..., NULL)` – args as list
- `execv(path, argv[])` – args as array
- `execlp/execvp` – searches PATH for executable

`fork()` + `exec()` pattern:

- 1 Parent calls `fork()` to create child
- 2 Child calls `exec()` to run a different program
- 3 Parent calls `wait()` to reap child

fork() + exec() Example

```
pid_t pid = fork();
if (pid == 0) {
    // Child: run "ls -l"
    execlp("ls", "ls", "-l", NULL);
    perror("exec failed");
    exit(1);
} else if (pid > 0) {
    wait(NULL);
    printf("Child finished\n");
}
```

Output:

```
-rw-r--r-- 1 user main.c
-rwxr-xr-x 1 user a.out
Child finished
```

*Child runs ls, parent prints after
wait()*

Thread Creation

- Threads are created within a process to execute tasks concurrently
- Uses `pthread_create()` (POSIX threads)
- Thread creation involves:
 - Allocating a stack for the thread
 - Initializing the thread control block (TCB)
 - Setting the thread's entry point (function to execute)
- Threads share code, data, heap; each has own stack

pthread_create() Example

```
#include <pthread.h>
#include <stdio.h>

void* thread_func(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d running\n", id);
    return NULL;
}

int main() {
    pthread_t tid;
    int arg = 1;
    pthread_create(&tid, NULL,
                  thread_func, &arg);
    pthread_join(tid, NULL); // wait for thread to finish
    printf("Thread joined\n");
    return 0;
}
```

Output:

```
Thread 1 running
Thread joined
```

Compile with:

```
gcc -pthread prog.c
```

Multi-threaded Calculation Example

```
#define NUM_THREADS 4
int partial_sum[NUM_THREADS]; // Each thread writes to its own index

void* compute_sum(void* arg) {
    int id = *(int*)arg;
    partial_sum[id] = 0;
    // Thread 0: 0-24, Thread 1: 25-49, Thread 2: 50-74, Thread 3: 75-99
    for (int i = id * 25; i < (id + 1) * 25; i++)
        partial_sum[id] += i;
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];
    for (int i = 0; i < NUM_THREADS; i++) {
        ids[i] = i;
        pthread_create(&threads[i], NULL, compute_sum, &ids[i]);
    }
    int total = 0;
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
        total += partial_sum[i];
    }
    printf("Sum 0-99: %d\n", total); // Arithmetic series:  $n(n+1)/2 = 4950$ 
}
```

Thread Join and Detach

Joinable Thread (default)

- Resources held until `pthread_join()`
- Calling thread **blocks** until target finishes
- Can retrieve return value
- Must join to avoid resource leak

```
pthread_join(tid, &retval);
```

Detached Thread

- Resources freed **automatically** on termination
- Cannot join or get return value
- "Fire and forget"

```
pthread_detach(tid);
```

Thread vs Process Termination

Process Termination

- `exit(status)` – terminates process
- return from `main()`
- Receiving fatal signal (e.g., `SIGKILL`)
- **All threads die** when process terminates
- OS cleans up: memory, file descriptors, locks

Thread Termination

- `pthread_exit(retval)`
- return from thread function
- `pthread_cancel(tid)`
- **Other threads continue** running
- Resources: freed by join or if detached

Zombie Processes

A process enters the **zombie (defunct)** state after finishing execution but before being reaped by its parent.

Mechanism:

- 1 **Termination:** Process calls `exit()`, OS frees memory and files
- 2 **Retention:** Kernel keeps entry in process table (PID + exit status)
- 3 **Reaping:** Parent calls `wait()/waitpid()`, zombie is removed

Risk – PID Exhaustion:

- Zombies consume no CPU/RAM but hold onto their PID
- Mass accumulation → process table full → no new processes
- Visible in `ps` with status **Z** or `<defunct>`

Orphans vs Zombies

Orphan Process

- A **living** process whose parent has **died**
- Adopted by `init` (PID 1)

Zombie Process

- A **dead** process waiting to be **reaped**
- Parent hasn't called `wait()`

The Role of `init` (PID 1):

`init` is the first process started by the kernel at boot – ancestor of all processes.

- Adopts orphans immediately when their parent dies
- If adopted child is a zombie → reaps it immediately
- If adopted child is running → waits and reaps upon termination

Exercise – `exit()` vs `pthread_exit()`

Key concept: `exit()` terminates the **entire process**. `pthread_exit()` terminates only the calling thread.

```
void* thread_A(void* arg) {
    printf("A1\n");
    printf("A2\n");
    return NULL;
}

void* thread_B(void* arg) {
    printf("B1\n");
    exit(0);           // intended to exit just this thread
    printf("B2\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_A, NULL);
    pthread_create(&t2, NULL, thread_B, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Done\n");
    return 0;
}
```


Solution – `exit()` vs `pthread_exit()`

Possible outputs include:

- B1
- A1, B1
- B1, A1
- A1, A2, B1
- A1, B1, A2
- B1, A1, A2
- ... and other interleavings

Never printed:

- B2 – after `exit()`, unreachable
- Done – process dies before main continues

Why?

`exit(0)` terminates the **entire process**, killing all threads immediately.

Fix:

```
void* thread_B(void* arg) {  
    printf("B1\n");  
    printf("B2\n");  
    pthread_exit(NULL);  
    // or: return NULL;  
}
```

Now thread B exits properly;
thread A and main continue.
Output includes Done.

Exercise – Fork Inside Thread

Key concept: A thread (not only the one running main) can call `fork()`. Only that thread survives in child.

```
void* thread_func(void* arg) {
    printf("T1\n");
    if (fork() == 0) {
        printf("T-child\n");
        exit(0);
    }
    printf("T2\n");
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, thread_func, NULL);
    printf("M\n");
    pthread_join(t, NULL);
    return 0;
}
```

Question: List all possible output orderings.

Exercise – Solution: Fork Inside Thread

Possible outputs (M can appear anywhere before join completes):

M
T1
T-child
T2

T1
M
T2
T-child

T1
T-child
T2
M

Constraints:

- T1 always before T2 and T-child (sequential in thread)
- T2 always after T1 (parent continues after fork)
- T-child always after T1 (child starts after fork)
- M can interleave with thread execution

Key insight: Child process has only the forking thread – main thread doesn't exist in child!

Exercise – Multiple Threads + Fork

```
void* print_A(void* arg) { printf("A\n"); return NULL; }
void* print_B(void* arg) { printf("B\n"); return NULL; }

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, print_A, NULL);
    if (fork() == 0) {
        pthread_create(&t2, NULL, print_B, NULL);
        printf("C\n");
        pthread_join(t2, NULL);
        exit(0);
    }
    printf("P\n");
    pthread_join(t1, NULL);
    return 0;
}
```

Questions:

- 1 Does the child process have thread t1?
- 2 What are the possible outputs?

Exercise – Solution: Multiple Threads + Fork

Answer 1: No! Child only has the "main" thread. Thread t1 exists only in parent.

Answer 2: Possible outputs (many permutations):

A	P	C
P	A	B
C	B	A
B	C	P

Constraints:

- A only in parent (t1 doesn't survive fork to child)
- B only in child (t2 created only in child)
- P only in parent, C only in child
- Parent: A and P can interleave
- Child: B and C can interleave
- Parent and Child run concurrently

Process and Thread Communication

Inter- and Intra-Process Communication

⚡ Communication Models

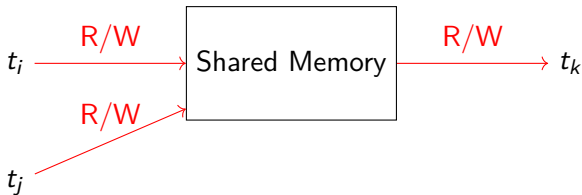
How do processes and threads exchange data?

- **Intra-process:** Threads within same process (shared memory by default)
- **Inter-process:** Between different processes (requires explicit mechanisms)
- Two main models:
 - **Shared Memory Model**
 - **Message Passing Model**

Shared Memory Model

How it Works:

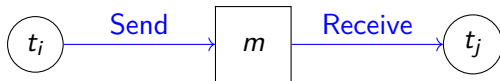
- Multiple processes or threads share a common memory space.
- They can read/write to the shared memory.
- Requires synchronization mechanisms (e.g., locks, semaphores).



Message Passing Model

How it Works:

- Processes send messages to communicate.
- No shared memory is required.
- Concept is also used in distributed systems and computer networks.



Only IPC model used in computer networks.

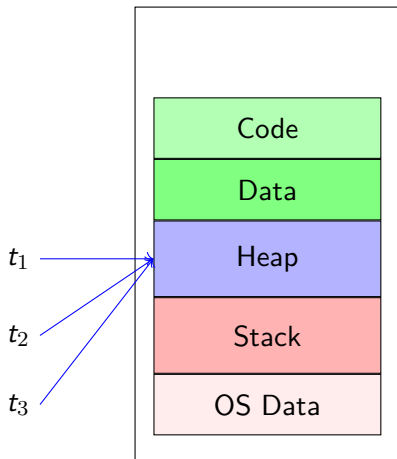
Shared Memory Details

Memory Sharing Mechanisms

- Threads in the same process share memory by default
- Different processes can share memory through **memory-mapped files**
- Used for fast communication, but requires synchronization

Shared Memory Visualization: Threads Sharing Memory

Process P_1 (Threads share memory)



The Synchronization Problem

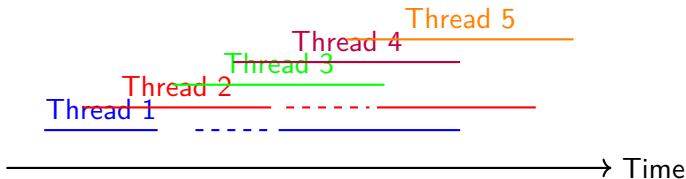
Threads Timing: Unpredictability

⚡ Thread Execution is Non-Deterministic

Unless we explicitly synchronize threads, they can:

- Run in **any order**
- Start/Stop at **any time**
- Run/Stop for **arbitrary amount of time**

Visualization: Thread Execution Timing



Threads execute at different times, may pause, resume, or stop unexpectedly.

Why Synchronization is Important?

Issues with Uncontrolled Thread Execution

- **Race Conditions:** Multiple threads modify shared data unpredictably
- **Deadlocks:** Threads waiting indefinitely for each other
- **Starvation:** Some threads may never get CPU time

Thread Synchronization Mechanisms

Synchronization Primitives

- **Mutex (Mutual Exclusion):** Ensures only one thread accesses a resource at a time
- **Semaphores:** Controls access to shared resources using counters
- **Locks:** Blocks other threads until the resource is free
- **Barriers:** Forces threads to wait at a specific point before proceeding
- **Atomic Operations:** Ensures single-step execution of critical updates

Low-Level Synchronization

What is an Atomic Operation?

⚙ Atomic Operation Definition

An operation that completes in a single step relative to other threads.

- Cannot be interrupted or left in an incomplete state (take one CPU cycle)
- Ensures consistency in concurrent environments
- Implemented at the hardware level using special CPU instructions
- Basis for implementing locks, semaphores, and other synchronization primitives.

Non-Atomic vs Atomic: Pseudo-Assembly

Non-Atomic (3 steps):

```
LW    r1, counter    ; Load
ADD   r1, r1, 1
SW    r1, counter    ; Store
```

Race Condition

Thread can be interrupted between any of these steps → lost updates

Atomic (1 step):

```
ATOMIC_ADD counter, 1
```

Hardware Support

Single indivisible instruction – cannot be interrupted

Key: LW = Load Word, SW = Store Word

Bank Account Interleaving

What are the possible final balances?

Initial balance: **\$100**. Two threads run concurrently (no synchronization):

Thread 1 – Deposit \$100:

1a: LW r1, balance
1b: ADD r1, r1, 100
1c: SW r1, balance

Thread 2 – Withdraw \$50:

2a: LW r2, balance
2b: SUB r2, r2, 50
2c: SW r2, balance

Expected: $\$100 + \$100 - \$50 = \150

Question: What values are actually possible? List all interleavings.

Bank Account – Solution

Possible final balances: \$50, \$150, \$200

Interleaving	Result
1a,1b,1c,2a,2b,2c	\$150 (correct)
2a,2b,2c,1a,1b,1c	\$150 (correct)
1a,2a,1b,2b,1c,2c	\$50 (T2 overwrites T1)
1a,2a,1b,2b,2c,1c	\$200 (T1 overwrites T2)

Lost Update Problem

Both threads load the **same** initial value, then one overwrites the other's result.

Critical Section Requirements

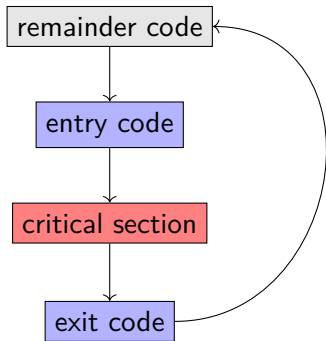
⚙️ Critical Section Structure

A section of code that accesses shared resources and must not be executed by more than one thread at a time.

✓ Requirements for Critical Sections

- **Mutual Exclusion:** No two processes are in their critical section at the same time
- **Deadlock-freedom:** If a process is trying to enter its critical section, then *some* process eventually enters a critical section
- **Starvation-freedom:** If a process is trying to enter its critical section, then *that* process must eventually enter its critical section

Critical Section Structure



- **Entry code:** Acquires lock, ensures only one thread enters
- **Critical section:** Access shared resources
- **Exit code:** Releases lock, allows others to enter

Busy Wait / Spinlock / Wait

Busy Wait – repeatedly *polls* for a condition until met.

Spinlock – busy wait where CPU "spins" until lock is freed.

Wait/Block – thread sleeps until another thread wakes it (in practice we give an "hint" to the scheduler).

Busy Wait:

```
while(locked) {  
    // spin - wastes CPU  
}
```

Wait/Block:

```
while(locked) {  
    sleep(1); // yield CPU  
}
```

	CPU	Overhead
Busy wait	Wastes	No context switch
Block	Efficient	May context switch

stdatomic.h – Overview

`#include <stdatomic.h>`

(C11, compile with `-std=c11`)

Types: `atomic_int`, `atomic_bool`, `atomic_flag`

Function	Returns	Description
<code>atomic_load(&x)</code>	value	Read atomically
<code>atomic_store(&x, val)</code>	void	Write atomically
<code>atomic_fetch_add(&x, val)</code>	old value	Add, return old
<code>atomic_fetch_sub(&x, val)</code>	old value	Subtract, return old
<code>atomic_exchange(&x, val)</code>	old value	Swap, return old
<code>atomic_compare_exchange_strong</code>	bool	CAS operation
<code>atomic_flag_test_and_set(&f)</code>	old (bool)	Set flag, return old
<code>atomic_flag_clear(&f)</code>	void	Clear flag

Atomic Operations – Basic Examples

```
#include <stdatomic.h>

atomic_int counter = 0;

// Atomic read
int val = atomic_load(&counter);           // val = 0

// Atomic write
atomic_store(&counter, 10);                 // counter = 10

// Atomic add (returns OLD value)
int old = atomic_fetch_add(&counter, 5);   // old = 10,
//counter = 15

// Atomic subtract (returns OLD value)
old = atomic_fetch_sub(&counter, 3);       // old = 15,
//counter = 12
```

Atomic Operations – Exchange

```
atomic_int x = 5;
```

```
// Exchange: swap and return old value
```

```
int old = atomic_exchange(&x, 20); // old = 5, x = 20
```

Atomic Operations – CAS

```
atomic_int x = 5;
int expected = 5;
int desired = 10;

// If x == expected: set x = desired, return true
// Else: set expected = x, return false
if (atomic_compare_exchange_strong(&x, &expected, desired))
    // Success: x was 5, now x = 10
} else {
    // Failed: x != 5, expected now contains
    // actual value of x
}
```

atomic_flag – Spinlock Primitive

- Guaranteed lock-free boolean (only atomic type with this guarantee)
- Two operations: `test_and_set` and `clear`

Spinlock Example:

```
#include <stdatomic.h>
```

```
atomic_flag lock = ATOMIC_FLAG_INIT;  // Initialize
```

```
void acquire() {  
    while (atomic_flag_test_and_set(&lock)) // Returns old  
        ; // Spin until we get false (was unlocked)  
}
```

```
void release() {  
    atomic_flag_clear(&lock);  
}
```

Exercise – Bank Account: Safe Withdrawal

Recall: The bank account problem showed race conditions with concurrent deposits/withdrawals.

Extension: Now we add a balance check before withdrawal. Is this safe?

```
int balance = 100;

bool withdraw(int amount) {
    if (balance >= amount) {    // CHECK
        balance -= amount;      // ACT
        return true;
    }
    return false;
}

// Thread 1: withdraw(80)
// Thread 2: withdraw(80)
```

Questions:

- 1 Can both withdrawals succeed? What's the final balance?
- 2 Fix using `atomic_compare_exchange_strong` (CAS loop)

Solution – Bank Account: Safe Withdrawal

The Bug:

- 1 T1: checks $100 \geq 80 \rightarrow \text{true}$
- 2 T2: checks $100 \geq 80 \rightarrow \text{true}$
- 3 T1: $\text{balance} = 100 - 80 = 20$
- 4 T2: $\text{balance} = 20 - 80 = -60$

Both succeed, balance goes **negative!**

Problem: Check-then-act is not atomic. Another thread can modify balance between check and act.

Fix with CAS:

```
#include <stdatomic.h>
atomic_int balance = 100;

bool withdraw(int amount) {
    int old = atomic_load(&balance);
    while (old >= amount) {
        // Try to update balance atomically
        if (atomic_compare_exchange_strong(
            &balance, &old, old - amount))
            return true;
        // CAS failed: Old automatically
        // updated to current balance
        // (CAS semantics), retry loop
    }
    return false;
}
```

Compare uses locals (safe). On CAS fail, old auto-updated to current value.

The Optimization Problem

⚠ Why Optimizations Can Break Concurrent Code

Compilers and CPUs aggressively optimize for performance, but these optimizations assume single-threaded execution.

Compiler Optimizations:

- Caches variables in registers
- Removes “redundant” memory reads
- Assumes variables don't change externally

CPU Optimizations:

- Reorders instructions for pipeline efficiency
- Writes may not be immediately visible
- Out-of-order execution

volatile – Preventing Compiler Caching

❌ Problem:

```
int flag = 0;

void wait_for_flag() {
    while (!flag);
    // Compiler may optimize to:
    // if (!flag) while(1);
    // (caches flag in register)
}

void signal() {
    flag = 1; // Never seen!
}
```

✅ Solution:

```
volatile int flag = 0;

void wait_for_flag() {
    while (!flag);
    // Compiler MUST read flag
    // from memory each iteration
}

void signal() {
    flag = 1; // Now visible!
}
```

⚠ Important

volatile does **NOT** guarantee atomicity or prevent CPU reordering!

Memory Fences – Preventing Instruction Reordering

❌ Problem:

```
int data = 0;
int flag = 0;

void writer() {
    data = 42;
    flag = 1;
    // CPU may reorder:
    // flag = 1 BEFORE data = 42!
}

void reader() {
    while (!flag);
    print(data); // May print 0!
}
```

✅ Solution:

```
int data = 0;
atomic_int flag = 0;

void writer() {
    data = 42;
    atomic_thread_fence(
        memory_order_release);
    atomic_store(&flag, 1);
}

void reader() {
    while (!atomic_load(&flag));
    atomic_thread_fence(
        memory_order_acquire);
    print(data); // Always 42!
}
```

Memory Fence Semantics

Release (Writer Side)

All writes **before** the fence are guaranteed to be visible **before** any write after the fence.

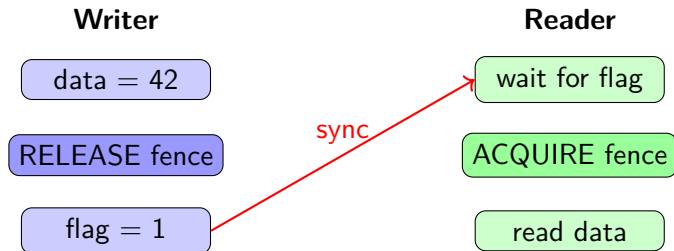
“Release my changes to other threads”

Acquire (Reader Side)

All reads **after** the fence see all writes that happened **before** a matching release fence.

“Acquire changes from other threads”

Memory Fence Semantics Example



Exercise – Does This Code Work?

```
typedef struct { int value; int id; } Object;

Object* obj = NULL;
int initialized = 0;

void init_thread() {
    obj = malloc(sizeof(Object));
    obj->value = 42;
    obj->id = 1;
    initialized = 1;
}

void use_thread() {
    while (!initialized);           // wait for init
    printf("value=%d\n", obj->value);
}
```

Exercise – Solution

Answer: No! CPU may reorder `initialized = 1` before `obj` is fully set up, **Possible bugs:** `obj` is NULL, or `obj->value` is garbage.

Fixed code:

```
Object* obj = NULL;
atomic_int initialized = 0;

void init_thread() {
    obj = malloc(sizeof(Object));
    obj->value = 42;
    obj->id = 1;
    atomic_thread_fence(memory_order_release); // ensure writes complete
    atomic_store(&initialized, 1);
}

void use_thread() {
    while (!atomic_load(&initialized));
    atomic_thread_fence(memory_order_acquire); // ensure we writes
    printf("value=%d\n", obj->value);         // always 42!
}
```

Synchronization Primitives

Semaphores in Operating Systems

Semaphores Overview

- Semaphores control access to shared resources in concurrent systems
- Prevents race conditions and ensures safe access

Binary Semaphore

Only two states (locked/unlocked). Also known as a **Mutex**.

Counting Semaphore

Allows multiple accesses up to a specified limit.

How Semaphores Work?

Operations:

- `wait()` (P operation): Decrements semaphore; blocks if value becomes negative.
- `signal()` (V operation): Increments semaphore; unblocks waiting process.

Bounded Semaphore

Has a maximum value limit.

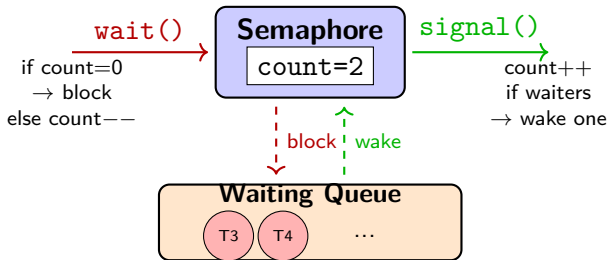
- `signal()` fails/blocks if at max
- Prevents overflow
- Example: limited resource pool

Unbounded Semaphore

No maximum value limit.

- `signal()` always succeeds
- Can grow indefinitely
- Example: counting events

Semaphore Mechanics



count > 0: Resources available, wait() decrements and proceeds.

count = 0: No resources, wait() blocks thread in queue.

Semaphore Example in C

```
sem_t sem;
int shared_resource = 0;

void* worker(void* arg) {
    long id = (long)arg;
    sem_wait(&sem); // P: acquire
    printf("T%d: enter (res=%d)\n", id, shared_resource);
    shared_resource++;
    sleep(1);
    printf("T%d: leave (res=%d)\n", id, shared_resource);
    sem_post(&sem); // V: release
    return NULL;
}

int main() {
    pthread_t t[3];
    // sem_init(sem, pshared, value): 0=local, 1=binary
    sem_init(&sem, 0, 1);
    for (long i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, worker, (void*)i);
    for (int i = 0; i < 3; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&sem);
}
```

Output:

```
T0: enter (res=0)
T0: leave (res=1)
T1: enter (res=1)
T1: leave (res=2)
T2: enter (res=2)
T2: leave (res=3)
```

Key points:

- Only one thread in critical section at a time
- `shared_resource` increments safely
- Threads execute sequentially

Exercise – Connection Pool Synchronization

Problem: A server has a pool of **N=3** database connections. Multiple threads handle requests, each needing one connection.

```
#define MAX_CONNECTIONS 3
int active_connections = 0;

void handle_request(int thread_id) {
    // Acquire connection
    active_connections++;
    printf("T%d: using connection (%d active)\n",
           thread_id, active_connections);
    sleep(1); // Use connection
    // Release connection
    active_connections--;
}
```

Questions:

- 1 What's wrong with this code?
- 2 How would you fix it with a mutex? What's the limitation?
- 3 How would an unbounded semaphore help? What's still wrong?

Solution – Connection Pool: Attempt 1 (Mutex)

```
pthread_mutex_t lock;
int active_connections = 0;

void handle_request(int id) {
    pthread_mutex_lock(&lock);
    active_connections++;
    printf("T%d: using (%d active)\n",
        id, active_connections);
    sleep(1);
    active_connections--;
    pthread_mutex_unlock(&lock);
}
```

Problem:

- Only **1 thread** can use a connection at a time
- We have 3 connections but only use 1!
- Throughput: 1 request/second

Mutex is too restrictive – we want up to N concurrent accesses, not just 1.

Solution – Connection Pool: Attempt 2 (Unbounded Semaphore)

```
sem_t sem;
// Initialize with 3
sem_init(&sem, 0, 3);

void handle_request(int id) {
    sem_wait(&sem);
    // ... use connection ...
    sem_post(&sem);
}
// But what if buggy code does
// extra sem_post()?
```

Problem:

- Semaphore starts at 3 (✓)
- But `sem_post()` can increment **beyond 3**
- Bug or malicious code could call `extra post()`
- Result: More than 3 concurrent connections!

Unbounded semaphore – value can grow past the limit.

Solution – Connection Pool: Bounded Semaphore

Conceptual bounded semaphore:

```
typedef struct {
    sem_t sem;
    sem_t limit; // Prevents overflow
    int max;
} bounded_sem_t;

void bounded_wait(bounded_sem_t* bs) {
    sem_wait(&bs->sem);
}

void bounded_post(bounded_sem_t* bs) {
    // Only post if below max
    if (current < bs->max)
        sem_post(&bs->sem);
}
```

Correct behavior:

- Max 3 concurrent connections
- wait() blocks at 0
- post() **cannot exceed** max
- Extra post() calls ignored or error

Bounded semaphore – enforces both lower (0) and upper (N) bounds.

Note: POSIX sem_t is unbounded. Some systems provide bounded variants.

Read-Write Locks



Read-Write Locks

Allows multiple threads to read concurrently while ensuring exclusive write access. Efficient for read-heavy workloads.

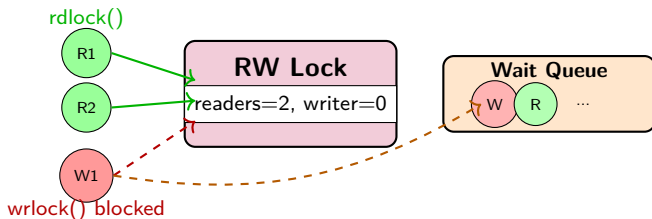
Read Lock (rdlock)

- Multiple readers simultaneously
- Blocks only if writer holds lock
- `pthread_rwlock_rdlock()`

Write Lock (wrlock)

- Only one writer at a time
- Blocks readers and writers
- `pthread_rwlock_wrlock()`

Read-Write Lock Mechanics



State	Behavior
readers=0, writer=0	rdlock() and wrlock() succeed
readers>0, writer=0	rdlock() succeeds, wrlock() blocks
writer=1	Both rdlock() and wrlock() block

Queue Priority Policies: *Reader priority* (readers first, may starve writers), *Writer priority* (writers first, may starve readers), *Fair* (FIFO order).

Read-Write Lock Example in C

```
pthread_rwlock_t rwlock;
int data = 0;

void* reader(void* arg) {
    long id = (long)arg;
    pthread_rwlock_rdlock(&rwlock);
    printf("R%d: read data=%d\n", id, data);
    sleep(1);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

void* writer(void* arg) {
    long id = (long)arg;
    pthread_rwlock_wrlock(&rwlock);
    data++;
    printf("W%d: wrote data=%d\n", id, data);
    sleep(1);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

int main() {
    pthread_t r1, r2, w1;
    pthread_rwlock_init(&rwlock, NULL);
    pthread_create(&r1, NULL, reader, (void*)1);
    pthread_create(&r2, NULL, reader, (void*)2);
    pthread_create(&w1, NULL, writer, (void*)1);
    .....
```

Possible output:

R1: read data=0
R2: read data=0
W1: wrote data=1

Key points:

- R1 and R2 can read concurrently
- W1 waits until all readers finish
- Writer has exclusive access

Where Are Read-Write Locks Used?

- Databases: Allow multiple read queries while preventing write conflicts.
- File Systems: Multiple processes can read a file, but writing is exclusive.
- Caching Mechanisms: Multiple threads can read cached data, but updating requires exclusive access.

Exercise – Writer Starvation

Scenario: A cache system with reader-priority RW lock. Readers arrive continuously.

```
pthread_rwlock_t rwlock;
int cache_data = 0;

void* reader(void* arg) {
    while (1) {
        pthread_rwlock_rdlock(&rwlock);
        printf("Read: %d\n", cache_data);
        usleep(100000); // 100ms hold time
        pthread_rwlock_unlock(&rwlock);
        usleep(50000); // 50ms between reads
    }
}

void* writer(void* arg) {
    pthread_rwlock_wrlock(&rwlock);
    cache_data++;
    printf("Write: %d\n", cache_data);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

int main() {
    // Start 5 readers, then 1 writer
    for (int i = 0; i < 5; i++)
        pthread_create(&readers[i], NULL, reader, NULL);
    sleep(1);
    pthread_create(&writer_thread, NULL, writer, NULL);
}
```

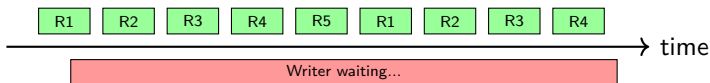
Writer Starvation – The Problem

What happens:

- 1 5 readers continuously acquire rdlock
- 2 Short gap between unlock and next lock
- 3 Always ≥ 1 active reader
- 4 Writer waits for readers=0
- 5 **Writer starves indefinitely!**

Timeline:

- > R1,R2,R3 hold lock
- > R1 releases, R4 acquires
- > R2 releases, R5 acquires
- > R3 releases, R1 acquires
- > Writer never gets a chance...



Writer Starvation – Solutions

Actual Solutions:

- 1 **Fair (FIFO) lock:** Requests served in arrival order – no starvation
- 2 **Application-level backoff:** All entities (readers **and** writers) back off periodically to avoid collision and give others a chance

Shifts problem:

Writer-priority lock

Blocks new readers when writer waits.

But: Continuous writers → **readers starve instead!**

Just moves starvation, doesn't solve it.

NOT a solution:

Timeout (timedwrlock)

Releases writer from blocking.

But: Writer **still doesn't write** – just gives up!

Operation fails, starvation not solved.

Exercise – Lock Upgrade Deadlock

Scenario: Thread reads data, decides it needs to modify, tries to "upgrade" to write lock.

```
pthread_rwlock_t rwlock;
int data = 0;

void* read_then_write(void* arg) {
    // First, read the data
    pthread_rwlock_rdlock(&rwlock);
    int val = data;
    printf("Read: %d\n", val);

    if (val < 10) {
        // Need to update -- try to get write lock
        pthread_rwlock_wrlock(&rwlock); // ???
        data = val + 1;
        printf("Wrote: %d\n", data);
        pthread_rwlock_unlock(&rwlock);
    }
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

int main() {
    pthread_create(&t1, NULL, read_then_write, NULL);
    pthread_create(&t2, NULL, read_then_write, NULL);
    ...
}
```

Solution – Lock Upgrade Deadlock

The Deadlock:

- 1 T1: acquires rdlock
- 2 T2: acquires rdlock (allowed!)
- 3 T1: tries wrlock → **blocks** (T2 holds rdlock)
- 4 T2: tries wrlock → **blocks** (T1 holds rdlock)
- 5 **Both wait forever!**

Key insight:

Cannot upgrade rdlock → wrlock while holding rdlock. Must release first, but then state may change.

Fix – Release and Re-acquire:

```
void* read_then_write(void* arg) {
    pthread_rwlock_rdlock(&rwlock);
    int val = data;
    pthread_rwlock_unlock(&rwlock);
    // Lock released!

    if (val < 10) {
        pthread_rwlock_wrlock(&rwlock);
        // Re-check: data may have changed!
        if (data < 10) {
            data++;
        }
        pthread_rwlock_unlock(&rwlock);
    }
    return NULL;
}
```

Note: Must re-check condition after acquiring wrlock – another thread may have modified data.

Alternative Solution – Use Mutex Instead

Simple fix: Replace rwlock with mutex

```
pthread_mutex_t lock;
int data = 0;

void* read_then_write(void* arg) {
    pthread_mutex_lock(&lock);
    int val = data;

    if (val < 10) {
        data = val + 1;
        printf("Wrote: %d\n", data);
    }
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

No upgrade needed – we already have exclusive access. No deadlock possible.

Mutex vs RWLock – Trade-offs

Mutex Advantages

- No upgrade deadlock
- Simpler code
- No re-check needed
- Lower overhead

Mutex Disadvantage

No concurrent reads!

Only 1 thread at a time, even for read-only operations.

When to use Mutex:

- Read-modify-write is common
- Critical section is short
- Simplicity over performance

When to use RWLock:

- Read-heavy workload
- Reads are long operations
- Writes are rare
- Need maximum read throughput

Condition Variables

Condition Variables

Allows threads to wait for a specific condition efficiently. Used with a mutex to avoid busy-waiting.

How it works:

- 1 Lock mutex, check condition
- 2 If false: `wait()` releases mutex and blocks
- 3 Another thread signals the condition
- 4 Waiting thread wakes and reacquires mutex

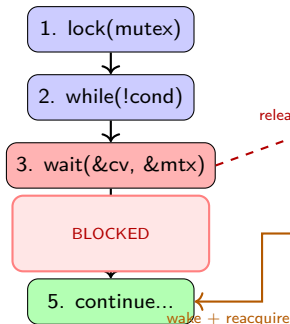
Key functions:

- > `pthread_cond_wait(&cv, &mtx)`
- > `pthread_cond_signal(&cv)`
- > `pthread_cond_broadcast(&cv)`

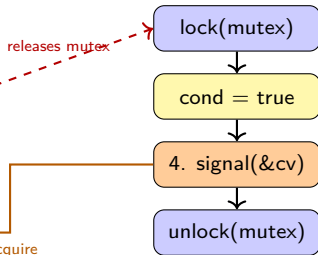
Note: Always check condition in a `while` loop (spurious wakeups).

Condition Variable Mechanics

Waiter Thread



Signaler Thread



Key: wait() atomically releases mutex and blocks. On wakeup, reacquires mutex before returning.

Condition Variable Example in C

```
pthread_mutex_t mtx;
pthread_cond_t cv;
int data_ready = 0;

void* waiter(void* arg) {
    long id = (long)arg;
    pthread_mutex_lock(&mtx);
    while (!data_ready) {           // check condition
        printf("W%d: waiting...\n", id);
        pthread_cond_wait(&cv, &mtx); // block + release
    }
    printf("W%d: got signal, data_ready=%d\n", id, data_ready);
    pthread_mutex_unlock(&mtx);
    return NULL;
}

void* signaler(void* arg) {
    sleep(1); // simulate work
    pthread_mutex_lock(&mtx);
    data_ready = 1;
    printf("S: signaling...\n");
    pthread_cond_signal(&cv); // wake one waiter
    pthread_mutex_unlock(&mtx);
    return NULL;
}

int main() {
    // init mutex, condition variable and run the threads
}
```

Output:

```
W1: waiting...
S: signaling...
W1: got signal, data_ready=1
```

Key points:

- wait(): release → block → reacquire → return
- signal(): wake one
- broadcast(): wake all (compete for mutex one-by-one)
- while loop: re-check condition after wake

Exercise – Two Conditions, One CV

Scenario: Counter with incrementing and decrementing threads sharing ONE condition variable.

```
pthread_cond_t cv; // ONE cv for both conditions!
pthread_mutex_t mtx;
int count = 0;
#define MAX 5

void* increment(void* arg) {
    pthread_mutex_lock(&mtx);
    while (count == MAX) // wait until not full
        pthread_cond_wait(&cv, &mtx);
    count++;
    printf("Incremented: count=%d\n", count);
    pthread_cond_signal(&cv); // signal ONE waiter
    pthread_mutex_unlock(&mtx);
    return NULL;
}

void* decrement(void* arg) {
    pthread_mutex_lock(&mtx);
    while (count == 0) // wait until not empty
        pthread_cond_wait(&cv, &mtx);
    count--;
    printf("Decrement: count=%d\n", count);
    pthread_cond_signal(&cv); // signal ONE waiter
    pthread_mutex_unlock(&mtx);
    return NULL;
}
```

Solution – Two Conditions, One CV

The Deadlock Scenario:

- 1 count=5 (max)
- 2 I1, I2 wait on CV (count at Max)
- 3 D1 decrements, signals CV
- 4 **I1 wakes**
- 5 I1 increments, count at max
- 6 I1 signals → **I2 wakes**
- 7 I2 sees count at max, waits again
- 8 D2 waiting, never signaled
- 9 **Deadlock!**

Problem:

Signal wakes *any* waiter, not necessarily one whose condition is true.

Solutions:

- 1 **Use broadcast():** Wake all, let them re-check
- 2 **Separate CVs:** not_max and not_zero (proper design)

Rule: Use separate CVs for different conditions, or use broadcast().

Exercise – Resource Pool with Cancellation

Scenario: Thread pool requests network connections.

```
pthread_cond_t cv;
pthread_mutex_t mtx;
int available_connections = 0;

typedef struct { int id; int cancelled; } Request;

void* worker(void* arg) {
    Request* req = (Request*)arg;
    pthread_mutex_lock(&mtx);
    while (available_connections == 0 && !req->cancelled)
        pthread_cond_wait(&cv, &mtx);

    if (req->cancelled) {
        printf("W%d: request cancelled, giving up\n", req->id);
        pthread_mutex_unlock(&mtx);
        return NULL; // Don't use connection
    }
    available_connections--;
    printf("W%d: got connection\n", req->id);
    pthread_mutex_unlock(&mtx);
    return NULL;
}

void release_connection() {
    pthread_mutex_lock(&mtx);
    available_connections++;
    pthread_cond_signal(&cv); // Wake ONE waiter
    pthread_mutex_unlock(&mtx);
}
```


Solution – Resource Pool with Cancellation

The Starvation Scenario:

- 1 W1, W2, W3 wait for connection
- 2 W1's request gets cancelled
- 3 Connection released, `signal()` called
- 4 **W1 wakes** (unlucky choice)
- 5 W1 sees cancelled, exits without using connection
- 6 W1 does **not** re-signal!
- 7 Connection available but W2, W3 still blocked
- 8 **Resource wasted, others starve**

Problem:

Woken thread doesn't consume resource but doesn't notify others.

Solutions:

- 1 **Use `broadcast()`:** All waiters check their status
- 2 **Re-signal on exit:**

```
if (req->cancelled) {  
    // Pass the signal to others!  
    pthread_cond_signal(&cv);  
    pthread_mutex_unlock(&mtx);  
    return NULL;  
}
```

Rule: If woken but not consuming, either broadcast or re-signal.

Classic Synchronization Problems

Producer-Consumer Problem

Producer-Consumer Problem

Classic synchronization problem: producers generate data, consumers retrieve data, both share a bounded buffer.

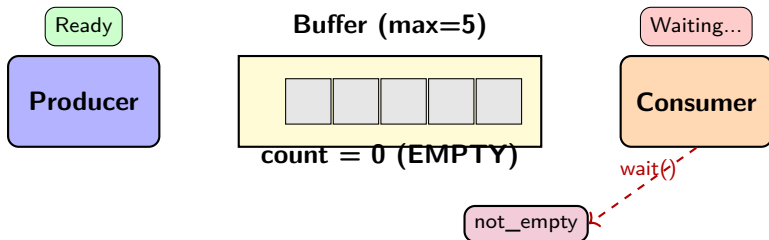
Challenges

- Buffer full → producer waits
- Buffer empty → consumer waits
- Concurrent access → race conditions

Solutions

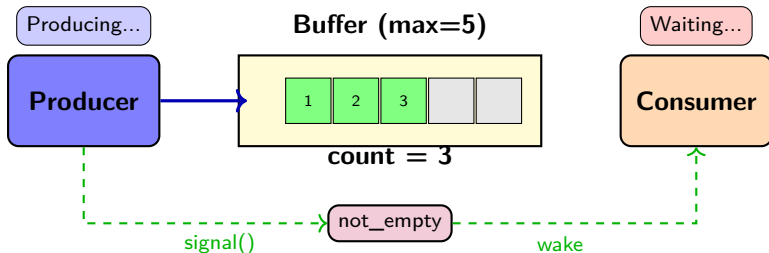
- Mutex for mutual exclusion
- Condition variable: `not_full`
- Condition variable: `not_empty`

Producer-Consumer: Step 1 – Initial State



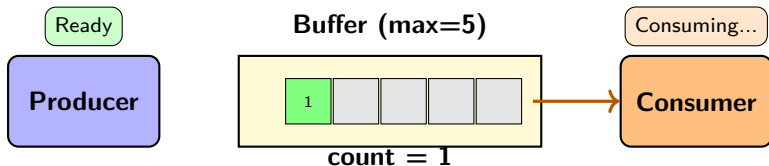
Buffer empty: Consumer calls `wait(not_empty)` and blocks.

Producer-Consumer: Step 2 – Producer Adds Data



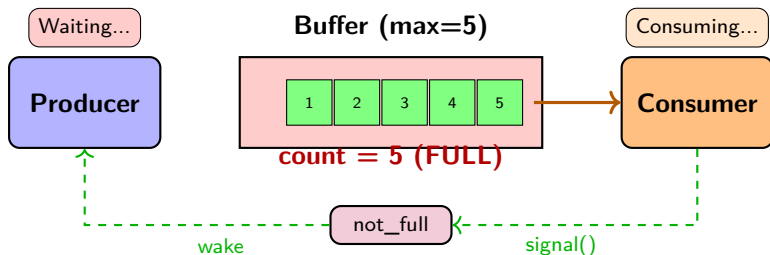
Producer: Adds items to buffer, then calls `signal(not_empty)` to wake consumer.

Producer-Consumer: Step 3 – Consumer Wakes & Consumes



Consumer: Wakes up, reacquires mutex, consumes items from buffer.

Producer-Consumer: Step 4 – Buffer Full



Buffer full: Producer calls `wait(not_full)` and blocks. Consumer signals `not_full`.

Producer-Consumer Example in C

```
#define MAX 5
int buffer[MAX], count = 0;
pthread_mutex_t mtx;
pthread_cond_t not_full, not_empty;

void* producer(void* arg) {
    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mtx);
        while (count == MAX)           // buffer full?
            pthread_cond_wait(&not_full, &mtx);
        buffer[count++] = i;
        printf("P: produced %d (count=%d)\n", i, count);
        pthread_cond_signal(&not_empty); // wake consumer
        pthread_mutex_unlock(&mtx);
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mtx);
        while (count == 0)             // buffer empty?
            pthread_cond_wait(&not_empty, &mtx);
        int val = buffer[--count];
        printf("C: consumed %d (count=%d)\n", val, count);
        pthread_cond_signal(&not_full); // wake producer
        pthread_mutex_unlock(&mtx);
    }
    return NULL;
}
```

Sample output:

```
P: produced 0 (count=1)
P: produced 1 (count=2)
C: consumed 1 (count=1)
C: consumed 0 (count=0)
P: produced 2 (count=1)
...
```

Key points:

- Two condition variables
- while loops for conditions
- Producer signals consumer
- Consumer signals producer

Producer-Consumer: Correctness Analysis

Race-Free

Why:

- All access to buffer, count protected by mutex
- wait() atomically releases mutex and blocks
- while loop re-checks condition after wake

Invariant:

Only one thread modifies shared state at any time.

Deadlock-Free

Why:

- Single mutex (no circular wait possible)
- wait() releases mutex while blocking
- No nested locking

Key insight:

Blocked thread always releases mutex, allowing others to proceed and signal.

Starvation-Free

Why:

- Separate CVs: not_full, not_empty
- Producer signals not_empty (wakes consumer)
- Consumer signals not_full (wakes producer)

Key insight:

Each signal targets the correct waiter.

See You Next Week!

Questions?

Recitation 3

Kernel and User Mode

Daniel Karalnik Liam Tal

Spring Semester 2026

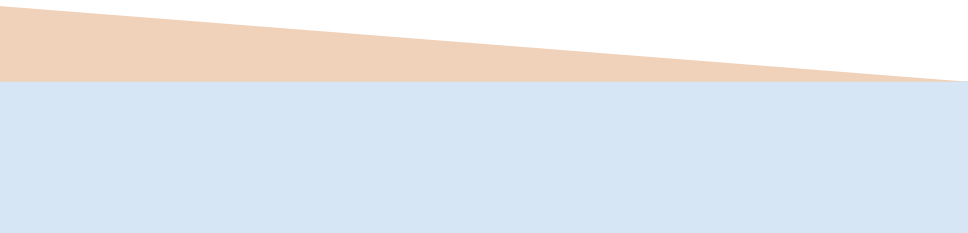


Table of Contents

- 1 OS Intro Continued
- 2 Kernel Mode and User Mode
- 3 System Calls
- 4 Interrupts and Exceptions
- 5 Signals
- 6 User Mode Threading

OS Intro Continued

Monolithic Operating Systems

Early System Design

In early monolithic operating systems, there was minimal separation between system and application code.

- Single process post-boot, often a terminal
- Early systems (e.g., Atari) ran directly on hardware
- Crashes in any program could crash the system
- Direct memory access increased vulnerability to bugs

Security Risk

Without isolation, any buggy or malicious code could access and corrupt any part of memory!

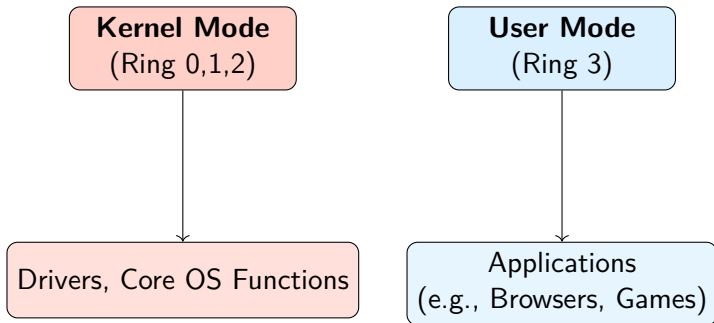
Kernel Mode and User Mode

Kernel Mode vs User Mode



Protection Rings

Modern CPUs provide hardware-enforced privilege levels to isolate critical system code from applications.



Kernel Mode vs User Mode - Comparison



Kernel Mode

- Can execute privileged instructions
- Privileged memory access
- Manages all resources



User Mode

- Limited hardware access
- Cannot execute privileged instructions
- Application memory only
- Must request resources via syscalls



Key Insight

The CPU hardware enforces these restrictions (with the CPL) – attempting privileged operations in user mode triggers an exception!

What is a Trap?

Definition

A **trap** is a **synchronous** event triggered by the currently executing instruction that transfers control to the kernel.

What is a Trap?

☰ Types of Traps

- **System calls** – intentional trap to request kernel service
- **Exceptions** – errors during execution (div by zero, page fault)
- **Breakpoints** – debugging (int 3)

⚡ Key Property

Synchronous means:

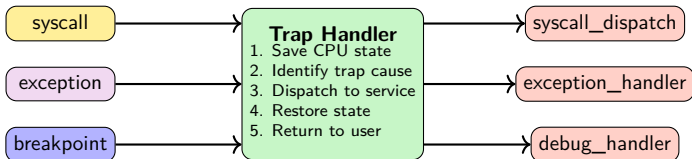
- Caused by the instruction itself
- Happens at a predictable point
- Same instruction = same trap

(Unlike interrupts, which are asynchronous)

What is a Trap Handler?

⚙️ Trap Handler

The **trap handler** is kernel code that executes when a trap occurs. It determines what caused the trap and dispatches to the appropriate service routine.

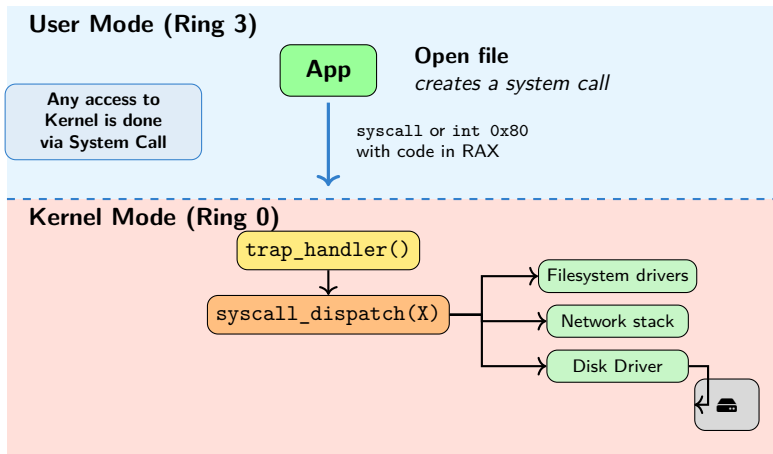


💡 Key Insight

The trap handler is the **entry point** into the kernel – the gatekeeper for all synchronous kernel requests.

System Calls

System Calls – Overview



System Calls – How It Works

Step-by-Step Flow

- 1 Application calls library function (e.g., `open()`)
- 2 Library places syscall number in RAX register
- 3 Executes `syscall` instruction (or `int 0x80`)
- 4 CPU finds entry point:
 - `int 0x80`: looks up **IDT** entry 128
 - `syscall`: reads **MSR_LSTAR** directly (faster)
- 5 CPU switches from Ring 3 to Ring 0, jumps to handler
- 6 Trap handler saves state, calls syscall dispatch
- 7 Syscall dispatch looks up handler in syscall table
- 8 Handler executes, result returned in RAX
- 9 CPU returns to Ring 3, execution continues

System Calls – How It Works

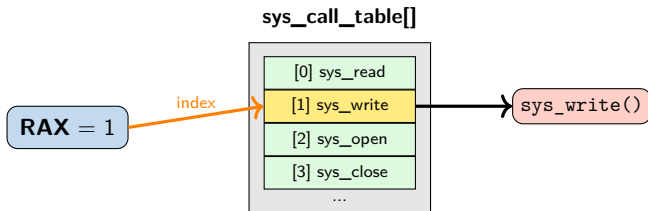
No Context Switch!

A system call is **not** a context switch – the same thread transitions from Ring 3 to Ring 0. The process remains the same; only the privilege level changes.

Syscall Dispatch and the Syscall Table

📍 Routing to the Right Handler

Syscall dispatch takes the syscall number (in RAX) and calls the corresponding kernel function from a **syscall table**.



Common syscalls: 0=read, 1=write, 2=open, 3=close,
39=getpid, 57=fork, 60=exit, 62=kill

Legacy Route: `int 0x80`

Interrupt-Based System Call

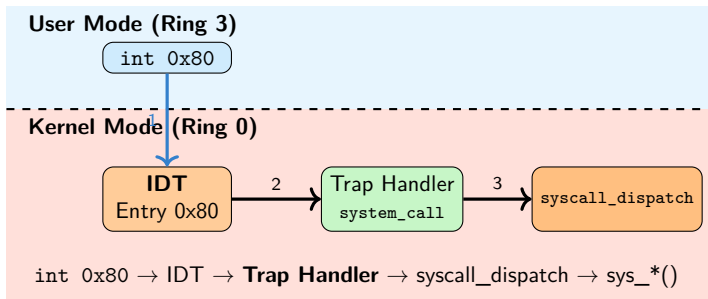
The legacy method uses a software interrupt – goes through the **general trap handler**.

- User program executes `int 0x80` instruction
- CPU looks up entry `0x80` in the **IDT** (Interrupt Descriptor Table)
- Jumps to the **general trap handler** (`system_call`)
- Trap handler identifies it as a syscall and calls `syscall_dispatch`
- Dispatch uses `RAX` to index into syscall table → `sys_*`()

Warning

Overhead: IDT memory lookup + general trap handler processing

Legacy Route: int 0x80 – Flow



Modern Route: `syscall` Instruction

⚡ Fast System Call Entry

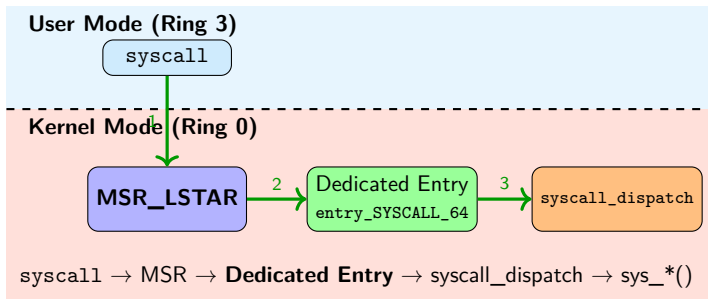
Modern x86-64 CPUs provide a dedicated `syscall` instruction that **bypasses both the IDT and the general trap handler.**

- User program executes `syscall` instruction
- CPU reads target address directly from **MSR_LSTAR** register
- Jumps to **dedicated syscall entry** (`entry_SYSCALL_64`)
- This entry point goes directly to `syscall_dispatch`
- Dispatch uses RAX to index into syscall table → `sys_*`()

💡 Key Insight

Key optimization: Skips IDT lookup AND general trap handler

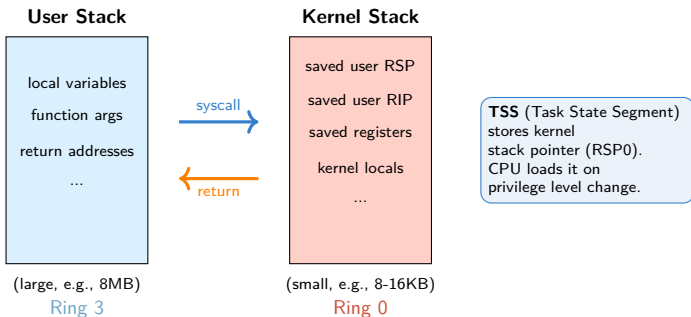
Modern Route: syscall – Flow



The Kernel Stack

Why a Separate Stack?

Each thread has **two stacks**: a user stack and a kernel stack. The kernel cannot trust the user stack!



Kernel Stack: Security and Design

⚠ Why Not Use User Stack?

- User controls their stack pointer
- Could point to invalid memory
- Could point to kernel memory!
- Stack overflow attacks
- Kernel must have reliable storage

✓ Kernel Stack Properties

- Fixed small size (8-16KB)
- One per thread
- In kernel memory (protected)
- Stores syscall/interrupt state
- Enables nested interrupts

System Call Examples

Common System Calls

Process:

- > fork()
- > exec()
- > exit()
- > getpid()
- > wait()

File System:

- > open()
- > close()
- > read()
- > write()
- > seek()
- > mkdir()

Network:

- > socket()
 - > bind()
 - > listen()
- ### Time:
- > time()
 - > settime()

Exercise 1: System Call Overhead

Problem

A program processes **1,200** data blocks in a tight loop. For each block:

- **User-mode computation:** 800 CPU cycles
- **System call (`write()`):**
 - User → kernel trap: 400 cycles
 - Kernel execution: 2,000 cycles
 - Kernel → user return: 300 cycles

Exercise: System Call Overhead - Questions



Calculate

- 1 Total CPU cycles in pure user-mode computation
- 2 Number of traps and total trap overhead
- 3 Total cycles executing `write()` in kernel
- 4 Grand total cycles and overhead percentage
- 5 Redo with batching (10 blocks per syscall)

Solution 1: Total User-Mode Compute

Given

Per block: 800 cycles | Blocks: 1,200

Solution

Total user-mode compute = $800 \times 1,200 = \mathbf{960,000}$ cycles

Solution 2: Mode-Switch Overhead

Given

Per syscall: User→kernel = 400 cycles, Kernel→user = 300 cycles

Solution

- Number of system calls: 1,200
- Overhead per syscall: $400 + 300 = 700$ cycles
- **Total trap overhead:**

$$700 \times 1,200 = \mathbf{840,000} \text{ cycles}$$

Solution 3: Kernel-Mode Work

Given

Kernel execution per `write()`: 2,000 cycles

Solution

Total kernel-mode work = $2,000 \times 1,200 = \mathbf{2,400,000}$ cycles

Solution 4: Grand Total and Overhead

✓ Solution

- User compute: 960,000 cycles
- Trap overhead: 840,000 cycles
- Kernel work: 2,400,000 cycles

Grand total = $960,000 + 840,000 + 2,400,000 = 4,200,000$ cycles

$$\text{Overhead fraction} = \frac{840,000}{4,200,000} \times 100\% = 20\%$$

Solution 5: Batching Optimization

✔ Batching 10 Blocks per Syscall

$$\text{New syscall count} = \frac{1,200}{10} = 120$$

$$\text{Trap overhead} = 700 \times 120 = 84,000 \text{ cycles}$$

$$\text{Kernel work} = 2,000 \times 120 = 240,000 \text{ cycles}$$

$$\text{User compute} = 960,000 \text{ cycles (unchanged)}$$

$$\textbf{Grand total} = 960,000 + 84,000 + 240,000 = \textbf{1,284,000}$$

$$\textbf{Overhead} = \frac{84,000}{1,284,000} \times 100\% \approx \textbf{6.54\%}$$

System Call Overhead - Key Takeaways

💡 Key Insight

Batching reduced overhead from **20%** to **6.54%** and total cycles from **4.2M** to **1.28M** – a 3.3x improvement!

📌 Real-World Implications

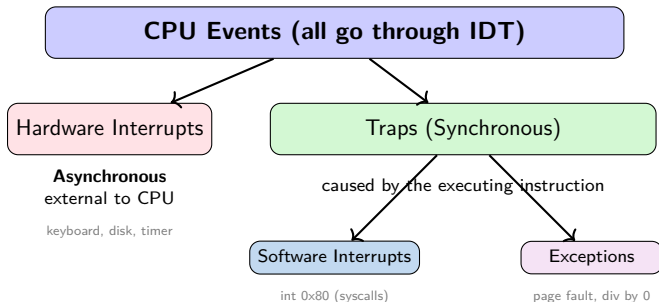
- Minimize syscall frequency when possible
- Buffer/Batch I/O operations

Interrupts and Exceptions

Connecting to Traps

Remember Traps?

Earlier we learned that **traps** are synchronous events caused by the executing instruction. Now let's see how interrupts and exceptions fit into this picture.



Hardware Interrupts vs Software Interrupts

⚡ Hardware Interrupts

- Triggered by external I/O devices
- **Asynchronous** – can occur at any time, between any instructions
- CPU was not expecting it
- Examples: keyboard, disk, network, timer

📄 Software Interrupts (Traps)

- Triggered by INT instruction
- **Synchronous** – caused by executing code
- CPU executes `int 0x80` intentionally
- Used for legacy system calls

Exceptions: Types and Examples

! What are Exceptions?

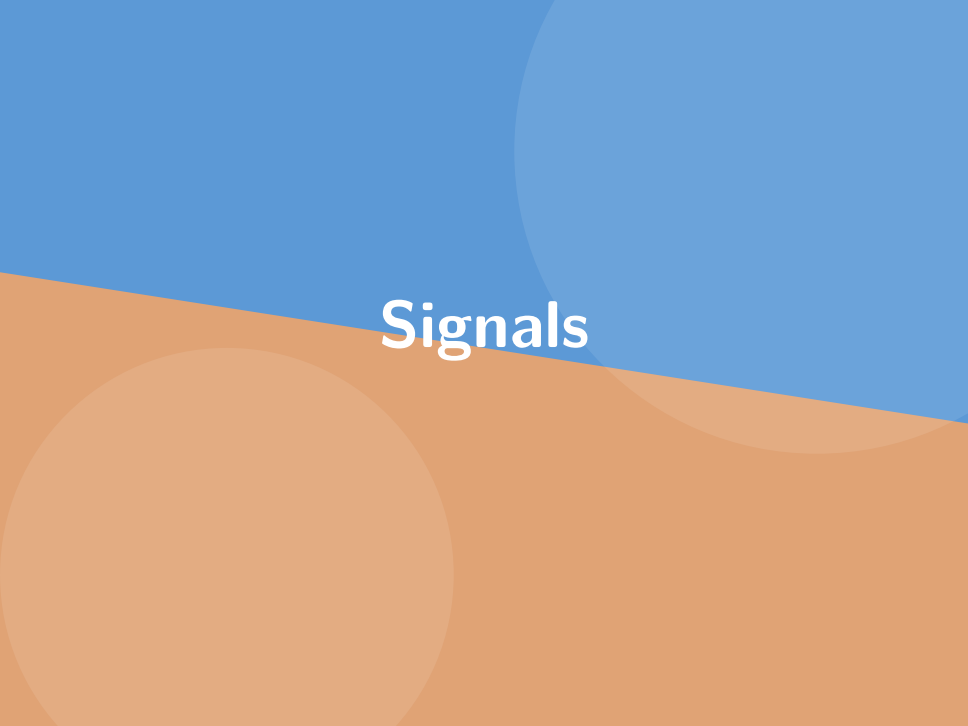
Exceptions are **synchronous traps** triggered when the CPU encounters problematic conditions during instruction execution.

↻ Kernel-Recoverable

- Page Fault – kernel loads page from disk, process continues
- Breakpoint (int 3) – debugger handles it

🔔 Signal to Process

- Invalid memory access → SIGSEGV
- Division by zero → SIGFPE
- Process dies unless it catches signal



Signals

What are Signals?

Signals: Software Notifications

Signals are a Unix mechanism for notifying a **process or thread** that an event has occurred. They can be sent by the kernel, by another process, or by the process/thread itself.

From the kernel:

- CPU exception (SIGSEGV, SIGFPE)
- Child terminated (SIGCHLD)
- Timer expired (SIGALRM)

From another process:

- `kill(pid, sig)`
- Ctrl+C (SIGINT)
- Custom (SIGUSR1/2)

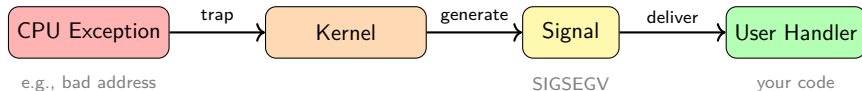
To specific thread:

- `pthread_kill(tid, sig)`
- Faults (SIGSEGV goes to faulting thread)

Signals: Connecting Exceptions to User Mode

💡 Key Insight

We just saw that CPU exceptions like invalid memory access result in SIGSEGV. Signals are how the kernel **communicates these events to user-mode** so the process/thread can respond.



Signal Handler Table

How the Kernel Manages Signals

In Linux, each process has a `task_struct` (its PCB), which includes a **signal handler table** mapping each signal number to a handler.

Signal	Handler
SIGINT (2)	my_handler (custom)
SIGTERM (15)	SIG_DFL → terminate
SIGCHLD (17)	SIG_DFL → ignore
SIGKILL (9)	Always terminate (forced)

Custom = your function

SIG_DFL = kernel default

Forced = cannot override

Signal Handler Table Caveats

Key Insight

SIG_DFL means "use the default action" – each signal has a predefined behavior (terminate, ignore, stop, or core dump) that applies when no custom handler is set.

Warning

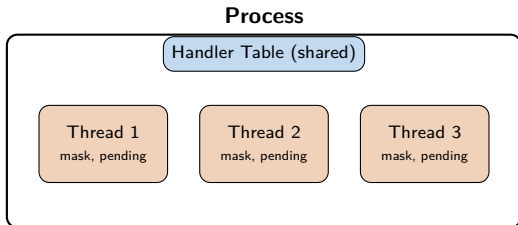
SIGKILL and **SIGSTOP** **cannot be caught or ignored** – ensures the system can always terminate/stop a process.

Two Levels of Signal State



Shared vs Per-Thread

In multi-threaded programs, some signal state is **shared** (process-level) and some is **per-thread**.



Process-Level vs Thread-Level



Process-Level (Shared)

- **Handler table** – all threads share the same signal handlers
- **Process pending queue** – signals sent to PID
- Set via `sigaction()`



Thread-Level (Per-Thread)

- **Signal mask** – each thread blocks different signals
- **Thread pending queue** – signals sent to TID
- Set via `pthread_sigmask()`

Common Signals

☰ Signals You Should Know

Signal	Default Action	Typical Cause
SIGINT (2)	Terminate	Ctrl+C from terminal
SIGKILL (9)	Terminate	kill -9 (cannot catch!)
SIGSEGV (11)	Terminate + core	Invalid memory access
SIGTERM (15)	Terminate	Polite termination request
SIGCHLD (17)	Ignore	Child process stopped/terminated
SIGSTOP (19)	Stop	Pause process (cannot catch!)
SIGCONT (18)	Continue	Resume stopped process

Registering Signal Handlers

⌘ Two Ways to Register Handlers

- `signal(sig, handler)` – simple, but **unpredictable behavior**
- `sigaction(sig, &act, &oldact)` – preferred, consistent and configurable

⚠ Warning

The problem with `signal()`: On some systems (System V), after the handler runs, the signal disposition is **reset to `SIG_DFL`**. On other systems (BSD), the handler **remains installed**. This inconsistency makes `signal()` unreliable for portable code.

signal() vs sigaction() – Comparison

Key Differences

	signal()	sigaction()
Handler persistence	May reset to SIG_DFL after handling	Stays installed
Blocking during handler	System-dependent	via sa_mask
Syscall restart	System-dependent	SA_RESTART flag
Portability	Behavior varies	Consistent

Sigation Flags

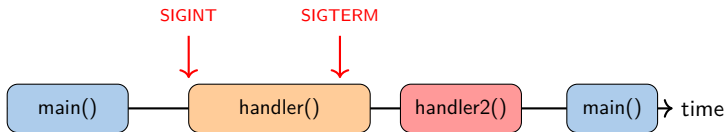
Useful sigaction Flags

- `SA_RESTART` – automatically restart interrupted syscalls (read, write, etc.)
- `SA_RESETHAND` – reset handler to `SIG_DFL` after one invocation
- `SA_NODEFER` – don't block this signal while handler runs

Signals During Handler Execution

⚠ What if a Signal Arrives While in a Handler?

By default, when a handler runs, the signal that triggered it is **blocked**. But other signals can still interrupt the handler!



⚠ Warning

This can cause **reentrancy problems** – the second handler interrupts the first, potentially corrupting shared state!

What is Async-Signal-Safe?

Definition: Async-Signal-Safe

A function is **async-signal-safe** if it can be safely called from within a signal handler, even if the signal interrupted that same function (or another non-safe function) in the main program.

A function is async-signal-safe if it:

- Uses no static/global data, OR
- Accesses shared data only atomically, OR
- Can be interrupted and re-entered without corruption

Dangers of not being Async-Signal-Safe

Warning

Most library functions are **NOT** async-signal-safe because they use internal locks, global buffers, or static variables. Calling them from a handler can cause **deadlocks** or **data corruption**.

Why printf() is NOT Async-Signal-Safe

The Reentrancy Problem

Imagine this scenario:

- 1 `main()` calls `printf("Hello")` – acquires internal lock, starts filling buffer
- 2 Signal arrives, handler runs
- 3 Handler calls `printf("Signal!")` – tries to acquire same lock...
DEADLOCK!

Or worse: buffer is in inconsistent state → **corruption**

Blocking Signals During Handler



Using sa_mask

sa_mask specifies signals to **add to the thread's signal mask** while the handler runs. These signals are blocked for the thread (not just the handler). The original mask is restored when the handler returns.

Block All Signals During Handler

```
struct sigaction sa;  
sa.sa_handler = handler;  
sigfillset(&sa.sa_mask); // block ALL  
sa.sa_flags = 0;  
sigaction(SIGINT, &sa, NULL);
```



Key Insight

By default, only the triggering signal is added to the mask. Use sa_mask to block additional signals.

Original Mask vs sa_mask

Original Mask

The thread's **existing signal mask** at the moment the kernel is about to run the handler.

- Set via `sigprocmask()` or `pthread_sigmask()`
- The **baseline** – kernel saves it and restores it on handler return

sa_mask

A **static per-handler setting** chosen when you install the handler.

- Meaning: "while this handler runs, **also** block these signals"
- **Temporary** – only for duration of handler

Original Mask and sa_mask

How They Combine

When signal S is delivered:

$$\text{handler_mask} = \text{original_mask} \cup \text{sa_mask} \cup \{S\}$$

On normal handler return:

$$\text{current_mask} := \text{original_mask}$$

SA_NODEFER

🚩 SA_NODEFER Flag

By default, the delivered signal S is **automatically blocked** during its own handler (prevents re-entry).

- **With SA_NODEFER:** Signal S is **not** added to the mask – handler can be interrupted by the same signal (re-entrant)
- Formula becomes: $\text{handler_mask} = \text{original_mask} \cup \text{sa_mask}$

Signal Mask Example

Concrete Example

Setup:

- Thread's original mask blocks: {SIGINT}
- Handler installed with `sa_mask` blocking: {SIGTERM}
- Signal delivered: SIGUSR1 (no SA_NODEFER)

Signal Mask Example

⚡ While Handler Runs

Blocked signals:

`{SIGINT, SIGTERM,
SIGUSR1}`

(original $\cup$ sa_mask $\cup$ delivered)

✓ After Handler Returns

Blocked signals:

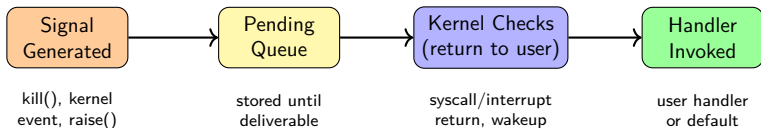
`{SIGINT}`

(back to original mask)

Signal Delivery Flow

🕒 How Signals Get Delivered

Signal delivery happens in stages: **generation** → **pending** → **delivery** → **handling**.



Signal Delivery Flow and Syscalls

⚡ Signals Can Interrupt Blocking Syscalls

A signal can wake a process blocked in a syscall (e.g., `read()`):

- Syscall returns `-1` with `errno = EINTR`
- **SA_RESTART flag**: kernel automatically restarts the syscall after handler returns

Process-Directed vs Thread-Directed Signals

Process-Directed

Sent to the **process as a whole**:

- `kill(pid, sig)`
- `SIGTERM`, `SIGINT`
- External events

Delivery: Any thread that doesn't have the signal blocked can receive it.

Thread-Directed

Sent to a **specific thread**:

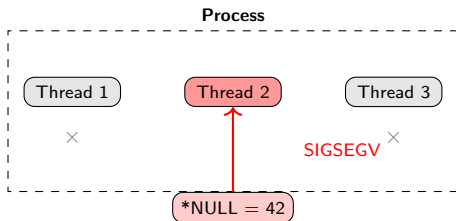
- `pthread_kill(tid, sig)`
- `SIGSEGV`, `SIGFPE`
- Faults caused by that thread

Delivery: Only that specific thread can receive it.

Synchronous Signals are Thread-Directed

Warning

Synchronous signals (SIGSEGV, SIGFPE, SIGBUS) are **always thread-directed** – they're caused by the executing thread's own actions.



Only Thread 2 (the faulting thread) receives SIGSEGV

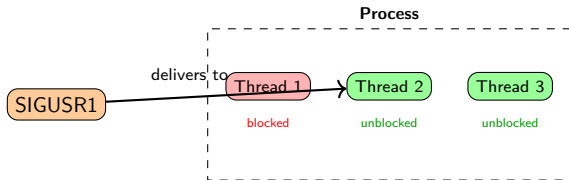
Multi-Threaded Signal Delivery

Which Thread Gets the Signal?

For **process-directed** signals, the kernel picks one thread that:

- 1 Does **not** have the signal blocked (via `pthread_sigmask`)
- 2 Is available to handle it

The choice is **arbitrary** – don't rely on which thread receives it!



Signal Masks and Pending Signals

🚫 Signal Mask (per thread)

Each thread has its own **signal mask** – a set of blocked signals.

- Set via `pthread_sigmask()`
- Inherited from parent on `fork()`
- Blocked signals stay pending

🕒 Pending Signals

Two levels of pending signals:

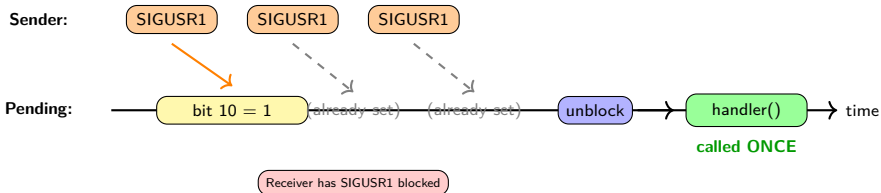
- **Process-level:** shared pending queue
- **Thread-level:** per-thread pending queue

Standard signals don't queue – only one pending per signal number.

Standard Signals: Coalescing Behavior

Bitmask, Not a Queue

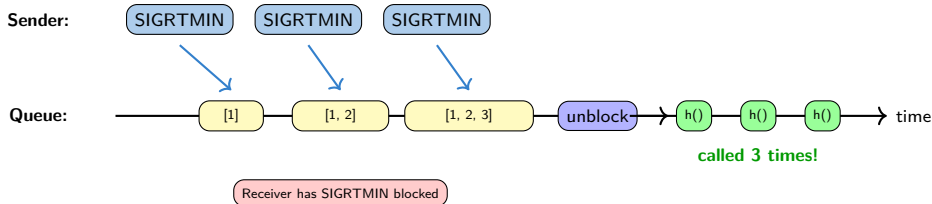
Standard signals (1–31) are stored as a **bitmask** in the kernel. Each signal number is a single bit: either pending (1) or not (0). Multiple instances of the same signal **collapse into one**.



Real-Time Signals: Queued Delivery

SIGRTMIN to SIGRTMAX

Real-time signals (typically 34–64) provide **reliable, queued** signal delivery – multiple instances are preserved, not coalesced.



Real-Time Signals: Features

☰ Key Properties

- **Range:** SIGRTMIN to SIGRTMAX (typically 34–64)
- **Delivery order:**
 - **Same signal:** FIFO (first sent = first delivered)
 - **Different signals:** Priority (lower number first)
- **Data:** Can carry payload via `sigqueue()`

💡 Use Cases

- Event counting
- IPC with data
- Timer notifications (`timer_create`)
- AIO completion

Sending Signals

> From Command Line

```
kill -SIGTERM <pid>
kill -9 <pid>    (SIGKILL)
kill -STOP <pid>
killall -SIGTERM name

Use ps aux to find PIDs
```

</> From Code

```
kill(pid, SIGTERM);
raise(SIGALRM);    (to self)
pthread_kill(tid, sig);

Returns 0 on success, -1 on error
```

Warning

Don't confuse with pthreads! `pthread_cond_signal()` is **not** a Unix signal – it uses futexes. Same word, different mechanism!

Signal Handler Example (using sigaction)

⚡ Proper Signal Handler Setup

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>

void handle_sigint(int sig) {
    const char *msg = "\nCaught SIGINT. Exiting...\n";
    write(STDOUT_FILENO, msg, strlen(msg));
    _exit(0); // async-signal-safe
}
```

⚠ Warning

Use `write()` not `printf()` in handlers – `printf` is not `async-signal-safe`!

Signal Handler Example (cont.)

⌘ Main Function with sigaction()

```
int main() {
    struct sigaction sa;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask); // don't block other signals
    sa.sa_flags = SA_RESTART; // restart interrupted syscalls

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }
    printf("Running. Press Ctrl+C to exit.\n");
    while (1) { sleep(1); }
    return 0;
}
```

Signal Masking as Synchronization

Protecting Critical Sections

Signal masking is a **synchronization mechanism** – like mutexes and semaphores, it prevents race conditions and deadlocks by controlling when handlers can interrupt your code.

Deadlock Scenario (Without Masking)

- 1 `main()` acquires `mutex_lock(&data_lock)`
- 2 Signal arrives, handler runs
- 3 Handler calls `mutex_lock(&data_lock)` → **DEADLOCK!** (In general behavior may be undefined, and not async-signal-safe)

Same thread holds the lock and waits for it – classic self-deadlock.

Signal Masking as Synchronization

✓ Solution: Mask During Critical Section

- 1 `sigprocmask(SIG_BLOCK, &mask, &old)` – block signals
- 2 `mutex_lock(&data_lock)` – safe, no handler can interrupt
- 3 Do critical work...
- 4 `mutex_unlock(&data_lock)`
- 5 `sigprocmask(SIG_SETMASK, &old, NULL)` – restore, pending signals now delivered

Masking and Unmasking in Code

⚡ Blocking Signals

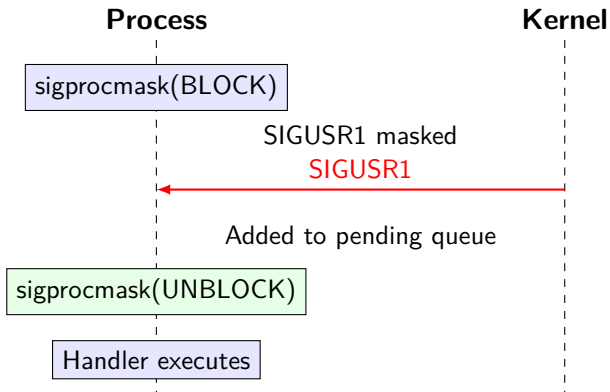
```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    sigset_t mask;
    sigemptyset(&mask);           // Initialize empty set
    sigaddset(&mask, SIGUSR1);    // Add SIGUSR1 to mask

    // Block SIGUSR1
    if (sigprocmask(SIG_BLOCK, &mask, NULL) == -1) {
        perror("sigprocmask");
        return 1;
    }
    // Critical section: SIGUSR1 is masked here
    printf("Entering critical section\n");
    do_critical_stuff();

    // Unblock SIGUSR1
    sigprocmask(SIG_UNBLOCK, &mask, NULL);
    // Pending SIGUSR1 delivered now
    printf("SIGUSR1 delivered after unmasking\n");
    return 0;
}
```


Signal Mask/Unmask Sequence



Warning: Async-Signal-Safety

Warning

The following exercises use `pthread_mutex_*` and `pthread_rwlock_*` in signal handlers. These functions are **NOT async-signal-safe!**

Key Insight

For teaching purposes: The following exercises assume signals arrive **between** function calls, not during. This lets us focus on signals/lock-related issues without undefined behavior.

Exercise 2: Logging with Signals

Shared State and Actors

```
char log_buf[4] = {0}; // Shared log buffer
int idx = 0;           // Next write position
pthread_rwlock_t lock; // Read-write lock

void handler(int sig) { // SIGUSR1 Handler
    pthread_rwlock_wrlock(&lock);
    log_buf[idx++] = '!';
    pthread_rwlock_unlock(&lock);
}

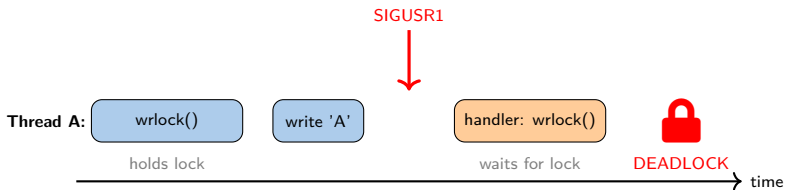
void* writer(void* arg) { // Thread A
    pthread_rwlock_wrlock(&lock);
    log_buf[idx++] = 'A';
    pthread_rwlock_unlock(&lock);
}

void* reader(void* arg) { // Thread B
    pthread_rwlock_rdlock(&lock);
    printf("%s\n", log_buf);
    pthread_rwlock_unlock(&lock);
}
```

Exercise: The Dangerous Interleaving

💀 What Could Go Wrong?

Assume SIGUSR1 is sent to **Thread A** while it holds the write lock...



⚠ Warning

Handler runs in **same thread** context! Thread A waits for a lock it already holds → **self-deadlock**.

Exercise: Your Task

Interleaving Analysis

Given the code on the previous slides, and assuming **no signal masking**:

- 1 List all possible interleavings of Thread A, Thread B, and the signal handler
- 2 Mark which interleavings result in **deadlock** (🔒)
- 3 For safe interleavings, write the **output** of Thread B's printf

Exercise: Assumptions

Assumptions

- SIGUSR1 is sent **once** during execution
- Signal can be delivered to **either** Thread A or Thread B
- Each thread runs its function exactly once

Solution: Interleaving Table

#	Order	Signal To	Result
1	A(lock) → SIG → handler(lock)	A (holding lock)	 Deadlock
2	B(rdlock) → SIG → handler(wrlock)	B (holding rdlock)	 Deadlock
3	A completes → SIG → handler → B	A (after unlock)	prints "A!"
4	A completes → B → SIG → handler	B (after unlock)	prints "A", then "!" added
5	B → A → SIG → handler	A (after unlock)	prints "", then "A!" added
6	SIG → handler → A → B	before any lock	prints "!A"

Deadlock Cases

Signal delivered while **any thread** holds the lock → handler blocks forever

Safe Cases

Signal delivered when **no lock held** → handler completes normally

Solution: The Fix (Signal Masking)



Mask Signals During Critical Section

```
void* writer(void* arg) {
    sigset_t mask, old;
    sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1);

    sigprocmask(SIG_BLOCK, &mask, &old); // Block signal
    pthread_rwlock_wrlock(&lock);
    log_buf[idx++] = 'A';
    pthread_rwlock_unlock(&lock);
    sigprocmask(SIG_SETMASK, &old, NULL); // Unblock, pending delivered
    return NULL;
}

void* reader(void* arg) {
    sigset_t mask, old;
    sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1);

    sigprocmask(SIG_BLOCK, &mask, &old); // Block signal
    pthread_rwlock_rdlock(&lock);
    printf("%s\n", log_buf);
    pthread_rwlock_unlock(&lock);
    sigprocmask(SIG_SETMASK, &old, NULL); // Unblock
    return NULL;
}
```


Solution: Safe Outputs Only

✓ With Signal Masking – No Deadlocks Possible

Signal delivery is **deferred** until lock is released. Handler always runs when lock is free.

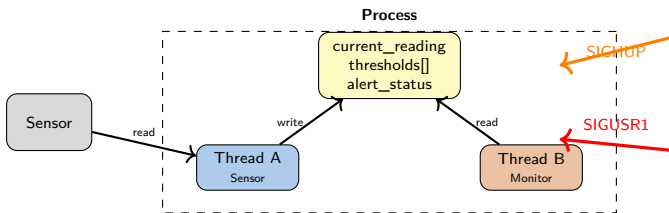
☰ Possible Outputs (All Safe)

Order	Thread B prints
handler → A → B	"!A"
A → handler → B	"A!"
A → B → handler	"A"
B → handler → A	""
B → A → handler	""
handler → B → A	"!"

Exercise 3: Sensor Monitor with Alerts

Shared State

```
int current_reading = 0;           // Latest sensor value
int thresholds[2] = {10, 90};     // {low, high}
int alert_status = 0;              // 0=normal, 1=alert
pthread_mutex_t sensor_lock;       // Protects current_reading
pthread_mutex_t config_lock;       // Protects thresholds
```



Exercise 3: Sensor Monitor with Alerts

Thread A: Sensor Reader

```
void* sensor_thread(void* arg) {
    while (1) {
        int val = read_sensor();
        pthread_mutex_lock(&sensor_lock)
            ;
        current_reading = val;
        pthread_mutex_unlock(&
            sensor_lock);
    }
}
```

Thread B: Monitor

```
void* monitor_thread(void* arg) {
    while (1) {
        pthread_mutex_lock(&config_lock)
            ;
        int lo = thresholds[0];
        int hi = thresholds[1];
        pthread_mutex_unlock(&
            config_lock);
        // check and update alert_status
    }
}
```

Exercise: Signal Handlers

SIGHUP: Reconfigure Thresholds

```
void sighup_handler(int sig) {  
    // Update thresholds from config  
    pthread_mutex_lock(&config_lock);  
    thresholds[0] = new_low; // Step  
    1  
    // <-- SIGUSR1 could arrive here!  
    thresholds[1] = new_high; // Step  
    2  
    pthread_mutex_unlock(&config_lock)  
    ;  
}
```

SIGUSR1: Force Alert Check

```
void sigusr1_handler(int sig) {  
    pthread_mutex_lock(&config_lock);  
    int lo = thresholds[0];  
    int hi = thresholds[1];  
    pthread_mutex_unlock(&config_lock)  
    ;  
  
    pthread_mutex_lock(&sensor_lock);  
    int val = current_reading;  
    pthread_mutex_unlock(&sensor_lock)  
    ;  
  
    alert_status = (val < lo || val >  
        hi);  
}
```

Exercise: Signal Handlers - the problem

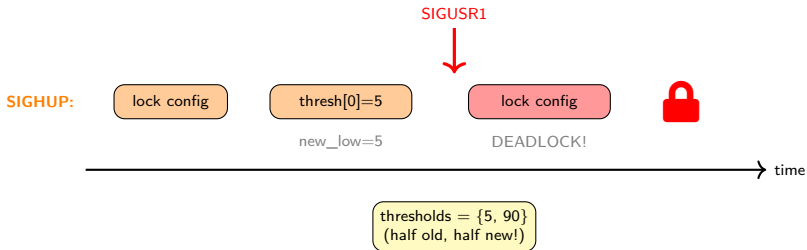
Warning

Both handlers try to acquire locks. Signals can be delivered to **either thread**. What could go wrong?

Exercise: Danger – Nested Handler

⚠ SIGUSR1 During SIGHUP Handler

If SIGUSR1 arrives while SIGHUP handler is between updating thresholds[0] and thresholds[1]...



Exercise: Danger – Nested Handler - The problems

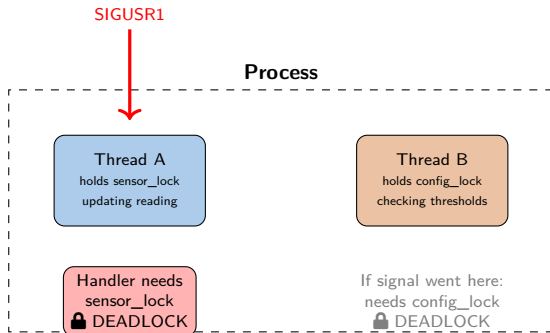
Problem 1: Deadlock

SIGUSR1 handler tries to acquire `config_lock` already held by SIGHUP handler (same thread)

Problem 2: Inconsistent State

Even without deadlock, SIGUSR1 might see `{new_low, old_high}` → wrong alert!

Exercise: Danger – Signal to Lock-Holding Thread



Warning

SIGUSR1 handler needs **both** locks. If delivered to a thread holding **either** lock → deadlock!

Exercise: Your Task

Interleaving Analysis

Complete the table: for each scenario, determine the **outcome**.

#	Signal	Delivered To	State	Outcome?
1	SIGHUP	Thread A	holds sensor_lock	
2	SIGHUP	Thread B	holds config_lock	
3	SIGUSR1	Thread A	holds sensor_lock	
4	SIGUSR1	Thread B	holds config_lock	
5	SIGUSR1	Thread A	in SIGHUP handler	
6	SIGUSR1	Thread B	no lock held	

Exercise: Solution – Interleaving Analysis

#	Signal	To	State	Outcome
1	SIGHUP	A	holds sensor_lock	✓ Safe (SIGHUP needs config_lock only)
2	SIGHUP	B	holds config_lock	🔒 Deadlock (SIGHUP needs config_lock)
3	SIGUSR1	A	holds sensor_lock	🔒 Deadlock (SIGUSR1 needs sensor_lock)
4	SIGUSR1	B	holds config_lock	🔒 Deadlock (SIGUSR1 needs config_lock)
5	SIGUSR1	A	in SIGHUP handler	🔒 Deadlock (config_lock held)
6	SIGUSR1	B	no lock held	✓ Safe

Exercise: Solution – The Fix Part 1



Fix 1: Block SIGUSR1 During SIGHUP

```
struct sigaction sa_hup;  
sa_hup.sa_handler = sighup_handler;  
sigemptyset(&sa_hup.sa_mask);  
sigaddset(&sa_hup.sa_mask, SIGUSR1); // Block SIGUSR1 during  
    SIGHUP handler  
sigaction(SIGHUP, &sa_hup, NULL);
```

Exercise: Solution – The Fix Part 2



Fix 2: Mask Signals While Holding Locks

```
void* sensor_thread(void* arg) {
    sigset_t mask, old;
    sigemptyset(&mask);
    sigaddset(&mask, SIGHUP);
    sigaddset(&mask, SIGUSR1);

    while (1) {
        int val = read_sensor();
        sigprocmask(SIG_BLOCK, &mask, &old);    // Block both
                                                signals
        pthread_mutex_lock(&sensor_lock);
        current_reading = val;
        pthread_mutex_unlock(&sensor_lock);
        sigprocmask(SIG_SETMASK, &old, NULL);    // Unblock
    }
}
```

User Mode Threading

setjmp and longjmp

⚡ Non-Local Jumps

- `setjmp`: Saves thread state in `jmp_buf`, returns 0
- `longjmp`: Restores thread state, returns non-zero
- Used for non-linear code execution and exception handling

💡 Key Insight

Think of `setjmp/longjmp` as a "checkpoint and restore" mechanism – save your position, and jump back to it later from anywhere in the call stack!

Error Recovery with setjmp/longjmp

</> Division with Error Recovery

```
#include <stdio.h>
#include <setjmp.h>

jmp_buf jump_buffer;

void divide(int numerator, int denominator) {
    if (denominator == 0) {
        printf("Error: Division by zero!\n");
        longjmp(jump_buffer, 1); // Jump back
    }
    printf("Result: %d\n", numerator / denominator);
}
```

Error Recovery with setjmp/longjmp (cont.)

⌞ Main with Checkpoint

```
int main() {
    int num = 10, denom = 0;
    if (setjmp(jump_buffer) == 0) {
        // First time: setjmp returns 0
        printf("Attempting division...\n");
        divide(num, denom);
        printf("Division succeeded!\n");
    } else {
        // After longjmp: execution resumes here
        printf("Recovered from error.\n");
    }
    printf("Program continues.\n");
    return 0;
}
```


Gotcha: sigsetjmp and siglongjmp

Warning

If you use `longjmp()` to exit a signal handler, the signal mask **might not be restored** – behavior is platform-dependent!

POSIX Solution

- `sigsetjmp(env, savemask)` – if `savemask` $\neq 0$, saves current signal mask
- `siglongjmp(env, val)` – restores the mask saved by `sigsetjmp`

User-Mode Threading

Threading Without Kernel

Use `setjmp/longjmp` to manage threads entirely in user space.

Pros

- No system call overhead
- Faster context switching
- Smaller state to save

Cons

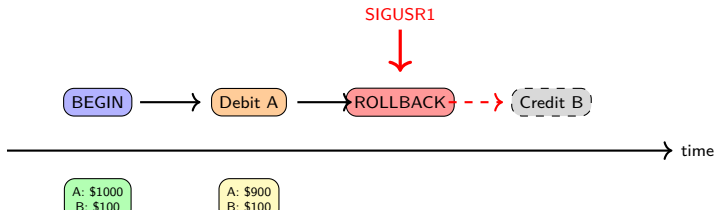
- OS unaware of threads
- No multi-core support
- Complex to manage

Exercise 4: Database Transaction Rollback

The SQL Analogy

In SQL, a transaction can be aborted with ROLLBACK:

```
BEGIN; UPDATE accounts SET balance=balance-100 WHERE
                        id='A';
        -- error occurs here → ROLLBACK
UPDATE accounts SET balance=balance+100 WHERE id='B';
        COMMIT;
```

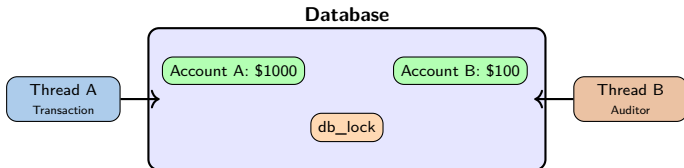


Exercise: Shared State

</> Database State

```
int account[2] = {1000, 100};    // Account A=1000, B=100
int total_deposited = 1100;      // Bank's record of total money in system
pthread_mutex_t db_lock;         // Protects all account data
sigjmp_buf checkpoint;          // Rollback point
volatile sig_atomic_t in_tx = 0; // In transaction flag
```

SIGUSR1 = ROLLBACK



Exercise: The Threads

Thread A: Transaction Worker

```
void* transaction(void* arg) {
    if (sigsetjmp(checkpoint, 1) != 0) {
        // ROLLED BACK - jumped here!
        printf("Transaction aborted\n");
        in_tx = 0;
        return NULL;
    }

    in_tx = 1;
    pthread_mutex_lock(&db_lock); //
        BEGIN
        account[0] -= 100;          //
            Debit A
        // <-- SIGUSR1 danger zone!
        account[1] += 100;          //
            Credit B
    pthread_mutex_unlock(&db_lock); //
        COMMIT
    in_tx = 0;
    printf("Transaction committed\n");
}
```

Thread B: Auditor

```
void* auditor(void* arg) {
    while (1) {
        pthread_mutex_lock(&db_lock);
        int sum = account[0] + account[1];
        int expected = total_deposited;
        ;
        pthread_mutex_unlock(&db_lock)
        ;

        if (sum != expected) {
            printf("INCONSISTENT!\n");
        }
        sleep(1);
    }
}
```



Key Insight

Exercise: The Bank

Key Insight

Bank's internal transfers: money moves between accounts but `total_deposited` stays constant.

Exercise: The Dangerous Handler

ROLLBACK Handler (DANGEROUS)

```
void rollback_handler(int sig) {  
    if (in_tx) {  
        siglongjmp(checkpoint, 1); // DANGER: lock  
                                   still held!  
    }  
}
```

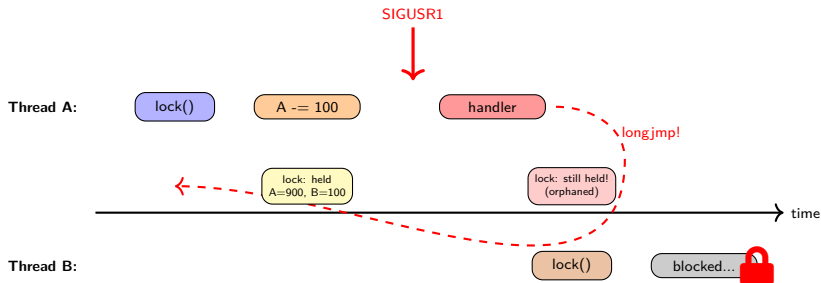
Problem 1: Lock Abandoned

siglongjmp jumps out while db_lock is held → lock **never released** → Auditor thread **deadlocks forever**

Problem 2: Inconsistent State

If signal after debit but before credit: A=\$900, B=\$100 → sum $\neq$ total_deposited!

Exercise: Danger Timeline



Result

- Thread A: continues from checkpoint, thinks rollback succeeded
- Thread B: blocked forever waiting for lock that will never be released
- Database: inconsistent state (A=900, B=100, sum=1000)

Exercise: Your Task

Interleaving Analysis

For each scenario, determine what happens when SIGUSR1 arrives:

#	Signal Arrives	Lock State	Outcome?
1	Before sigsetjmp	not held	
2	After sigsetjmp, before lock()	not held	
3	After lock(), before debit	held	
4	After debit, before credit	held	
5	After credit, before unlock()	held	
6	After unlock() (committed)	not held	

Exercise: Solution – Analysis

#	Signal Arrives	Lock	Outcome
1	Before sigsetjmp	none	⊘ Handler jumps to unset checkpoint = undefined behavior!
2	After sigsetjmp, before lock()	none	✓ Safe rollback, no cleanup needed
3	After lock(), before debit	held	🔒 Deadlock (lock orphaned), data OK
4	After debit, before credit	held	🔒⚠️ Deadlock + Inconsistent (A=900, B=100)
5	After credit, before unlock()	held	🔒 Deadlock (lock orphaned), data consistent
6	After unlock()	none	⊘ Too late, transaction already committed

Exercise: Solution – The Fix Part 1

✓ Fixed Handler: Cleanup Before Jump

```
volatile sig_atomic_t saved_state = 0; // Track saved state
int saved_A, saved_B;                // For actual rollback

void rollback_handler(int sig) {
    if (in_tx) {
        // Restore original values (actual rollback!)
        if (saved_state) {
            account[0] = saved_A;
            account[1] = saved_B;
            saved_state = 0;
        }
        pthread_mutex_unlock(&db_lock); // Release lock!
        siglongjmp(checkpoint, 1);
    }
}
```

Exercise: Solution – The Fix Part 2

✓ Fixed Transaction: Save State

```
void* transaction(void* arg) {
    if (sigsetjmp(checkpoint, 1) != 0) { printf("Aborted\n");
        return NULL; }

    in_tx = 1;
    pthread_mutex_lock(&db_lock);

    saved_A = account[0]; saved_B = account[1]; // Save for
        rollback
    saved_state = 1;

    account[0] -= 100;
    account[1] += 100;

    saved_state = 0;
    pthread_mutex_unlock(&db_lock);
    in_tx = 0;
}
```

See You Next Week!

Questions?

Recitation 4

Scheduling

Daniel Karalnik Liam Tal

Spring Semester 2026

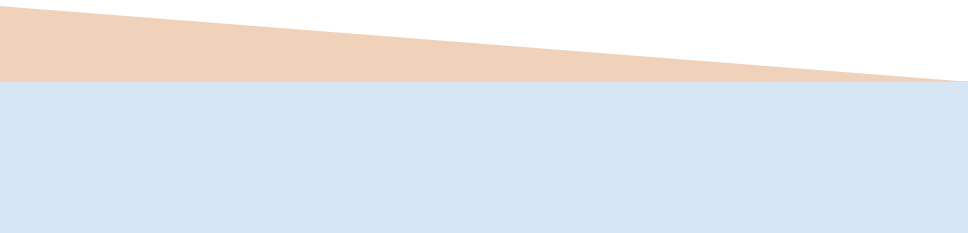


Table of Contents

- 1 Scheduling Intro
- 2 First Come First Serve
- 3 Shortest Job First
- 4 Round Robin
- 5 Multi Level Feedback Queues
- 6 Rotating Staircase Deadline Scheduler
- 7 Concluding Questions

Scheduling Intro

CPU Scheduler Overview

The CPU Scheduler

The **CPU Scheduler** is responsible for allocating CPU time among threads.

- Thread states: **New, Running, Ready, Wait, Terminated**
- Context switching **may** occur when:
 - A thread calls `yield()` – *a hint to the scheduler, not a guarantee*
 - A syscall blocks on a synchronization primitive (thread enters Wait state)
 - A timer interrupt fires (preemption)
- **Note:** `yield()` only suggests the scheduler consider switching; if no other thread is ready, the same thread may continue running

Scheduling Mechanism vs Policy



Policy

Algorithm that determines **which** threads run **when**.



Mechanism

Maintains thread states, handles context switches, and tracks performance metrics.



Why It Matters

- **Responsiveness:** Timely reaction to user inputs
- **Fairness:** Ensuring all threads get CPU time, preventing starvation
- **CPU Utilization:** Keeping the CPU busy and efficient

Preemptive Scheduling & Quantum

Preemptive Scheduling

The scheduler can **interrupt and switch out** a running thread.

- **Quantum:** The maximum time a thread can run before being preempted
- Considerations:
 - Long quantum $\Rightarrow$ Poor responsiveness
 - Short quantum $\Rightarrow$ High context-switch overhead

Typical Values

Quantum typically ranges from **10ms to 100ms** – balancing responsiveness with efficiency.

Types of Scheduling Algorithms

Offline Scheduling

Job count and length are **known in advance**.

- First Come First Serve (FCFS)
- Shortest Job First (SJF)

Online Scheduling

No prior knowledge of job arrival or length.

- Round Robin
- Random Scheduling



First Come First Serve

First Come First Serve (FCFS)

☰ FCFS Overview

Threads are executed **in the order they arrive**. Simple but can lead to the **convoy effect**.

- Simple to implement – just a FIFO queue
- Short threads get stuck behind long ones (convoy effect)
- Suitable for batch systems but not interactive systems

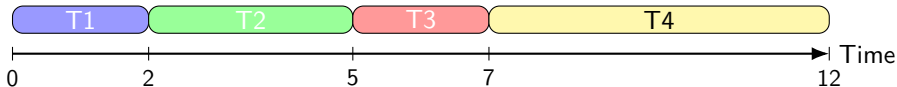
⚠ Convoy Effect

A single long-running thread can delay all subsequent threads, even if they are very short!

First Come First Serve (FCFS)

FCFS Scheduling

Threads run in arrival order.



Scenario:

Threads arrive at times 0, 1, 2, and 3.

They are scheduled strictly in arrival order.

Shortest Job First

Shortest Job First (SJF)

↓ SJF Overview

Executes the **shortest thread available** first. Optimal for minimizing average waiting time.

- **Preemptive SJF** (Shortest Remaining Time First) improves responsiveness
- Challenges:
 - Requires accurate prediction of thread burst lengths
 - Risk of **starvation** for longer threads

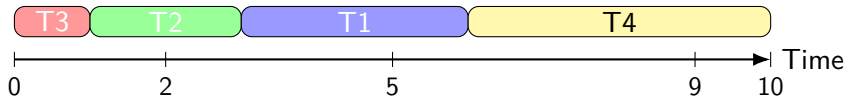
💡 Optimality

SJF is **provably optimal** for minimizing average waiting time when all threads are known in advance.

Shortest Job First (SJF)

SJF (Shortest Job First) Scheduling

Threads run in order of increasing CPU burst time.



Scenario:

All threads arrive at time 0.

Round Robin

Round Robin Scheduling

Round Robin Overview

Each thread gets an **equal share of CPU time** in a cyclic order.

- Good for time-sharing systems
- Performance depends on the chosen quantum
- Too small quantum $\Rightarrow$ High overhead
- Too large quantum $\Rightarrow$ Poor responsiveness

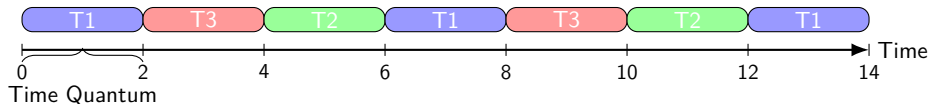
Fairness

Round Robin guarantees that no thread waits more than $(n - 1) \times q$ time units, where n is the number of threads and q is the quantum.

Round Robin Scheduling

Round Robin Scheduling

Time-sliced execution with preemption.



Scenario:

Each thread gets a fixed time slice.

If unfinished, it re-enters the back of the queue. (all threads arrive at time 0, order: T1, T3, T2)

Multi Level Feedback Queues

Multi-level Feedback Queues

MLFQ Overview

Threads are assigned to **different priority queues** (0, 1, 2, ... where 0 is highest). Each queue has a **constant quantum**. Higher priority queues have smaller quanta.

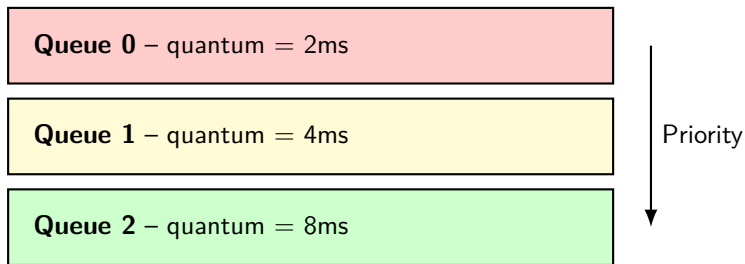
- **Higher priority** for interactive (I/O-bound) tasks (smaller quantum)
- **Lower priority** for CPU-bound tasks (larger quantum)
- Rules:
 - New threads start at queue 0 (highest priority)
 - Threads are **promoted** if they yield or block before using full quantum
 - Threads are **demoted** if they use their entire quantum (preempted)

Multi-level Feedback Queues

Starvation Risk

Without periodic priority boosting, low-priority threads may starve indefinitely!

MLFQ Overview

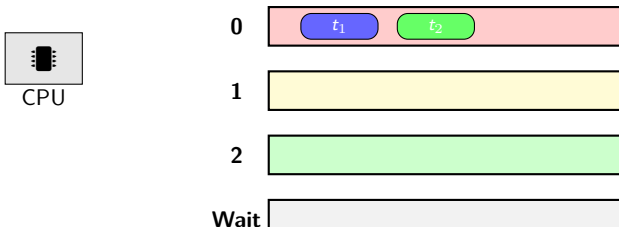


- New threads enter at queue 0 (highest priority)
- Each queue has a **constant quantum** (increases with lower priority)
- Threads using full quantum are demoted to lower-priority queues

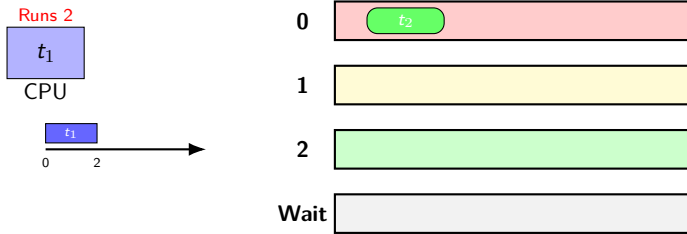
MLFQ Example: Setup

Parameters

- **Queue 0:** quantum = 2ms (highest priority)
- **Queue 1:** quantum = 4ms
- **Queue 2:** quantum = 8ms (lowest priority)
- All threads start in Queue 0
- t_1 : CPU-bound, t_2 : I/O-bound (blocks after 1ms)

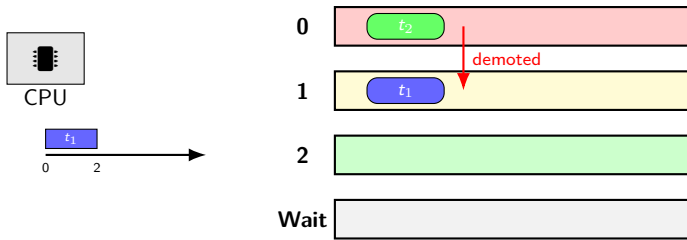


MLFQ Example: t_1 Runs (Step 1)



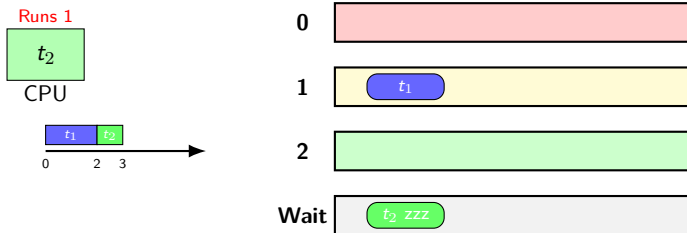
- t_1 uses full quantum (2ms) → **preempted**
- Preempted at end of quantum → **demote to Queue 1**

MLFQ Example: t_1 Demoted (Step 2)



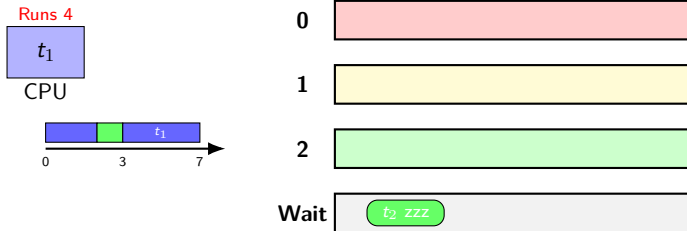
- t_1 moved to Queue 1 (lower priority)
- Scheduler picks from Queue 0 first → t_2 runs next

MLFQ Example: t_2 Blocks for I/O (Step 3)



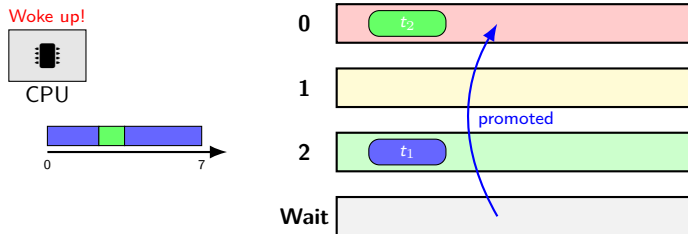
- t_2 runs 1ms then **blocks for I/O** (before quantum expires)
- Block/yield → **promoted to Queue 0** (stays at highest)
- t_2 moves to Wait area while I/O completes

MLFQ Example: t_1 Runs Again (Step 4)



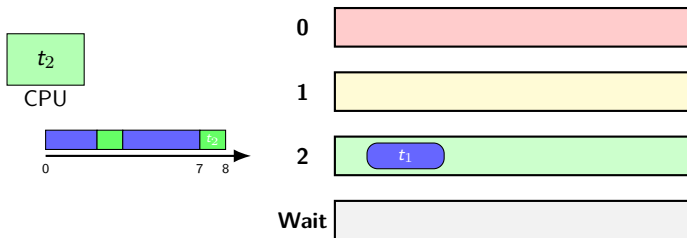
- Queue 0 empty → scheduler picks t_1 from Queue 1
- t_1 uses full quantum (4ms) → **demote to Queue 2**

MLFQ Example: t_2 Wakes Up (Step 5)



- t_2 I/O completes $\rightarrow$ returns to **Queue 0** (was promoted for blocking)
- t_1 demoted to Queue 2 (used full quantum in Queue 1)
- I/O-bound t_2 gets priority over CPU-bound t_1

MLFQ Example: Final State (Step 6)



💡 MLFQ Behavior

- I/O-bound threads (t_2) stay at high priority – responsive!
- CPU-bound threads (t_1) sink to lower queues – fair sharing
- Block/yield → promote; preempted → demote

Rotating Staircase Deadline Scheduler

How RSDL Works

☰ RSDL Overview

Threads are scheduled in a **round-robin** manner within each priority level. Uses **quotas** and **epochs** to ensure fairness and avoid starvation.

- **RR Interval:** Time slice for round-robin within a queue
- **Thread Quota:** Total CPU time a thread can use per epoch
- **Queue Quota:** Sum of all thread quotas in that queue
- **Epoch:** Period after which all quotas reset and threads return to starting priority

RSDL Demotion Rules

Thread Demotion Rules

- 1 **Thread quota exhaustion:** Thread demoted to next lower queue, **quota refreshed**
- 2 **Queue quota exhaustion:** **ALL threads** in queue demoted together, **quotas refreshed**
- 3 **I/O blocking:** Thread **stays at same priority level** (no demotion)

Note: Queue quotas are fixed per epoch – NOT increased when threads are demoted into a queue.

RSDL Epoch End Rules

↻ Epoch End and Reset

Epoch ends when:

- All queue quotas depleted (lowest queue exhausted), OR
- No runnable threads (all waiting on I/O)

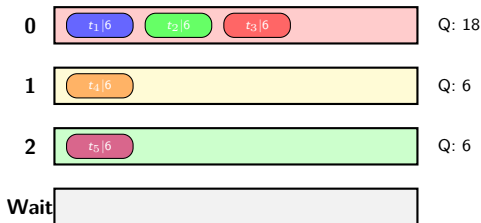
On reset:

- All thread quotas reset to initial values
- All queue quotas reset
- All threads return to their **original starting priority**

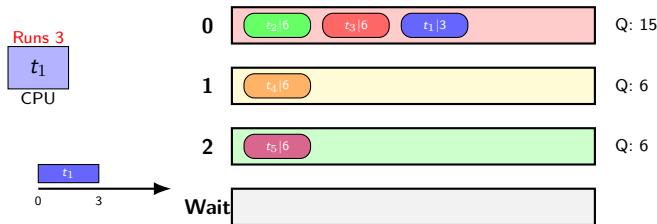
RSDL Example: Setup

Parameters

- **RR Interval:** 3ms, **Thread Quota:** 6ms each
- 5 threads: t_1, t_2, t_3 in Queue 0; t_4 in Queue 1; t_5 in Queue 2
- Queue quota = sum of thread quotas (fixed per epoch, NOT changed by demotion)

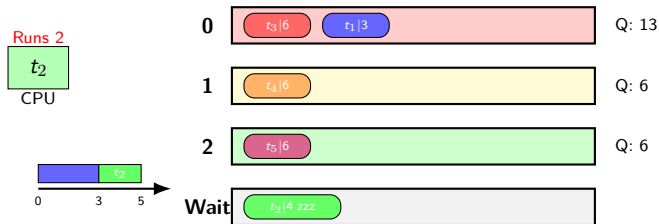


RSDL Example: t_1 Runs (Step 1)



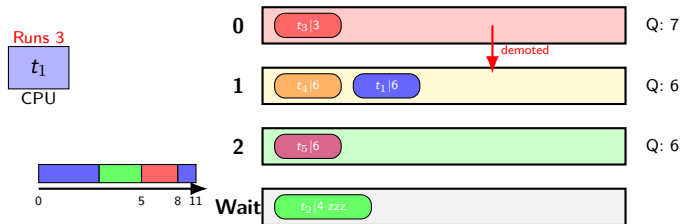
- t_1 runs for RR interval (3ms), quota: 6→3
- Queue 0 quota: 18→15 (decremented by time used)
- t_1 moves to end of Queue 0 (round-robin)

RSDL Example: t_2 Blocks for I/O (Step 2)



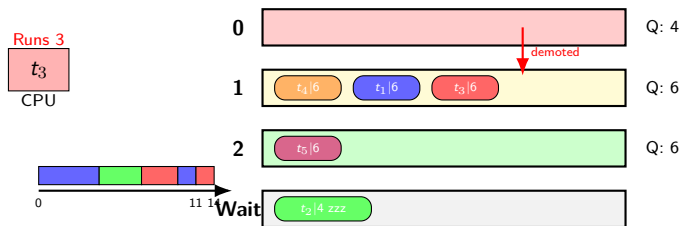
- t_2 runs 2ms then **blocks for I/O**, quota: 6→4
- Queue 0 quota: 15→13
- t_2 **stays in Queue 0** (I/O block = no demotion)

RSDL Example: t_1 Exhausts Quota (Step 3)



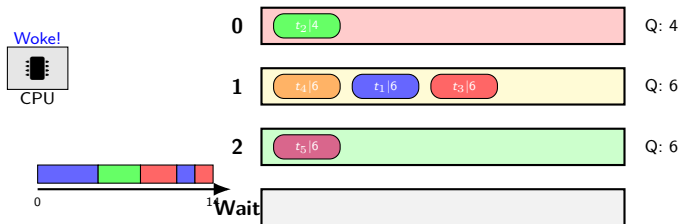
- After t_3 runs 3ms, t_1 runs 3ms (quota: $3 \rightarrow 0$)
- t_1 **quota exhausted** → **demoted to Q1, quota refreshed to 6**
- Q1 quota stays 6 (fixed per epoch, not increased by demotion)

RSDL Example: t_3 Exhausts Quota (Step 4)



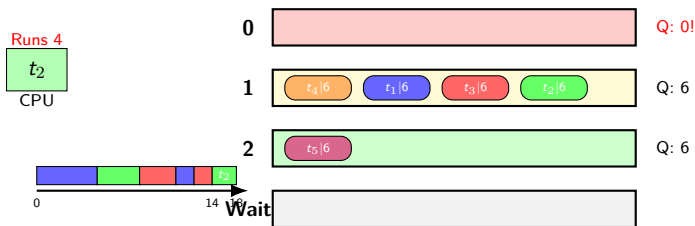
- t_3 runs 3ms (quota: $3 \rightarrow 0$) $\rightarrow$ demoted to Q1
- Q0 quota: $7 \rightarrow 4$ (only t_2 's 4ms remaining, sleeping)
- Q1 quota stays 6 (fixed per epoch)

RSDL Example: t_2 Wakes Up (Step 5)



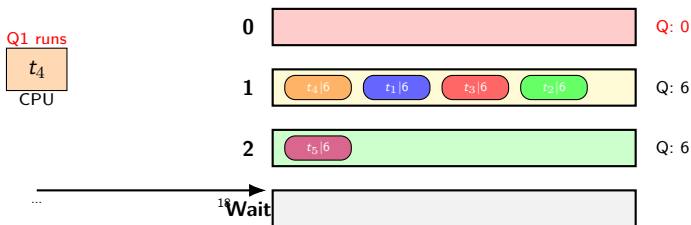
- t_2 I/O completes, returns to **Queue 0** (same level)
- t_2 has 4ms quota, Q0 quota is 4ms
- Both will exhaust together!

RSDL Example: t_2 Runs, Q0 Exhausts (Step 6)



- t_2 runs 4ms, quota: $4 \rightarrow 0$, **Q0 quota: $4 \rightarrow 0$**
- Thread quota exhausted $\rightarrow t_2$ demoted, quota refreshed to 6
- Q1 quota stays 6 (NOT increased by demotion)

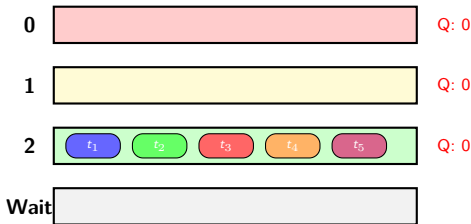
RSDL Example: Q1 Runs, Q1 Exhausts (Step 7)



- Q1 has 4 threads (24ms quota) but only 6ms queue quota!
- After t_4 runs 3ms, t_1 runs 3ms → **Q1 quota exhausted**
- **All Q1 threads mass-demoted to Q2, quotas refreshed**

RSDL Example: Epoch Ends (Step 8)

Epoch Ends!



! Epoch End Conditions

Epoch ends when:

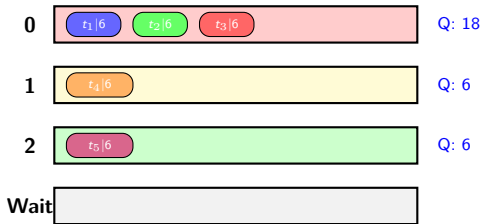
- All queue quotas exhausted (no more runnable time), OR
- No runnable threads (all waiting on I/O)

RSDL Example: Epoch Reset (Step 9)

New Epoch!



reset



💡 Epoch Reset

- All thread quotas reset to 6ms
- All threads return to **original starting priorities**
- Queue quotas reset: $Q_0=18$, $Q_1=6$, $Q_2=6$

Concluding Questions

General Scheduling Concepts (1/2)

True or False

- The CPU scheduler is responsible for determining which thread gets to use the CPU next.

General Scheduling Concepts (1/2)

True or False

- The CPU scheduler is responsible for determining which thread gets to use the CPU next.

Answer: True

General Scheduling Concepts (1/2)

True or False

- The CPU scheduler is responsible for determining which thread gets to use the CPU next.

Answer: True

- A context switch occurs only when a thread voluntarily yields the CPU.

General Scheduling Concepts (1/2)

True or False

- The CPU scheduler is responsible for determining which thread gets to use the CPU next.

Answer: True

- A context switch occurs only when a thread voluntarily yields the CPU.

Answer: False

General Scheduling Concepts (2/2)

True or False

- The goal of scheduling policies is to maximize fairness, responsiveness, and CPU utilization.

General Scheduling Concepts (2/2)

True or False

- The goal of scheduling policies is to maximize fairness, responsiveness, and CPU utilization.

Answer: True

General Scheduling Concepts (2/2)

True or False

- The goal of scheduling policies is to maximize fairness, responsiveness, and CPU utilization.

Answer: True

- Preemptive scheduling allows the scheduler to interrupt a running thread before it completes execution.

General Scheduling Concepts (2/2)

True or False

- The goal of scheduling policies is to maximize fairness, responsiveness, and CPU utilization.

Answer: True

- Preemptive scheduling allows the scheduler to interrupt a running thread before it completes execution.

Answer: True

General Scheduling Concepts (2/2)

True or False

- The goal of scheduling policies is to maximize fairness, responsiveness, and CPU utilization.

Answer: True

- Preemptive scheduling allows the scheduler to interrupt a running thread before it completes execution.

Answer: True

- The quantum in preemptive scheduling is the maximum amount of time a thread can run before being preempted.

General Scheduling Concepts (2/2)

True or False

- The goal of scheduling policies is to maximize fairness, responsiveness, and CPU utilization.

Answer: True

- Preemptive scheduling allows the scheduler to interrupt a running thread before it completes execution.

Answer: True

- The quantum in preemptive scheduling is the maximum amount of time a thread can run before being preempted.

Answer: True

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.

Answer: False

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.

Answer: False

- FCFS scheduling may lead to the convoy effect, where short threads are delayed by long threads.

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.

Answer: False

- FCFS scheduling may lead to the convoy effect, where short threads are delayed by long threads.

Answer: True

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.
Answer: False
- FCFS scheduling may lead to the convoy effect, where short threads are delayed by long threads.
Answer: True
- SJF scheduling always ensures fair CPU time distribution among all threads.

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.
Answer: False
- FCFS scheduling may lead to the convoy effect, where short threads are delayed by long threads.
Answer: True
- SJF scheduling always ensures fair CPU time distribution among all threads.
Answer: False

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.
Answer: False
- FCFS scheduling may lead to the convoy effect, where short threads are delayed by long threads.
Answer: True
- SJF scheduling always ensures fair CPU time distribution among all threads.
Answer: False
- Preemptive SJF (SRTF) can prevent long threads from experiencing starvation.

Scheduling Algorithms (1/3)

True or False

- FCFS scheduling is optimal for minimizing the average waiting time in all scenarios.

Answer: False

- FCFS scheduling may lead to the convoy effect, where short threads are delayed by long threads.

Answer: True

- SJF scheduling always ensures fair CPU time distribution among all threads.

Answer: False

- Preemptive SJF (SRTF) can prevent long threads from experiencing starvation.

Answer: False

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

- In Round Robin scheduling, using an extremely small quantum reduces context switch overhead.

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

- In Round Robin scheduling, using an extremely small quantum reduces context switch overhead.

Answer: False

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

- In Round Robin scheduling, using an extremely small quantum reduces context switch overhead.

Answer: False

- MLFQ prioritizes interactive (I/O-bound) tasks by assigning them higher priority queues.

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

- In Round Robin scheduling, using an extremely small quantum reduces context switch overhead.

Answer: False

- MLFQ prioritizes interactive (I/O-bound) tasks by assigning them higher priority queues.

Answer: True

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

- In Round Robin scheduling, using an extremely small quantum reduces context switch overhead.

Answer: False

- MLFQ prioritizes interactive (I/O-bound) tasks by assigning them higher priority queues.

Answer: True

- In MLFQ, each queue has a constant quantum, with higher priority queues having smaller quanta.

Scheduling Algorithms (2/3)

True or False

- Round Robin scheduling ensures that each thread gets an equal share of CPU time in a cyclic order.

Answer: True

- In Round Robin scheduling, using an extremely small quantum reduces context switch overhead.

Answer: False

- MLFQ prioritizes interactive (I/O-bound) tasks by assigning them higher priority queues.

Answer: True

- In MLFQ, each queue has a constant quantum, with higher priority queues having smaller quanta.

Answer: True

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

- In RSDL, when a thread blocks for I/O, it is demoted to the next lower queue.

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

- In RSDL, when a thread blocks for I/O, it is demoted to the next lower queue.

Answer: False (stays at same level)

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

- In RSDL, when a thread blocks for I/O, it is demoted to the next lower queue.

Answer: False (stays at same level)

- In RSDL, when a queue's quota is exhausted, all threads in that queue are demoted together.

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

- In RSDL, when a thread blocks for I/O, it is demoted to the next lower queue.

Answer: False (stays at same level)

- In RSDL, when a queue's quota is exhausted, all threads in that queue are demoted together.

Answer: True

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

- In RSDL, when a thread blocks for I/O, it is demoted to the next lower queue.

Answer: False (stays at same level)

- In RSDL, when a queue's quota is exhausted, all threads in that queue are demoted together.

Answer: True

- In RSDL, at the end of an epoch, all thread quotas reset and threads return to their starting priority.

Scheduling Algorithms (3/3)

True or False

- MLFQ scheduling guarantees that all threads will eventually receive CPU time, preventing starvation completely.

Answer: False

- In RSDL, when a thread blocks for I/O, it is demoted to the next lower queue.

Answer: False (stays at same level)

- In RSDL, when a queue's quota is exhausted, all threads in that queue are demoted together.

Answer: True

- In RSDL, at the end of an epoch, all thread quotas reset and threads return to their starting priority.

Answer: True

Open Question: Autonomous Drone Scheduling

Scenario

An autonomous delivery drone must schedule the following tasks efficiently:

- **Obstacle Detection (OD)** – Must process data within 50ms to avoid collisions
- **Route Calculation (RC)** – Runs every 2 seconds but can tolerate slight delays
- **Battery Monitoring (BM)** – Runs every 5 seconds and is the least time-sensitive

Open Question: Autonomous Drone Scheduling

? Questions

- Which scheduling algorithm would you use, and why?
- How do you ensure that OD always meets its deadline?
- What happens if RC and BM take too long?
- Would you use preemptive or non-preemptive scheduling?

Open Question: Autonomous Drone Scheduling

✓ Answer

- **MLFQ is the best choice:** OD gets the highest-priority queue, ensuring it runs first. RC and BM are in lower queues.
- **Preemption ensures OD meets its 50ms deadline:** Lower-priority threads are preempted if OD needs CPU time.
- **RC and BM are scheduled fairly:** RC runs in a mid-priority queue and BM in the lowest queue, with priority boosting to prevent starvation.
- **Why not RSDL?** RSDL enforces fairness through epoch resets, but doesn't guarantee OD always runs first, making it unsuitable for real-time constraints.
- **Dynamic priority adjustments:** If RC or BM is delayed too often, priority boosting ensures they eventually execute without delaying OD.

Scheduling Performance Question

Problem Setup

Three threads with arrival times, CPU bursts, and I/O:

Thread	Arrival	CPU ₁	I/O	CPU ₂
T_1	0	3	2	2
T_2	1	3	—	—
T_3	2	4	3	4

(T_2 has no I/O; times in ms; total burst: $T_1=5$, $T_2=3$, $T_3=8$)

Scheduling Performance Question

⚙ Parameters

- **Round Robin:** quantum = 2ms
- **Modified MLFQ:**
 - Q1 quantum = 2ms, Q2 quantum = 4ms, Q3 FCFS policy
 - New threads start in Q1; demote on full-quantum; stay on I/O block
- **Modified RSDL:**
 - Thread quotas: $T_1=5$, $T_2=3$, $T_3=8$
 - Queue quotas = sum of thread quotas; RR Interval = 2ms
 - Demote on quota exhaustion; stay on I/O block; reset at epoch end

Scheduling Performance Question

Tasks

Compute the following for each algorithm:

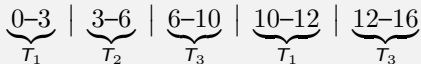
- 1 Completion time
- 2 Turnaround time (completion – arrival)
- 3 Waiting time (turnaround – burst)

For each of these algorithms:

- 1 First-Come, First-Serve (FCFS)
- 2 Round Robin
- 3 Modified Multi-Level Feedback Queue
- 4 Modified RSDL

FCFS Solution (with I/O)

Gantt Chart



T_1 I/O: 3-5, T_3 I/O: 10-13

FCFS Solution (with I/O)

✓ Solution

➤ **Completion times:** $C_1 = 12$, $C_2 = 6$, $C_3 = 16$

➤ **Turnaround times** $T_i = C_i - A_i$:

$$T_1 = 12 - 0 = 12, \quad T_2 = 6 - 1 = 5, \quad T_3 = 16 - 2 = 14$$

➤ **Waiting times** $W_i = T_i - B_i - IO_i$:

$$W_1 = 12 - 5 - 2 = 5, \quad W_2 = 5 - 3 = 2, \quad W_3 = 14 - 8 - 3 = 3$$

Round Robin (Quantum = 2ms, with I/O)

Execution Trace

- 0–2: T_1 (rem 1), 2–4: T_2 (rem 1), 4–6: T_3 (rem 2)
- 6–7: T_1 finishes CPU₁ → I/O (7–9)
- 7–8: T_2 finishes, 8–10: T_3 (rem 0) → I/O (10–13)
- 10–12: T_1 returns, finishes CPU₂
- 13–17: T_3 returns, runs CPU₂ (4ms)

Round Robin (Quantum = 2ms, with I/O)

✓ Solution

➤ **Completion:** $C_1 = 12$, $C_2 = 8$, $C_3 = 17$

➤ **Turnaround:** $T_1 = 12$, $T_2 = 7$, $T_3 = 15$

➤ **Waiting:**

$$W_1 = 12 - 5 - 2 = 5, \quad W_2 = 7 - 3 = 4, \quad W_3 = 15 - 8 - 3 = 4$$

Multi-Level Feedback Queue (with I/O)

Execution Steps

- 1 Q1 (2ms each):** T_1 0–2 (rem 1) $\downarrow$ Q2, T_2 2–4 (rem 1) $\downarrow$ Q2, T_3 4–6 (rem 2) $\downarrow$ Q2
- 2 Q2:** T_1 6–7 $\rightarrow$ I/O (7–9), stays Q2; T_2 7–8 fin; T_3 8–10 $\rightarrow$ I/O (10–13), stays Q2
- 3 Q2 cont:** T_1 returns 9, runs 9–11 fin; T_3 returns 13, runs 13–17 fin

✓ Solution

Completion: $C_1 = 11$, $C_2 = 8$, $C_3 = 17$

Turnaround: $T_1 = 11$, $T_2 = 7$, $T_3 = 15$

Waiting: $W_1 = 11 - 5 - 2 = 4$, $W_2 = 7 - 3 = 4$, $W_3 = 15 - 8 - 3 = 4$

RSDL (Modified, with I/O) – Part 1

Parameters

Thread quotas: $T_1=5$, $T_2=3$, $T_3=8$. Queue quota = sum of thread quotas = 16.

RR Interval = 2ms. Three queues: High, Medium, Low.

Execution Trace

- 1 **High queue, RR:** T_1 0–2 (quota 5→3), T_2 2–4 (quota 3→1), T_3 4–6 (quota 8→6)
- 2 T_1 6–7 → I/O (7–9), **stays High** (I/O block)
- 3 T_2 7–8 finishes (quota=0)
- 4 T_3 8–10 → I/O (10–13), **stays High**

RSDL (Modified, with I/O) – Part 2

Execution Trace (Continued)

- 5 T_1 returns at 9, runs 9–11 (finishes CPU₂, quota exhausted)
- 6 T_3 returns at 13, runs 13–17 (finishes CPU₂)

Note: T_1 and T_3 stayed in High queue due to I/O blocking rule.

✓ Solution

$$C_1 = 11, \quad \text{Turnaround}_1 = 11, \quad W_1 = 11 - 5 - 2 = 4$$

$$C_2 = 8, \quad \text{Turnaround}_2 = 7, \quad W_2 = 7 - 3 = 4$$

$$C_3 = 17, \quad \text{Turnaround}_3 = 15, \quad W_3 = 15 - 8 - 3 = 4$$

Recitation 5

Memory Management

Daniel Karalnik Liam Tal

Spring Semester 2026



Table of Contents

- 1 Intro to Physical Memory
- 2 The Evolution of Memory Management
- 3 Virtual Memory
- 4 Virtual Memory to Physical Memory Mapping
- 5 The Memory Management Unit (MMU)
- 6 Paging
- 7 Page Swapping

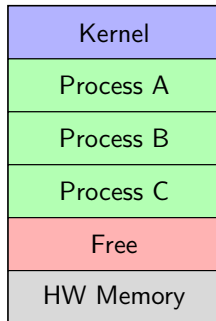
Intro to Physical Memory

What is Physical Memory?

Physical Memory

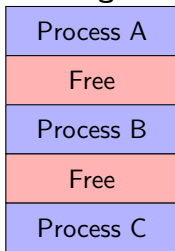
The actual RAM installed in a computer, organized as a **contiguous address space** accessible by the CPU.

- Managed by the **OS** to allocate space for processes and kernel
- Divided into **fixed-size frames** or **variable partitions**
- Addresses are **physical locations** in the RAM chip

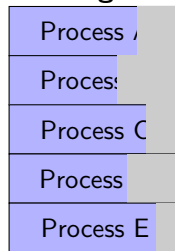


External vs Internal Fragmentation

External Fragmentation



Internal Fragmentation



Warning

Gaps between blocks. Total free space is enough, but no single gap fits a large request.

Warning

Wasted space within blocks. Fixed-size allocation wastes the unused portion.

Why Not Direct Physical Addressing?

Problems with Direct Access

If each process accessed physical memory directly, we would face serious issues.

- **No Process Isolation:** Process A could overwrite Process B's memory
- **No Relocation:** Programs must know exact physical addresses at compile time
- **Fragmentation:** Memory becomes scattered and unusable over time
- **Limited Multiprogramming:** Hard to run multiple programs efficiently

The Evolution of Memory Management

What are Virtual and Physical Addresses?

Physical Address

The actual location in RAM hardware. Example: byte 0x5000 refers to a specific cell in the RAM chip.

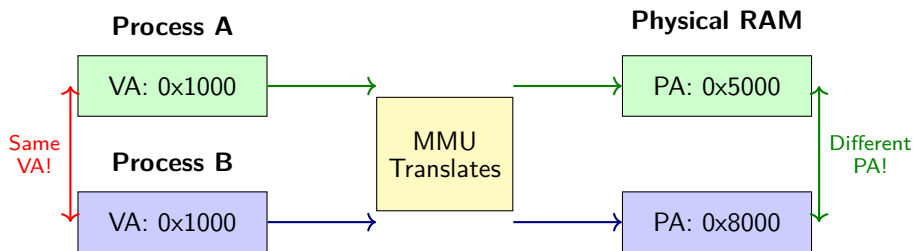
Virtual Address

An address used by programs. The hardware translates it to a physical address before accessing RAM.

What are Virtual and Physical Addresses?

- Programs **only see virtual addresses** - they never access physical RAM directly
- Hardware (MMU) **translates** every memory access automatically
- Each process has its **own virtual address space** starting at 0
- The **same virtual address** in different processes maps to **different physical locations**

How Virtual Addresses Enable Isolation



💡 Benefits of Virtual Addresses

Isolation: Processes cannot access each other's memory

Flexibility: OS can place data anywhere in physical RAM

Simplicity: Every process thinks it has memory starting at address 0

Why do we bulk translate memory addresses?



Translation Efficiency

If we translated each virtual address individually, it would require frequent lookups, significantly increasing memory access time.

- Individual translation introduces considerable overhead
- Excessive context switches and performance degradation
- **Bulk translation** returns a **set of relevant addresses** (concept of locality)
- Reduces translation frequency and minimizes CPU stalls



Key Insight

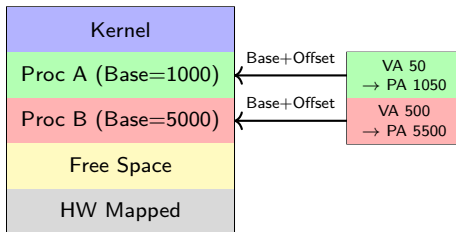
Bulk translation allows each translation to cover an entire page (bulk of address), not just a single address!

Base and Bound Addressing

Base and Bound

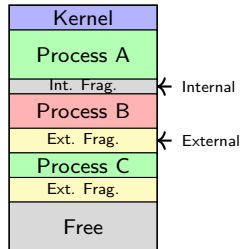
Early technique using two registers to define a process's memory region.

- **Base:** Where process starts
- **Bound:** Max allowed address
- **Translation:** $VA + Base = PA$



Limitations of Base and Bound

- **External Frag.:** Small gaps unusable
- **Internal Frag.:** Wasted space in blocks
- **Contiguous:** Process in one block
- **Compaction:** Costly to defragment
- **No Sharing:** Can't share libraries
- **No Growth:** Can't expand in place



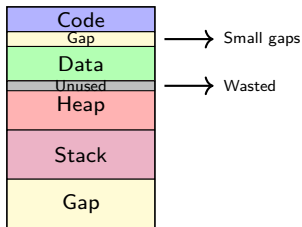
Memory Segmentation: Reducing External Fragmentation



Segmentation

Segments allow better memory utilization by fitting into available free space.

- Heap and Stack grow independently
- Segments can fit into gaps
- Process split into logical parts



Warning

External fragmentation reduced but internal fragmentation still exists within segments!

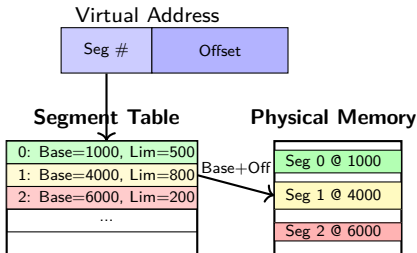
How Segmentation Works



Segment Table

Each process has a segment table with entries for each segment:

- **Base:** Starting physical address
- **Limit:** Segment size (bounds check)
- **Permissions:** R/W/X bits



Translation

Virtual Address = **Segment #** + **Offset**
Physical = **Base[Seg#]** + **Offset**
(If Offset > Limit → Seg Fault)

Why We Need Paging

⚠ Segmentation Still Has Problems

Even with segmentation, we face issues that motivate a better solution.

- **Variable-sized segments** still cause external fragmentation
- **Entire segment** must be in memory (no partial loading)
- **Segment table** grows with number of segments
- **Sharing** is complex (need matching segment sizes)

💡 The Solution: Paging

Fixed-size pages eliminate external fragmentation entirely and enable partial loading, sharing, and efficient memory management!

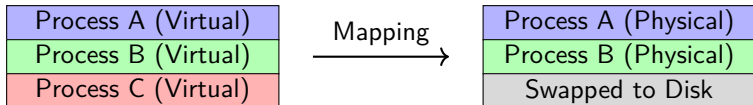
Virtual Memory

What is Virtual Memory?

Virtual Memory - Expanding upon Virtual Addresses

Virtual memory is an abstraction that allows processes to **use more memory than physically available**, providing each process its own isolated address space.

- The OS uses **paging** to map virtual addresses to physical memory
- Enables **efficient memory management**, **process isolation**, and **swapping**



Paging and Swapping

Paging

Dividing memory into fixed-size blocks called **pages** (typically 4KB).

- Virtual memory → **Pages**
- Physical memory → **Frames**
- **Page table** maps pages to frames
- Eliminates **external fragmentation**

Swapping

Moving pages between RAM and disk when physical memory is full.

- **Page out:** RAM → Disk
- **Page in:** Disk → RAM
- Triggered by **page faults**
- Enables using more memory than available RAM

Page States: Clean and Dirty

✓ Clean Page

Unmodified—matches its backing store.

- Can be **discarded** anytime
- Just reload from backing store if needed

✎ Dirty Page

Modified in memory—differs from backing store.

- **Must be saved** before discard
- Written to swap or file

Anonymous Pages (heap, stack)

`malloc()` → **Clean** (no backing store yet) → Write → **Dirty**

Evict dirty → Write to **swap**, allocate swap slot

Load from swap → **Dirty** + swap slot **freed** (must re-swap if evicted again)

32-bit vs 64-bit Systems

32-bit CPUs

- Address up to **4GB** RAM
- 32-bit OS runs **only** 32-bit programs

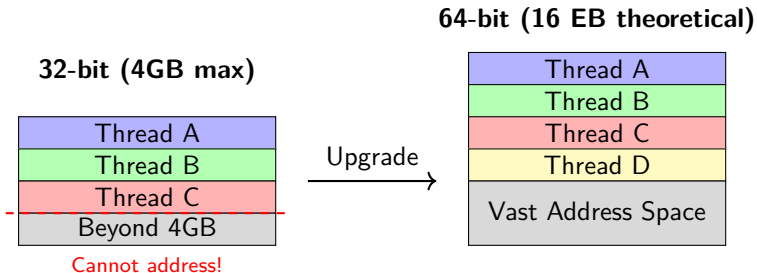
64-bit CPUs

- Theoretically **16 exabytes**
- **Wider registers**, better multi-threading
- Runs **both** 32-bit and 64-bit programs

Performance

64-bit CPUs support wider registers, more memory, and better multi-threading capabilities.

32-bit vs 64-bit Virtual Memory



Types of Virtual Memory



Anonymous Memory

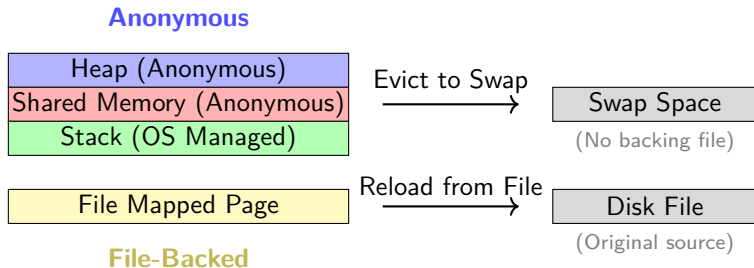
- **Not associated with a file**
- Used for **heap, shared memory, stack**
- Allocated via `mmap(MAP_ANONYMOUS)`
- When paged out, stored in **swap space**



File-Backed Memory

- Mapped from a **file on disk**
- Used for **executables, shared libraries**
- When paged out, can be **reloaded from file**

Virtual Memory Diagram



What is mmap()?

Memory Mapping

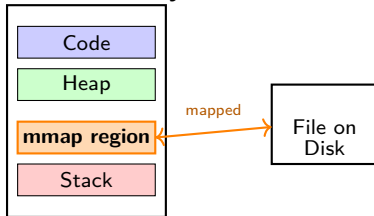
`mmap()` maps files or memory directly into a process's virtual address space.

Instead of `read()/write()`:

- File contents appear as **memory**
- Read/write using **pointers**
- No explicit I/O syscalls after mapping

```
ptr = mmap(fd, ...);  
char c = ptr[0]; // reads file  
ptr[1] = 'A';    // writes file
```

Virtual Memory



Key Insight

`mmap()` returns a pointer—use it like any memory!

The Page Cache

Page Cache

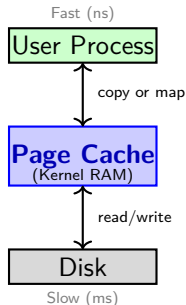
Kernel memory that **caches file data** in RAM for faster access.

Why cache file data?

- Disk access: **milliseconds**
- RAM access: **nanoseconds**
- Avoid repeated disk reads

Key Insight

Both `read()/write()` and `mmap()` use the **same page cache**!



Dirty pages in cache must be written back to disk (writeback).

Buffered I/O vs. Memory-Mapped I/O: Read & Write



Buffered I/O

READ — `read(fd, buf, n)`

- Disk → Page Cache
- Page Cache → **copy** to user buffer

WRITE — `write(fd, buf, n)`

- User buffer → **copy** to Page Cache
- Page marked **dirty**
- `fsync()` forces write to disk



Memory-Mapped I/O

READ — access `ptr[i]`

- Page fault → load to Page Cache
- **Direct access** (no copy)

WRITE — `ptr[i] = x`

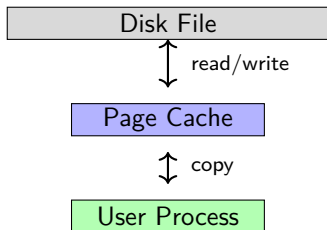
- Modify Page Cache **directly**
- Page marked **dirty**
- `msync()` forces write to disk

2 copies per operation

Zero-copy access

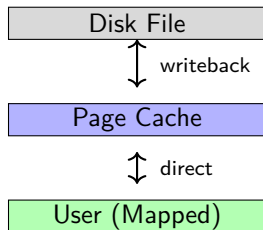
Buffered I/O vs. Memory-Mapped I/O (Diagram)

Buffered I/O



2 copies per I/O

Memory-Mapped I/O



Zero-copy access

💡 Writeback

Dirty pages are written to disk by kernel **periodically** or when explicitly requested (`fsync/msync`).

Private vs. Shared Memory in Virtual Memory



Private Memory

- Exclusive to a process
- Heap (`malloc`)
- Stack



Shared Memory

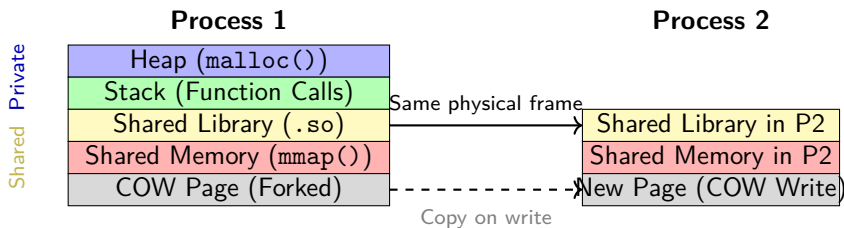
- Dynamic libraries
- `mmap()` regions
- IPC mechanisms



Copy-on-Write (COW)

Optimizes `fork()`: pages are **shared until modified**, saving memory and time!

Private vs. Shared Memory in Virtual Memory



Virtual Memory to Physical Memory Mapping

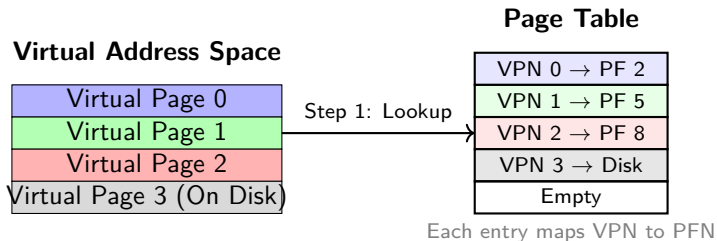
Virtual to Physical Address Translation

Address Translation

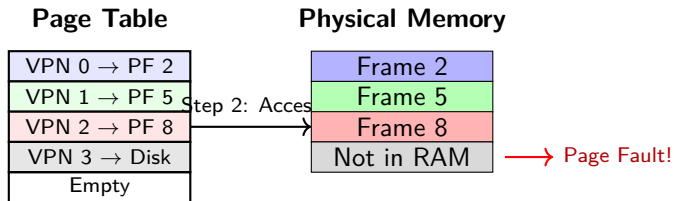
The OS and MMU (Memory Management Unit) work together to translate virtual addresses to physical addresses.

- The **Page Table** stores mappings of **VPNs** to **PFNs**
- A **TLB** (Translation Lookaside Buffer) caches recent translations
- If a page is **not in memory**, a **page fault** occurs

Virtual to Physical Address Translation



Virtual to Physical Address Translation



The Memory Management Unit (MMU)

What is the MMU?

Memory Management Unit

The **MMU** is a hardware component (part of the CPU) that translates virtual addresses to physical addresses on every memory access.

- Performs address translation **automatically in hardware**
- Contains the **TLB** (Translation Lookaside Buffer) for caching
- Enforces **memory protection** (R/W/X permissions)
- Triggers **page faults** when pages are not present

Key Insight

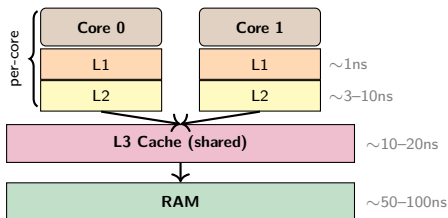
The OS sets up page tables, but the MMU does the actual translation on every memory access!

CPU Cache Levels

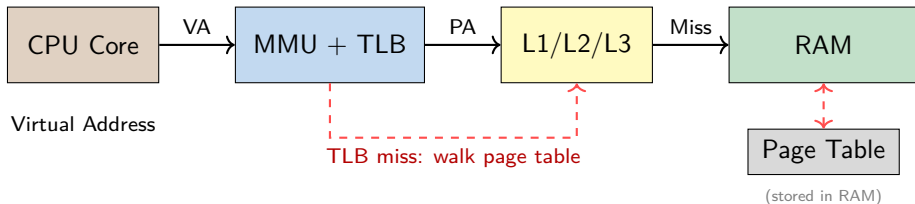
CPU Cache

Small, fast memory built into the CPU that stores recently accessed data to avoid slow RAM access. (chunks that are smaller than pages usually)

- **L1:** Per-core, fastest ($\sim 1\text{ns}$), smallest (KB)
- **L2:** Per-core, fast ($\sim 3\text{--}10\text{ns}$), larger (MB)
- **L3:** Shared across cores ($\sim 10\text{--}20\text{ns}$), largest (MB)



MMU in the Memory Hierarchy



💡 Key Insight

The MMU translates every VA to PA before it reaches the cache. On TLB miss, the page table walk **also goes through the cache**—so cached page table entries speed up translation!

TLB and Context Switches

The Problem

When the OS switches from Process A to Process B, the TLB contains A's translations - these are **invalid** for B!

TLB Flush

- Clear entire TLB on switch
- Simple but expensive
- Many TLB misses after switch

ASID/PCID Tags

- Tag entries with process ID
- No flush needed
- TLB can hold multiple processes

Paging

What Are Pages?

Paging

Paging divides memory into fixed-size **pages** (virtual memory) and **frames** (physical memory), eliminating external fragmentation.

- Allows **non-contiguous allocation**
- **Page Table** maps virtual pages to physical frames
- Enables **sharing of common memory pages** (e.g., shared libraries)

Key Insight

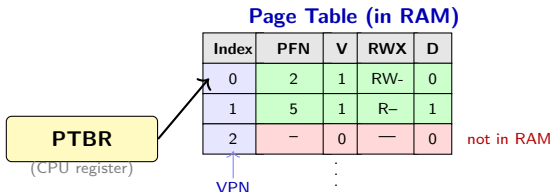
Paging is the foundation of modern virtual memory systems!

Page Table Structure & PTBR

Page Table

An array in RAM, indexed by VPN. Each entry (PTE) holds the PFN and metadata.

- **PTBR**: register pointing to table base in RAM
- **VPN** = index into the array
- **OS** creates the table
- **MMU** reads it



What is in a Page Table Entry (PTE)?

Page Table Entry

A PTE stores the physical frame number and metadata for a virtual page. The MMU reads these fields on every address translation.

Field	Purpose
Page Frame Number (PFN)	Physical frame where the page is stored
Valid Bit	1 if the mapping exists (OS allocated it); 0 → page fault
Present Bit	1 if page is currently in RAM; 0 → page fault (may be on disk)
RWX Bits	Access permissions: read, write, execute
Dirty Bit	1 if page was modified in memory; must be written back before eviction
Accessed Bit	Set by MMU on access; helps OS decide which pages to evict
User/Supervisor Bit	Controls user-mode vs. kernel-mode access

Page Sizes and Virtual Address Lookup (64-bit)

Address Splitting

Virtual addresses are split into VPN (Virtual Page Number) and Offset.

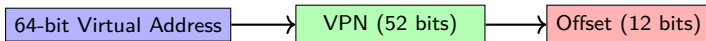
- **Larger pages** → Smaller page tables, but more internal fragmentation
- **Smaller pages** → More entries, increasing TLB pressure

4KB Pages

VPN = 52 bits, Offset = 12 bits

2MB Pages

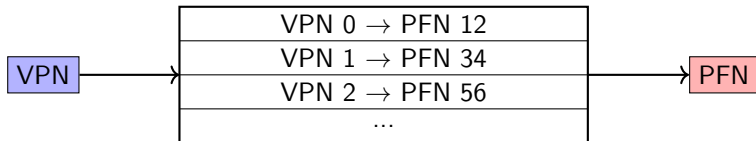
VPN = 43 bits, Offset = 21 bits



Page Table as an Array

☰ Array-Based Page Table

Each process has an array-based page table mapping VPN to PFN. Direct indexing makes lookups fast.



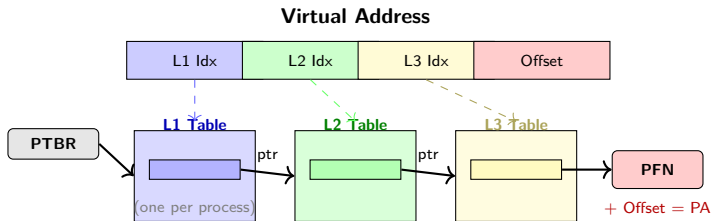
⚠ Scalability

Table size is too large in 64-bit systems! This leads to multi-level page tables.

Multi-Level Page Tables

Why Multi-Level?

Single-level tables too large for 64-bit. Multi-level breaks them into chunks—only **needed parts** are stored in memory.



Page Table Walk Example (2-Level) — Setup

Given

Virtual Address: 0x00403004, Page Size: 4KB, 32-bit system, 10-10-12 split

Step 1: Split the Address

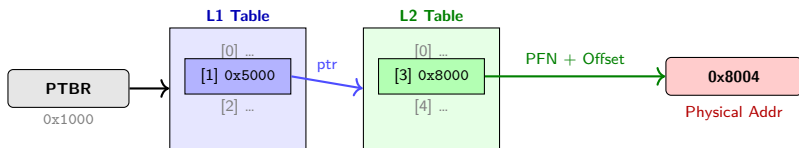
0x00403004 in binary (32 bits):

L1 Index (10 bits)	L2 Index (10 bits)	Offset (12 bits)
0000000001	0000000011	000000000100
= 1	= 3	= 0x004

Page Table Walk Example (2-Level) — Walk

✓ Step 2: Walk the Page Tables

- 1 PTBR points to L1 table at physical address 0x1000
- 2 $L1[1] = 0x5000$ (points to L2 table)
- 3 $L2[3] = 0x8000$ (physical frame number)
- 4 Physical Address = $0x8000 + 0x004 = 0x8004$



Page Swapping

What is Swap Space?

Swap Space

Swap space is a dedicated area on disk (partition or file) used to store pages that have been evicted from RAM. It extends virtual memory beyond physical RAM limits.

- Located on disk (much slower than RAM)
- Used for **anonymous pages** (heap, stack) when evicted
- File-backed pages don't need swap - they reload from original file
- Size typically 1-2x physical RAM (varies by system)

Performance

Excessive swapping indicates memory pressure. If swap is constantly used, consider adding more RAM!

Demand Paging

Demand Paging

Pages are loaded into memory **only when accessed**, not when the program starts. This is also called **lazy loading**.

- Process starts with **no pages in memory**
- First access to a page triggers a **page fault**
- OS loads the page from disk and resumes execution
- Enables running programs **larger than physical memory**

Benefit

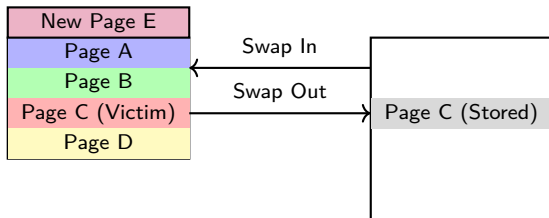
Programs often don't use all their code/data. Why load pages that may never be accessed?

Swapping Pages In and Out of Memory

Page Swapping

When RAM is full, inactive pages are swapped to disk. Page faults trigger swapping pages back into RAM.

- OS selects a **victim page** using an algorithm (LRU, FIFO, Clock)
- **Dirty pages** must be written back to disk before eviction



FIFO Page Replacement

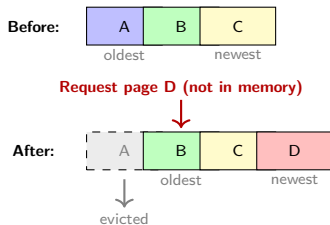
First-In-First-Out

The oldest page in memory is evicted first, regardless of how recently it was used.

- Simple to implement (just a queue)
- Does not consider page usage
- Can evict frequently-used pages

Example: 3 frames

Reference: A, B, C, D



Belady's Anomaly

Belady's Anomaly

With FIFO, **more frames can lead to more page faults!** This counterintuitive behavior doesn't occur with LRU or Clock.

The Counterintuitive Result

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Belady's Anomaly: A Runtime Example

3 Frames

Ref	F1	F2	F3	Fault?
1	1	-	-	✓
2	1	2	-	✓
3	1	2	3	✓
4	4	2	3	✓
1	4	1	3	✓
2	4	1	2	✓
5	5	1	2	✓
1	5	1	2	✗
2	5	1	2	✗
3	5	3	2	✓
4	5	3	4	✓
5	5	3	4	✗

9 Page Faults

4 Frames

Ref	F1	F2	F3	F4	Fault?
1	1	-	-	-	✓
2	1	2	-	-	✓
3	1	2	3	-	✓
4	1	2	3	4	✓
1	1	2	3	4	✗
2	1	2	3	4	✗
5	5	2	3	4	✓
1	5	1	3	4	✓
2	5	1	2	4	✓
3	5	1	2	3	✓
4	4	1	2	3	✓
5	4	5	2	3	✓

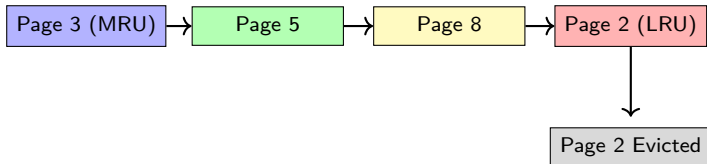
10 Page Faults!

LRU Page Replacement (Linked List Implementation)

☰ LRU Algorithm

LRU evicts the least recently used page using a doubly linked list.

- **Front** → Most recently used page
- **Tail** → Least recently used page (evicted on fault)
- Accessed pages are moved to the front



Clock Algorithm (Approximate LRU)

Clock Algorithm

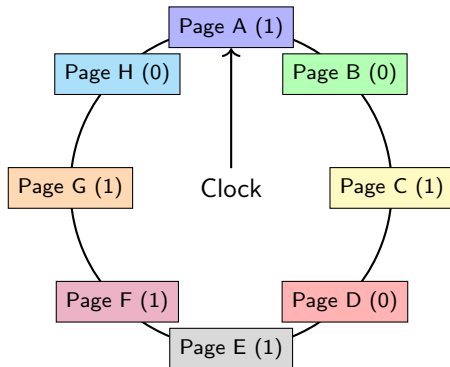
Clock Algorithm approximates LRU with lower overhead using a circular queue with reference bits.

- 1 Check the page at the clock hand
- 2 If **reference bit** = 1, set it to 0 and move the hand
- 3 If **reference bit** = 0, replace the page

Why Clock over LRU?

- LRU requires updating a linked list on **every memory access** - too expensive!
- Clock only updates on **page faults** - much lower overhead
- Clock approximates LRU behavior with near-constant time operations

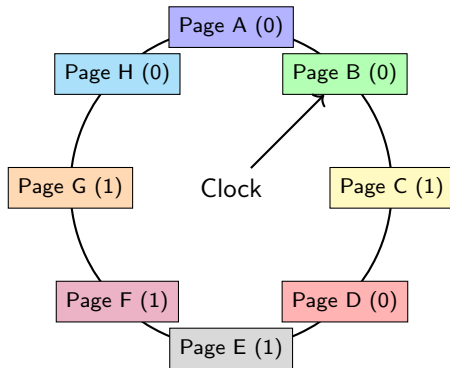
Clock Algorithm Example Part 1



💡 Key Insight

Numbers in parentheses are reference bits. Clock hand moves until it finds a 0!

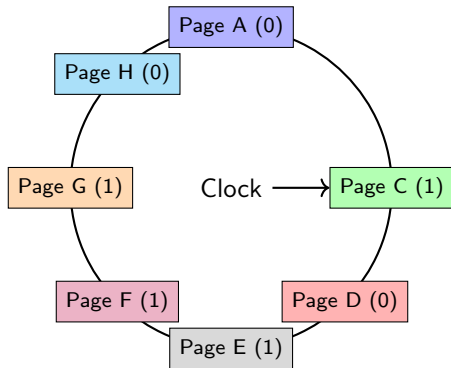
Clock Algorithm Example Part 2



✓ Step 1

Page A had reference bit = 1, so we set it to 0 and move to Page B.

Clock Algorithm Example Part 3



✓ Step 2

Page B had reference bit = 0, so it was **evicted!** Hand moves to Page C.

Working Set Model

Working Set

The **working set** of a process is the set of pages it has referenced in the last Δ time units (the working set window).

- Based on **locality of reference**: programs tend to access the same pages repeatedly
- If working set fits in available frames $\rightarrow$ few page faults
- If working set exceeds available frames $\rightarrow$ thrashing

Thrashing

Thrashing

Thrashing occurs when the system spends more time handling page faults than executing useful work.

Symptoms:

- High page fault rate
- Low CPU utilization (waiting on disk I/O)
- Disk constantly busy
- System becomes unresponsive

What is Effective Access Time (EAT)?

Effective Access Time

The **EAT** is the average time required to access memory, accounting for fast and slow cases:

- TLB hits vs. TLB misses (requiring page table lookup)
- Page faults (requiring disk access)

What is Effective Access Time (EAT)?



General Formula

$$\text{EAT} = (1 - p) \times (\alpha T_{\text{hit}} + (1 - \alpha) T_{\text{miss}}) + p \times T_{\text{fault}}$$



Variables

α TLB hit ratio

p Page fault rate

T_{hit} Access time on TLB hit

T_{miss} Access time on TLB miss

T_{fault} Service time on page fault

Solution Part 1: TLB Hits and Misses

✓ Calculating Access Times

$$T_{\text{hit}} = T_{\text{TLB}} + T_{\text{mem}} = 20 + 100 = 120 \text{ ns}$$

$$T_{\text{miss}} = T_{\text{TLB}} + 2 \times T_{\text{mem}} = 20 + 2 \times 100 = 220 \text{ ns}$$

$$\text{EAT}_{\neg\text{fault}} = 0.9 \times 120 + 0.1 \times 220 = 130 \text{ ns}$$

Solution Part 2: Incorporating Page Faults

Given

Using $p = 0.001$ and $T_{\text{fault}} = 10 \text{ ms} = 10 \times 10^6 \text{ ns}$

Final Calculation

$$\begin{aligned} \text{EAT} &= (1 - p) \times 130 + p \times 10 \times 10^6 \\ &= 0.999 \times 130 + 0.001 \times 10,000,000 \\ &\approx 10,129.87 \text{ ns} \approx 10.13 \mu\text{s} \end{aligned}$$

$$\begin{aligned} \text{Total time} &= 10,000 \times 10,129.87 \text{ ns} \\ &= 1.012987 \times 10^8 \text{ ns} \approx 0.1013 \text{ s} \end{aligned}$$

Impact of Page Fault Rate

Small Changes, Big Impact

Even tiny increases in page fault rate dramatically increase effective access time.

Page Fault Rate (p)	EAT	Slowdown
0 (no faults)	130 ns	1x
0.0001 (1 in 10,000)	1,130 ns	9x
0.001 (1 in 1,000)	10,130 ns	78x
0.01 (1 in 100)	100,130 ns	770x



Exercises

Exercise 1: Page Table Size Calculation

Page Table Size

Given a system with:

- 32-bit virtual address space
- 4KB page size
- Each PTE is 4 bytes

Calculate:

- 1 How many pages in the virtual address space?
- 2 Size of a single-level page table?
- 3 Why is this problematic?

Exercise 1: Solution

✔ Page Table Size Solution

- Page size = 4KB = 2^{12} bytes $\rightarrow$ Offset = 12 bits
- VPN = $32 - 12 = 20$ bits
- Number of pages = $2^{20} = 1,048,576$ pages
- Page Table Size = $2^{20} \times 4$ bytes = **4MB per process**

⚠ Problem

Every process needs 4MB just for its page table! With 100 processes, that's 400MB of page tables alone. This is why we use **multi-level page tables**.

Exercise 2: VPN and Offset Calculation

Exercise

Given:

- 32-bit virtual address: 0x00005A4B
- Page size: 4KB

Calculate the VPN and Offset.

Exercise 2: VPN and Offset Calculation - Solution

✓ Solution

- Page size = 4KB = 2^{12} bytes $\rightarrow$ Offset = 12 bits
- 0x00005A4B = 0000 0000 0000 0000 0101 1010 0100 1011
- Offset = lower 12 bits = 0xA4B = 2635
- VPN = upper 20 bits = 0x00005 = 5

Exercise 3: Multi-Level Address Splitting

Exercise

Given:

- 32-bit virtual address: 0x05307100
- Page size: 4KB
- 3-level page table: L1 has 256 entries, L2 has 64 entries, L3 has 64 entries

Calculate the number of index bits per level, and split the address into L1, L2, L3 indices and page offset.

Exercise 3: Multi-Level Address Splitting - Solution

✓ Solution

- Index bits: $L1 = \log_2(256) = 8$, $L2 = \log_2(64) = 6$, $L3 = \log_2(64) = 6$
- Offset = $\log_2(4KB) = 12$ bits. Verify: $8 + 6 + 6 + 12 = 32$ ✓

0x05307100 = 0000 0101 0011 0000 0111 0001 0000 0000

L1 (8 bits)	L2 (6 bits)	L3 (6 bits)	Offset (12 bits)
00000101	001100	000111	000100000000
= 5	= 12	= 7	= 0x100

Exercise 4: 4D Array Memory Access Patterns

4D Array Page Faults

Consider a 4D array declared as: `int A[8][8][8][8];`

System Parameters

- `sizeof(int) = 4 bytes`
- Page size = 64 bytes (small for illustration)
- Array stored in **row-major order** (rightmost index varies fastest)
- Array starts at page boundary
- Process has **4 page frames** available
- LRU page replacement

Exercise 4: Access Patterns

</> Pattern A: Sequential

```
// Rightmost varies
// fastest
for(i=0; i<8; i++)
  for(j=0; j<8; j++)
    for(k=0; k<8; k++)
      for(l=0; l<8; l++)
        sum += A[i][j][k][
          l];
```

</> Pattern B: Strided

```
// Leftmost varies
// fastest
for(l=0; l<8; l++)
  for(k=0; k<8; k++)
    for(j=0; j<8; j++)
      for(i=0; i<8; i++)
        sum += A[i][j][k][
          l];
```

💡 Think About It

Which pattern matches how the array is stored in memory? How does this affect page faults?

Exercise 4: Solution

✓ Memory Layout Analysis

- Total array size: $8 \times 8 \times 8 \times 8 \times 4 = 16,384$ bytes
- Elements per page: $64/4 = 16$ integers
- Total pages needed: $16,384/64 = 256$ pages
- Row-major: $A[i][j][k][l]$ and $A[i][j][k][l+1]$ are adjacent

Pattern A: ~256 faults

Sequential access: each page loaded once, 16 elements accessed before next page.

Pattern B: ~4096 faults

Strided access: jumps 512 bytes between accesses. Every access causes a fault!

Exercise 5: LRU Page Replacement Trace

Exercise

Given page reference string: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3**
With **3 frames**, trace LRU and count page faults.

Exercise 5: LRU Page Replacement Trace - Solution

✓ LRU Trace

Ref	Frame 1	Frame 2	Frame 3	Fault?
7	7	-	-	Yes
0	7	0	-	Yes
1	7	0	1	Yes
2	2	0	1	Yes (evict 7)
0	2	0	1	No
3	2	0	3	Yes (evict 1)
0	2	0	3	No
4	4	0	3	Yes (evict 2)
2	4	0	2	Yes (evict 3)
3	4	3	2	Yes (evict 0)

Total page faults: 8

Exercise 6: FIFO Page Replacement Trace

Exercise

Same reference string: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3**
With **3 frames**, trace FIFO and count page faults.

Exercise 6: FIFO Page Replacement Trace - Solution

✓ FIFO Trace

Ref	Frame 1	Frame 2	Frame 3	Fault?
7	7	-	-	Yes
0	7	0	-	Yes
1	7	0	1	Yes
2	2	0	1	Yes (evict 7)
0	2	0	1	No
3	2	3	1	Yes (evict 0)
0	2	3	0	Yes (evict 1)
4	4	3	0	Yes (evict 2)
2	4	2	0	Yes (evict 3)
3	4	2	3	Yes (evict 0)

Total page faults: 9 (vs 8 for LRU!)

Exercise 7: Clock Algorithm Trace

Exercise

Reference string: **1, 2, 3, 4, 1, 2, 5, 1, 2, 3**

With **3 frames**, trace Clock algorithm. Show reference bits.

Exercise 7: Clock Algorithm Trace - Solution Part 1

✓ Clock Trace (abbreviated)

- 1, 2, 3: Load pages, all ref bits = 1
- 4: Hand at 1 (ref=1), clear to 0, move; at 2 (ref=1), clear to 0, move; at 3 (ref=1), clear to 0, move; back at 1 (ref=0), **evict 1**, load 4
- 1: Hand at 2 (ref=0), **evict 2**, load 1
- 2: Hand at 3 (ref=0), **evict 3**, load 2
- 5: Hand at 4 (ref=1), clear, move; at 1 (ref=1), clear, move; at 2 (ref=1), clear, move; back at 4 (ref=0), **evict 4**, load 5

Exercise 7: Clock Algorithm Trace - Solution Part 2

✓ Clock Trace (abbreviated)

- 1: Already in frame → **hit**, set ref bit = 1
- 2: Already in frame → **hit**, set ref bit = 1
- 3: Hand at 1 (ref=1), clear, move; at 2 (ref=1), clear, move; at 5 (ref=1), clear, move; back at 1 (ref=0), **evict 1**, load 3

Total page faults: 8

Exercise 8: Page Table Walk with TLB

Exercise

Reference string (VPNs): **1, 2, 3, 1, 4, 1, 5, 1**

TLB: 2 entries, **LRU** replacement

Physical Frames: 4 frames, **LRU** replacement

Page Table: 2-level (L1 PT pinned in RAM, L2 PT is a regular page)

All structures start **empty** (including L2 PT—not yet loaded).

For each access:

- 1 TLB hit or miss? (miss → page table walk)
- 2 During walk: is the L2 PT page in a frame? If not → **page fault (PT)**
- 3 Is the data page in a frame? If not → **page fault (data)**

Exercise 8: Solution (Part 1)

Important

L2 PT is a **regular page** that competes with data pages for frames. A single access can cause up to 2 page faults (L2 PT + data).

Ref	TLB	PT Fault?	Data Fault?	TLB State	Frames (MRU → LRU)
1	Miss	Yes (load L2PT)	Yes (load Pg1)	{1}	[Pg1, L2PT]
2	Miss	No (L2PT in F0)	Yes (load Pg2)	{1,2}	[Pg2, L2PT, Pg1]
3	Miss	No	Yes (load Pg3)	{2,3}	[Pg3, L2PT, Pg2, Pg1]
1	Miss	No	No (Pg1 in frame)	{3,1}	[Pg1, L2PT, Pg3, Pg2]

Access 1: Cold start—L2 PT itself is not in memory! Double page fault.

Access 4 (VPN 1): TLB miss (VPN 1 was evicted from TLB at access 3), but Pg1 is still in a frame → **no page fault**. This shows **TLB miss $\neq$ page fault**.

Exercise 8: Solution (Part 2)

✓ Trace — Accesses 5–8

Ref	TLB	PT Fault?	Data Fault?	TLB State	Frames (MRU → LRU)
4	Miss	No	Yes (evict Pg2)	{1,4}	[Pg4, L2PT, Pg1, Pg3]
1	Hit	—	No	{4,1}	[Pg1, Pg4, L2PT, Pg3]
5	Miss	No	Yes (evict Pg3)	{1,5}	[Pg5, L2PT, Pg1, Pg4]
1	Hit	—	No	{5,1}	[Pg1, Pg5, L2PT, Pg4]

Total page faults: 6

1 PT fault + 5 data faults

TLB misses: 6

TLB hits: 2 (accesses 6, 8)

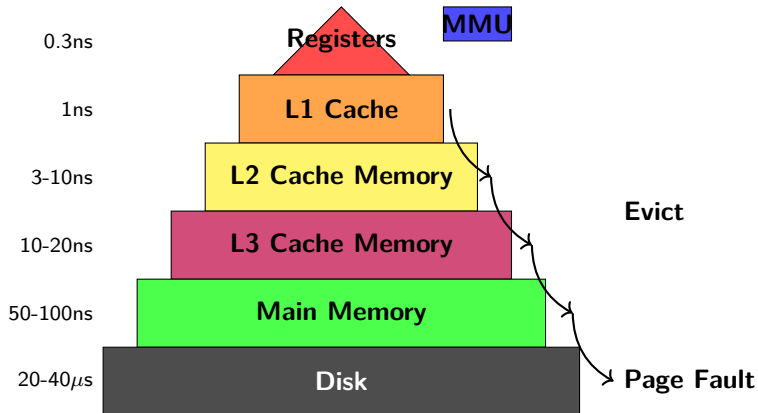
💡 Key Insight

L2 PT stays resident—every TLB miss walks the PT, refreshing L2PT's LRU position.

Accesses 6 & 8: **best case**—TLB hit, no walk, no fault.

Summary

Memory Hierarchy Pyramid



See You Next Week!

Questions?

Recitation 6

File Systems

Daniel Karalnik Liam Tal

Spring Semester 2026



Table of Contents

- 1 File System Hardware
- 2 File System Abstraction Layer
- 3 Linking
- 4 File Descriptors
- 5 Pipes
- 6 RAID

File System Hardware

What Does a Hard Disk Do?

Hard Disk Drive (HDD)

A **non-volatile** storage device that stores and retrieves digital data using **magnetic storage** on rapidly spinning platters.

- Data **persists** even without power (unlike RAM)
- Read and written using a **mechanical arm with a read/write head**
- HDDs are used in:
 - Operating systems and application storage
 - File storage (documents, images, videos)
 - Databases and large-scale data management

What is an SSD?

Solid-State Drive (SSD)

A storage device that uses **flash memory** instead of mechanical parts. Much faster than HDDs due to the lack of moving components.

- Uses NAND flash memory for data storage
- Found in laptops, desktops, servers, and high-performance storage systems

SSD Flash Memory Organization

Flash Memory Organization

Data is stored in flash memory cells, grouped into hierarchical units.

- **Pages** – Smallest writable unit (typically 4KB)
- **Blocks** – Group of pages (e.g., 128 pages per block)

Warning

Data **cannot be overwritten directly** – a block must be erased before rewriting.

Limited Write Ability in SSDs

⚠ Write Wear

Flash memory cells **wear out** after a certain number of write cycles.

- **Wear Leveling Algorithms** help distribute writes evenly to extend lifespan
- SSD controllers use **garbage collection** to manage writes efficiently

File System Abstraction Layer

File System as an Abstraction Layer

File System

A **software layer** that abstracts raw hardware into structured storage, organizing and managing **files** and directories.

- Provides an interface for users and applications to access data
- Hides the complexities of hardware (HDD, SSD, etc.)

Why is it Needed?

- Without a file system, data would be stored as **raw bytes**
- Enables features like **naming, permissions, and metadata**
- Supports different storage media (HDD, SSD, etc) with a **common interface**

What is a File?

File

A **named collection of data** stored on a disk, representing **logical storage** rather than physical locations.

- Files can contain:
 - **Text** (documents, source code)
 - **Binary Data** (executables, images, videos)
 - **Metadata** (permissions, timestamps, size)
- The **file system keeps track** of file locations and access permissions

Example File

Name: report.pdf

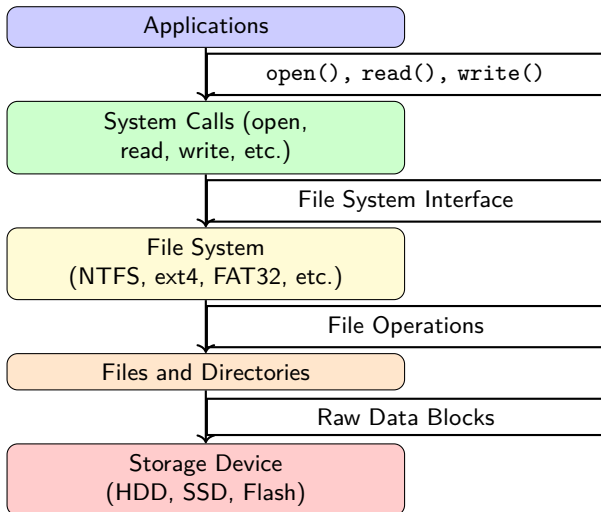
Path:

/home/user/documents/

Size: 1.5MB

Permissions: rw-r--r--

File System Abstraction



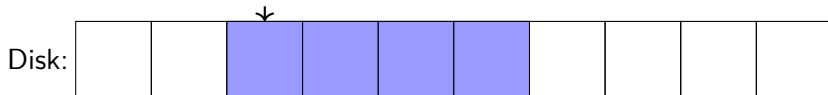
Contiguous File Allocation

Contiguous Allocation

A file is stored in **consecutive disk blocks**. Requires pre-allocating space; the file system tracks **starting block and file length**.

Example:

File A (Contiguous Blocks)

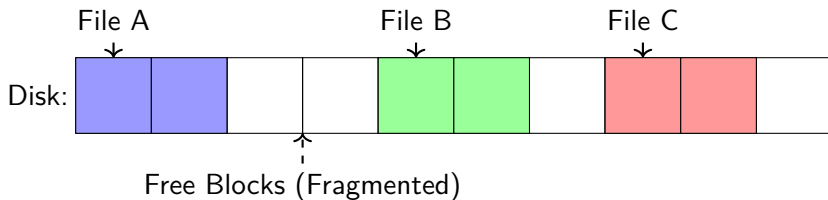


Fragmentation Problem in Contiguous Allocation

⚠ External Fragmentation

When files are deleted, small **free gaps** appear between allocated blocks. New files may not fit if they require **larger** contiguous space, leading to **wasted storage**.

Example:



Pros and Cons of Contiguous Allocation

✓ Advantages

- **Fast access** – Sequential reading is efficient
- **Simple management** – Only starting block and length needed

⚠ Disadvantages

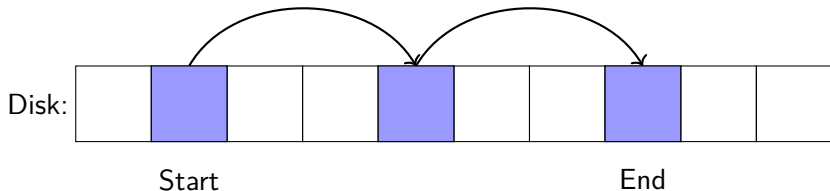
- **External fragmentation** – Small free spaces may become unusable
- **Difficult resizing** – Expanding a file is hard if next blocks are occupied
- **Pre-allocation issue** – Requires knowing file size in advance

Linked List File Allocation

Linked List Allocation

A file is stored as **scattered blocks** on disk. Each block contains **data** and a **pointer** to the next block. The file system tracks the **first block (head)**.

Example:



Pros and Cons of Linked List Allocation

✓ Advantages

- **No External Fragmentation** – Blocks can be anywhere
- **Efficient Use of Space** – No need for pre-allocation
- **Easy File Growth** – Just add new blocks
- **Simple** – File at starting index

⚠ Disadvantages

- **Slow Random Access** – Must traverse the list sequentially with the risk of multiple disk reads
- **Pointer Overhead** – Each block stores a pointer, reducing usable space

Exercise: I/O in Linked List Allocation

Parameters

- Block size: 1 KiB (1024 bytes)
- Pointer size: 4 bytes (stored at the end of each block)
- Usable data per block: 1020 bytes
- File `data.bin` occupies 10 blocks in linked list allocation
- **Assume** all file system metadata (directory entries, etc.) is **cached in RAM** – only data block reads count as disk I/O

Exercise: Questions

Questions

- 1 How many disk I/O operations to read the **first byte** of the file?
- 2 How many disk I/O operations to read the byte at **offset 5000**?
- 3 How many disk I/O operations to read the **last byte** of the file?
- 4 If **contiguous allocation** were used instead, how many I/O operations would question 2 require?

Solution: I/O in Linked List Allocation (Q1–Q2)

✓ Solution

Q1: The starting block number is already in RAM (directory entry). Read that one block from disk.

$$\text{I/O operations} = 1$$

Q2: Block index = $\lfloor 5000 \div 1020 \rfloor = 4$ (zero-indexed). Must traverse blocks $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$:

$$\text{I/O operations} = 5$$

Solution: I/O in Linked List Allocation (Q3–Q4)

✓ Solution

Q3: Total data = $10 \times 1020 = 10,200$ bytes. Last byte at offset 10,199 → block 9. Must traverse all 10 blocks:

$$\text{I/O operations} = 10$$

Q4: Starting block is in RAM. Block index = $\lfloor 5000 \div 1024 \rfloor = 4$. Jump directly to `start_block + 4`:

$$\text{I/O operations} = 1 \quad (\text{no traversal needed})$$

File Allocation Table (FAT)

FAT Allocation

A table-based version of **linked list allocation**. The FAT stores file block links, enabling **random access**.

- Each file has a **starting block**, and FAT entries point to the next block
- FAT table tracks file block links
- Allows **random access** compared to standard linked list allocation

FAT Directory Table

Directory Entry

In FAT, each directory contains a table of **32-byte entries** – one per file or subdirectory. The directory entry is the **starting point** for locating a file's data on disk.

What a Directory Entry Stores

- **Filename** (8 bytes) + **Extension** (3 bytes)
- **Attributes** – read-only, hidden, system, directory, archive
- **Timestamps** – creation, last access, last modification
- **Starting cluster number** – entry point into the FAT chain
- **File size** (4 bytes)

FAT File Access Flow

Complete Access Path

To read a file (e.g., /docs/report.txt):

- 1 **Root directory** – look up the entry for docs → get its starting cluster
- 2 **FAT chain for docs** – read directory data blocks to find report.txt entry
- 3 **Directory entry** – extract the **starting cluster** of report.txt
- 4 **FAT chain** – follow cluster → next cluster links until EOF
- 5 **Data blocks** – read each cluster's data from disk

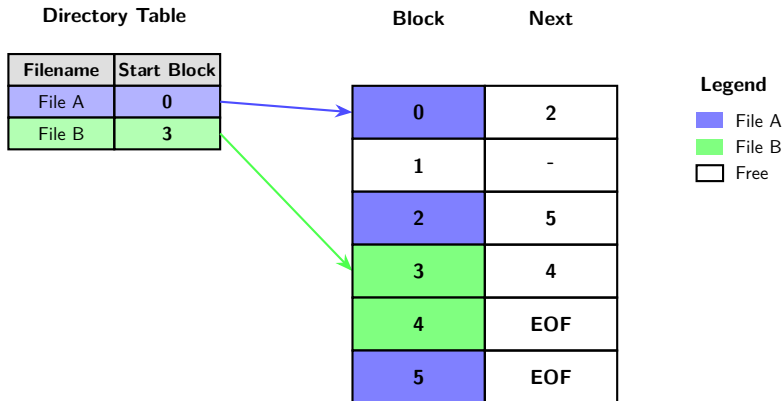
A Note about Directories



Directories Are Files

In FAT, a directory is just a file whose data blocks contain directory entries. The root directory is special – in FAT12/FAT16 it has a fixed location and size; in FAT32 it is a regular cluster chain.

File Allocation Table (FAT)



Pros and Cons of FAT Allocation

✓ Advantages

- **No External Fragmentation** – Blocks can be anywhere
- **Better than Linked List** – FAT allows faster random access

⚠ Disadvantages

- **Table Overhead** – FAT must be stored in memory

Size of the File Allocation Table (FAT)

FAT Table Size

The FAT table **grows** as disk size increases. Each entry corresponds to a block. Size depends on **number of blocks** and **entry size** (FAT12, FAT16, FAT32).

Example Calculation

A 1TB disk with **4KB block size**:

- Total blocks = $\frac{1\text{TB}}{4\text{KB}} = 2^{28}$ (268 million blocks)
- **FAT32 uses 4 bytes per entry**, so FAT size = $2^{28} \times 4\text{B} \approx 1\text{GB}$

Exercise: I/O in FAT Allocation

Parameters

- Block size: 1 KiB (1024 bytes)
- FAT entry size: 4 bytes
- File `data.bin` occupies 10 blocks in FAT allocation
- Each data block stores **1024 bytes** of usable data (no pointer overhead – pointers are in the FAT)
- The directory entry (in RAM) provides the starting block number

Exercise: I/O in FAT Allocation: Questions

Part A – FAT Cached in RAM

- 1 How many disk I/O operations to read the byte at **offset 5000**?
- 2 How many disk I/O operations to read the **last byte** of the file?

Part B – FAT Stored on Disk

Assume each FAT entry lookup requires **1 additional disk I/O**.

- 3 How many disk I/O operations to read the byte at **offset 5000**?
- 4 How many disk I/O operations to read the **last byte** of the file?

Solution: FAT I/O – Part A (FAT in RAM)

✓ Solution

Q1: Block index = $\lfloor 5000 \div 1024 \rfloor = 4$ (zero-indexed). Follow the FAT chain **in memory** (0 disk I/O) to find the physical block number, then read it:

$$\text{I/O operations} = 1$$

Q2: Total data = $10 \times 1024 = 10,240$ bytes. Last byte at offset 10,239 $\rightarrow$ block 9. FAT chain traversal is entirely in RAM:

$$\text{I/O operations} = 1$$

💡 Key Insight

When the FAT is cached in RAM, **any** random access costs just **1 disk I/O** – a huge improvement over linked list allocation.

Solution: FAT I/O – Part B (FAT on Disk)

✓ Solution

Q3: Block index = 4. Must read FAT entries for blocks $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ from disk (5 FAT reads), then read the data block:

$$\text{I/O operations} = 5 + 1 = 6$$

Q4: Block 9. Must read FAT entries for blocks $0 \rightarrow 1 \rightarrow \dots \rightarrow 9$ from disk (10 FAT reads), then read the data block:

$$\text{I/O operations} = 10 + 1 = 11$$

⚠ FAT on Disk $\approx$ Linked List

When the FAT cannot fit in RAM, performance degrades back toward linked list levels.

Defragmentation in HDDs

⚠ Fragmentation

Files stored in **non-contiguous** blocks over time force the read/write head to **seek across disk platters**, slowing down access.

💡 How Defragmentation Works

Moves fragmented blocks into **sequential order**, reducing seek time and improving **performance**.

Example: Before and After Defragmentation

Before



Why Defragmentation is Not Needed for SSDs

Key Differences

- **HDDs are mechanical** – Moving parts make fragmented access slower
- **SSDs have no moving parts** – Access time is the same for any block

Harmful for SSDs

Defrag writes data unnecessarily, causing more wear on flash memory cells.

Introduction to NTFS (New Technology File System)

NTFS

The **New Technology File System** – the default file system for modern Windows.

- **File metadata stored in MFT (Master File Table)**
- **Journaling** prevents corruption by tracking changes before writing
- **Access Control Lists (ACLs)** manage file permissions

Master File Table (MFT)

Master File Table (MFT)

The MFT is a table where each row is a **1 KB record** describing one file or directory on the volume. The first 16 entries are reserved for NTFS **system metadata**.

MFT #	Name	Purpose
0	\$MFT	The MFT itself
1	\$MFTMirr	Backup of first MFT entries
2	\$LogFile	Transaction journal
3	\$Volume	Volume metadata
4	\$AttrDef	Attribute definitions
5	. (root)	Root directory
6–15	(reserved)	Other system metadata
16+	User files	Files and directories

Resident vs Non-Resident Files

Resident Data

If a file's data is **small enough** (typically < 700 bytes), NTFS stores it **directly inside the MFT entry** – no external clusters needed. This is called **resident** storage.

Resident File

- Data lives **inside** the 1 KB MFT record
- **Single disk read** to get metadata + data
- Ideal for very small files and short directory listings

Non-Resident File

- Data is too large for the MFT record
- MFT entry stores **data runs** (start cluster, length) pointing to external clusters
- Requires **additional disk reads** to fetch file contents

B-Tree Directory Index (\$INDEX_ROOT)

B-Tree Index

NTFS directories store their child entries in a **B-tree** structure, enabling $O(\log n)$ lookups. The root of this tree is the \$INDEX_ROOT attribute inside the directory's MFT entry.

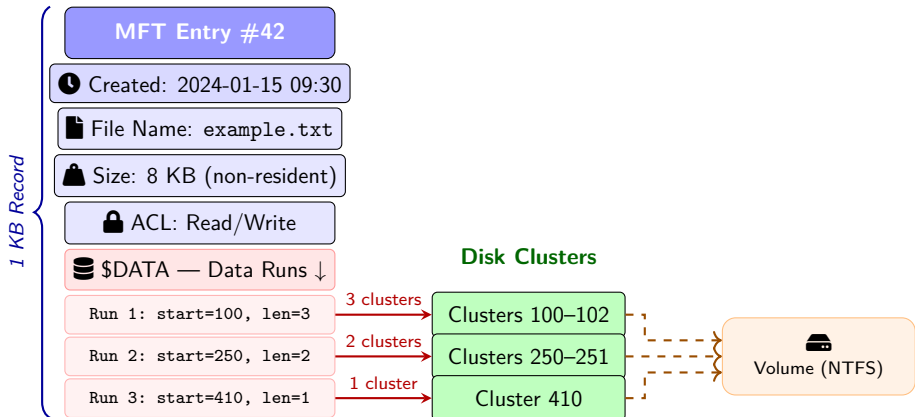
Small Directory (Resident)

- All entries fit inside \$INDEX_ROOT
- B-tree is **entirely resident** in the MFT record
- Single disk read to list the directory

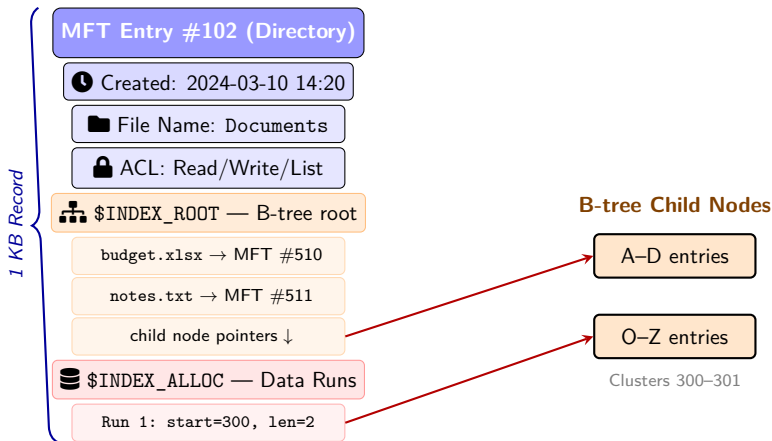
Large Directory (Non-Resident)

- \$INDEX_ROOT holds the B-tree root node
- Additional nodes spill into \$INDEX_ALLOCATION clusters on disk
- Tree is traversed by following pointers from root to leaf

MFT Entry Structure



MFT Entry Structure — Directory



MFT Entry Structure — Directory: Resident and Non-Resident

💡 Resident vs Non-Resident Index

Small directories keep the **entire B-tree inside** \$INDEX_ROOT (resident). When the directory grows, child nodes spill to disk via \$INDEX_ALLOCATION data runs — just like non-resident file data.

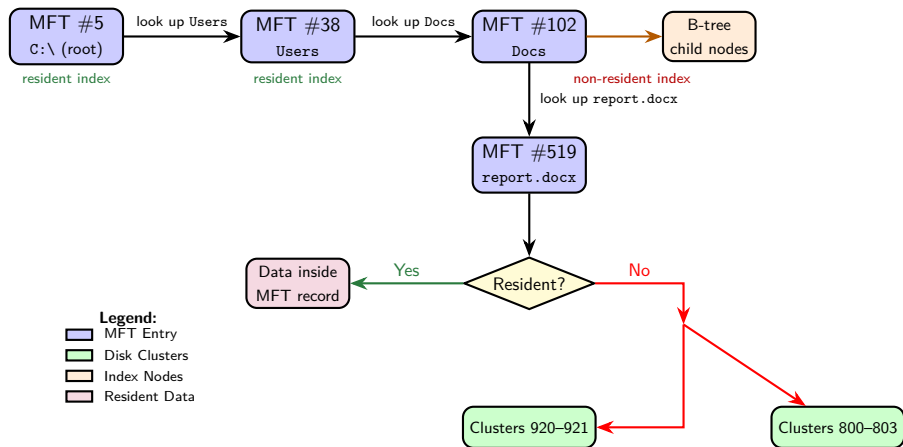
NTFS File Access Flow

NTFS File Access Flow

To read a file (e.g., C:\Users\Documents\report.docx):

- 1 Start at **MFT entry #5** (always the root directory in NTFS)
- 2 Read the root directory's **B-tree index** → look up Users → get its MFT record number
 - If the directory is **small**: B-tree is **resident** in \$INDEX_ROOT – single MFT read
 - If the directory is **large**: follow \$INDEX_ALLOCATION data runs to read B-tree child nodes from disk
- 3 Repeat for each path component (Users → Documents → report.docx)
- 4 Read the report.docx MFT entry:
 - If the file is **resident**: data is stored directly in the MFT record – done
 - If the file is **non-resident**: follow **data runs** to read file contents from disk clusters

NTFS File Access Flow: Diagram



Exercise: I/O in NTFS File Access

Parameters

- Cluster size: 4 KiB, MFT entry size: 1 KiB
- Path to resolve: C:\Users\Documents\report.docx
- **MFT entries are NOT cached** – each MFT entry read = 1 disk I/O
- Root directory (MFT #5): **resident** index
- Users directory: **non-resident** index, B-tree depth = 3
- Documents directory: **non-resident** index, B-tree depth = 2
- report.docx: **non-resident** file, 3 data runs

B-tree Traversal Cost

The B-tree **root node** is always inside the MFT entry (no extra I/O). Each additional level requires **1 disk I/O** to read the child node. A B-tree of depth d costs $d - 1$ extra I/O operations.

Exercise: I/O in NTFS File Access

Questions

- 1 How many disk I/O operations to resolve the full path and read the **first byte** of `report.docx`?
- 2 If `report.docx` were a **resident** file (< 700 bytes), how would the answer change?
- 3 If **all** directories had **resident** indexes, how would Q1 change?

Solution: NTFS I/O (Q1)

✓ Solution

Q1: Trace each step of the path resolution:

- 1 Read MFT #5 (root): **1 I/O** → index is resident, look up Users in memory
- 2 Read Users MFT entry: **1 I/O** → B-tree depth = 3, root is in entry, 2 child levels on disk: **2 I/O** → find Documents
- 3 Read Documents MFT entry: **1 I/O** → B-tree depth = 2, root is in entry, 1 child level on disk: **1 I/O** → find report.docx
- 4 Read report.docx MFT entry: **1 I/O** → get data runs, calculate target cluster
- 5 Read data cluster from disk: **1 I/O**

$$\text{Total} = 1 + 1 + 2 + 1 + 1 + 1 + 1 = 8 \text{ I/O}$$

Solution: NTFS I/O (Q2–Q3)

✓ Solution

Q2 (Resident file): Same path traversal (7 I/O), but the file data is **inside the MFT record** – no final cluster read:

$$\text{Total} = 8 - 1 = \mathbf{7} \text{ I/O}$$

Q3 (All resident indexes): Remove all B-tree child node reads ($2 + 1 = 3$ I/O saved). Only MFT entry reads + data cluster remain:

$$\text{Total} = 8 - 3 = \mathbf{5} \text{ I/O}$$

Journaling in NTFS

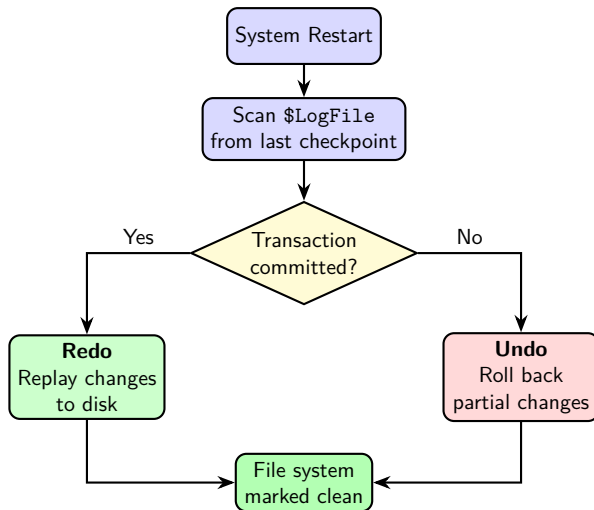
Write-Ahead Logging

NTFS uses a **write-ahead log** (\$LogFile) to ensure file system consistency. Every metadata change is recorded in the journal **before** being applied to disk.

Transaction Lifecycle

- 1 **Begin transaction** – record the intended operation in \$LogFile
- 2 **Write metadata changes** to disk (e.g., update MFT entry, directory index)
- 3 **Commit transaction** – mark the journal entry as complete

NTFS Journal Recovery on Restart



Journaling Limitations in NTFS

Metadata-Only Journaling

NTFS journaling supports only **metadata** (MFT entries, directory indexes, bitmaps) – not file data. This balances performance with reliability.

Single Inode System

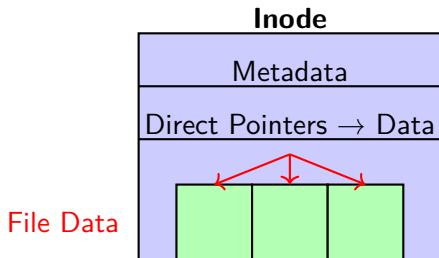
Inode

Each file has an **inode** storing file metadata (size, timestamps, permissions) and **direct pointers** to disk blocks.

- Simple and efficient for **small files**
- Stored on a pre-defined location on disk (can run out of inode space)
- **Limitation:** Random access depends on the number of direct pointers, can make locating a large file very slow

Single Inode System - An Example

Example: A Single Inode System



Multi-Level Inode System

💡 Solution: Multi-Level Inodes

Adds **Indirect Pointers** to extend file size limits beyond what direct pointers allow.

- **Direct Pointers** (point directly to data blocks) (48KB max size)
- **Single Indirect Pointer** (points to a block storing more pointers) (4MB max size)
- **Double Indirect Pointer** (points to a block storing indirect blocks) (4GB max size)
- **Triple Indirect Pointer** (points to a block storing double indirect pointers) (4TB max size)
- **Number of Indirections depends on size of file** in advanced implementations to avoid performance hit

Pros and Cons of Multi-Level Inodes

✓ Advantages

- **Scalable Storage** – Can support very large files
- **Efficient for Both Small and Large Files** – Direct pointers work for small files
- **No Preallocation Needed** – Grows dynamically

⚠ Disadvantages

- **Access Overhead** – Large files may require multiple lookups
- **Complexity** – More layers mean more management
- **Random Access Can Be Slow** – Deeply nested pointers require extra indirection

What is EXT?

EXT (Extended File System)

The default Linux file system family, based on the **multi-level inode system**.

- Internal algorithm attempts to ensure files on disk are located near each other to reduce internal fragmentation
- Advanced versions store **journaling information** (EXT3, EXT4) to prevent corruption

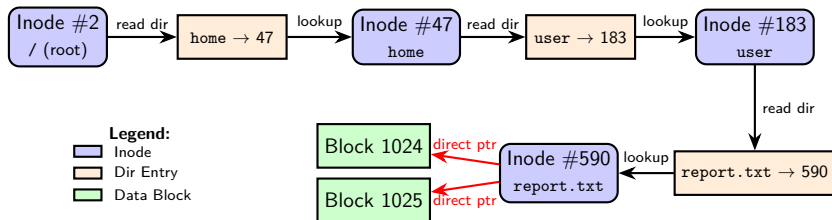
EXT4 File Access Flow

EXT4 File Access Flow

To read a file (e.g., `/home/user/report.txt`):

- 1 Start at **inode #2** (always the root directory in ext-family file systems)
- 2 Read the root directory's data blocks → look up `home` → get its inode number
- 3 Read inode for `home` → read its data blocks → look up `user` → get its inode number
- 4 Read inode for `user` → read its data blocks → look up `report.txt` → get its inode number
- 5 Read inode for `report.txt` → follow **direct/indirect pointers** to read file data

EXT4 File Access Flow



Exercise: Inode

UNIX-like File System Parameters

- Total volume size: 200 GiB
- Block size: 4 KiB
- Inode size: 256 bytes
- 1 inode per 8 data blocks
- Pointer size: 4 bytes
- Inode pointers: 10 direct, 2 single-indirect, 2 double-indirect, 1 triple-indirect

Assume indirect blocks are 4 KiB and store only block addresses.

Exercise: Inode

Questions to Solve

- 1 Total blocks in the file system
- 2 Inode table sizing
 - 2 Number of inodes
 - 2 Blocks for inode table
 - 2 Size of inode table in MiB
 - 2 Percentage of volume
- 3 Maximum file size
 - Contributions from direct, single-, double-, triple-indirect pointers
 - Express in bytes, MiB, GiB

Solution: Total Blocks

✓ Solution

$$\text{Total blocks} = \frac{200 \times 2^{30}}{2^{12}} = 52,428,800$$

Explanation: We first convert 200 GiB to bytes (200×2^{30} bytes), then divide by the block size 4 KiB (2^{12} bytes) to find the number of blocks.

Solution: Inode Table Sizing

✓ Solution

$$\text{Inodes} = \frac{52,428,800}{8} = 6,553,600 \quad (1 \text{ inode per } 8 \text{ blocks})$$

$$\text{Inode table size (bytes)} = 6,553,600 \times 256 = 1,677,721,600$$

(inode count $\times$ 256 bytes)

$$\text{Blocks for inode table} = \frac{1,677,721,600}{4096} = 409,600 \quad (\text{bytes} \div \text{block size})$$

$$\text{Size (MiB)} = \frac{1,677,721,600}{2^{20}} = 1,600 \quad (\text{bytes} \div 2^{20})$$

$$\text{Percentage} = \frac{1,600}{200 \times 1024} \times 100 \approx 0.78\% \quad (\text{MiB} \div \text{total MiB})$$

Solution: Maximum File Size

✓ Solution

Pointer contributions:

- Direct (10): $10 \times 4\text{KiB} = 40\text{ KiB} = 40,960\text{ B}$
 - 10 pointers $\times$ block size
- Single-indirect (2): $2 \times (1024 \times 4\text{KiB}) = 8,388,608\text{ B}$
 - 2 blocks of 1024 pointers each $\times$ block size
- Double-indirect (2): $2 \times (1024^2 \times 4\text{KiB}) = 8,589,934,592\text{ B}$
 - 2 sets of 1024^2 pointers $\times$ block size
- Triple-indirect (1): $1 \times (1024^3 \times 4\text{KiB}) = 4,398,046,511,104\text{ B}$
 - 1 set of 1024^3 pointers $\times$ block size

Total max file size = 4,406,644,875,264 B

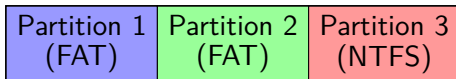
What is a Partition?

Partition

A **logically divided section** of a storage device. Each partition acts as an **independent volume**. File systems are applied to partitions, not the entire disk.

Why Partition a Disk?

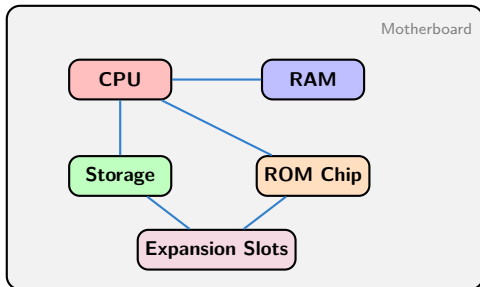
- Allows multiple **file systems** on a single disk
- Can **separate system files, user data, and backups**
- Helps with **dual-booting multiple operating systems**



What is a Motherboard?

Motherboard

The **main circuit board** of a computer. It physically connects and allows communication between the CPU, RAM, storage devices, and all other peripherals.



What is ROM & Firmware?

ROM — Read-Only Memory

A **non-volatile memory chip** on the motherboard that retains its contents **without power**. It stores the firmware. (Modern systems use flash memory that can be updated, but the term ROM persists.)

Firmware

Low-level software stored in ROM (or flash). It is the **first code that runs** when the computer powers on — before any operating system.

Why Do We Need Firmware?

Without firmware, the CPU has **no instructions at power-on**. Firmware initializes hardware, runs diagnostics, and hands control to the OS bootloader. It **bridges hardware and software**.

BIOS — Basic Input/Output System

The **legacy firmware standard**, found on older computers. Stored in ROM on the motherboard.

How BIOS Works

- Runs **POST** (Power-On Self-Test) to verify hardware is functional
- Searches for a bootable disk in a configured order
- Reads the disk's **first 512 bytes** — the Master Boot Record (MBR)
- Executes the **bootloader code** (446 bytes) found in the MBR

BIOS - Limitations

Limitations

- Runs in **16-bit real mode** — can only address **1 MB of memory** and use simple instructions; no multitasking or drivers
- No built-in security verification — any code in the MBR is executed blindly
- Tied to the **MBR** partition scheme (max 2 TB disks, 4 primary partitions)

UEFI - Solving BIOS' Limitations

UEFI — Unified Extensible Firmware Interface

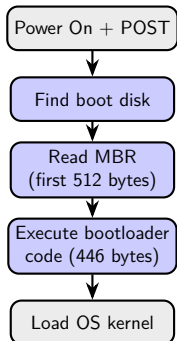
The **modern replacement** for BIOS. Found on virtually all computers made after ~2012.

✔ Advantages over BIOS

- Runs in **32/64-bit mode** — can address **GBs of memory** and run richer pre-boot code
- Has its own **boot manager** — no 446-byte boot-loader limit, can load different OS's boot-loading code and is not limited to a pre-set MBR.
- Reads **GPT** partition tables — supports large disks and many partitions
- Loads boot-loaders from a **dedicated UEFI boot partition** (explained in the GPT slide)
- Supports **Secure Boot** — only executes boot-loaders with a **trusted cryptographic signature**, preventing malware from replacing the boot-loader

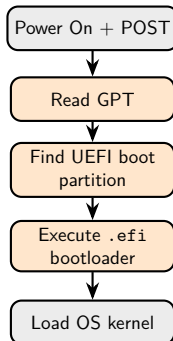
The Boot Process

BIOS Boot



16-bit real mode
No verification

UEFI Boot



32/64-bit mode
Secure Boot supported

MBR (Master Boot Record) Disk Partitions

MBR

The **traditional partitioning scheme**, primarily used with **BIOS-based booting**. Stores partitioning and boot information in the **first 512 bytes** of the disk.

MBR Layout (512 Bytes)

- **Bootloader code** (446 bytes) – first-stage code that locates and loads the OS bootloader
- **Partition table** (64 bytes) – 4 entries $\times$ 16 bytes each, defining up to 4 primary partitions
- **Boot signature** (2 bytes) – 0x55AA magic number confirming a valid MBR

MBR - Limitations

MBR Limitations

The bootloader must fit in 446 bytes. Only **4 primary partitions** are supported (or 3 primary + 1 extended with logical partitions inside). Maximum addressable disk size: **2 TB**.

GPT (GUID Partition Table)

GPT

A **modern partitioning scheme** that replaces MBR. Used in **UEFI-based boot mode**, with a backup header at the end of the disk.

Advantages of GPT

- **Supports up to 128 partitions** on Windows (Linux may support more)
- **Supports disks larger than 2TB** (max 9.4 ZB)
- **More reliable:** Stores multiple copies of the partition table with CRC32 checksums
- **Protective MBR:** First sector contains a fake MBR so legacy tools don't treat the disk as unformatted

GPT and UEFI

EFI System Partition (ESP)

A small **FAT32 partition** (typically 100–512 MB) that stores `.efi` boot-loader files. UEFI firmware reads this partition directly to find and execute the OS bootloader. Every UEFI system has one.

Linking

What is Linking?

Link

A **link** is a reference to a file or directory. Types include **Hard Links**, **Symbolic Links**, and **Junctions** (Windows).

- **Hard Links** – Multiple filenames pointing to the same inode, FAT entry or MFT record
- **Symbolic Links (Soft Links)** – A separate file pointing to another file's path

Why Use Links?

Avoid duplication of large files, maintain references even if files move, and support shared libraries and system shortcuts.

Hard Links

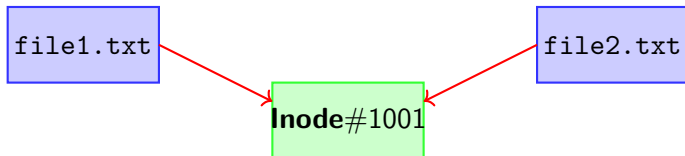
Hard Link

A **direct reference** to an existing file system entry. All hard links share the **same inode** and **file data**.

- Deleting one hard link does **not** remove the file until all links are deleted
- Can only be done on files on same partition
- Only works on files, not directories to avoid loops (except for '.' and '..')

Hard Links

Example: Hard Link Behavior with Inode



Creating a Hard Link:

```
$ ln file1.txt file2.txt
```

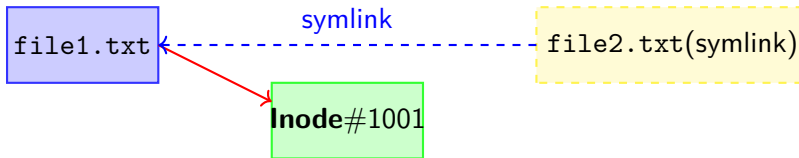
Symbolic (Soft) Links

Symbolic Link

A **shortcut** that points to another file's **path**. Unlike hard links, it does **not share the same inode**.

- If the original file is deleted, the symbolic link **breaks** (becomes invalid)
- Works with directories (Linux)

Example: Symbolic Link Behavior



Real-World Use Cases of Linking

Package Managers

Package managers like **APT**, **YUM**, **Homebrew** use **symbolic links** to ensure applications use the latest installed version.

```
/usr/local/bin/python ->  
/usr/local/Cellar/python3/3.9.1/bin/python3
```

Shared Libraries

Systems use **symbolic links** to manage shared libraries, helping different applications use **the same version**.

```
/lib/libssl.so -> /lib/libssl.so.1.1
```

File Descriptors

What is a File Descriptor?

File Descriptor (FD)

A **non-negative integer** representing an open file. Every process in Unix manages file descriptors through the kernel. Used for **files, pipes, sockets, and devices**.

Standard File Descriptors

- 0 → **Standard Input (stdin)**
- 1 → **Standard Output (stdout)**
- 2 → **Standard Error (stderr)**

Common File Operations in Unix

</> File System Calls

```
// 1. Open a file (returns file descriptor)
```

```
int fd = open("file.txt", O_RDONLY);
```

```
// 2. Read from a file
```

```
char buffer[100];
```

```
read(fd, buffer, 100);
```

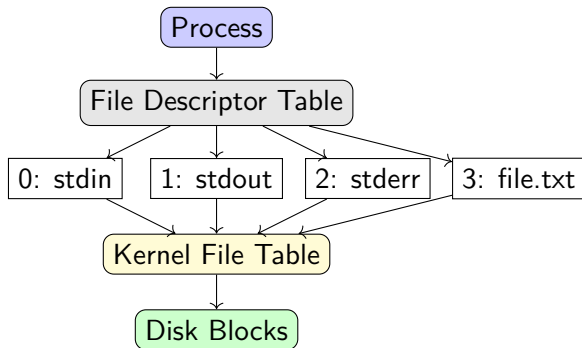
```
// 3. Write to a file
```

```
write(fd, "Hello", 5);
```

```
// 4. Close a file
```

```
close(fd);
```

File Descriptor Table and Process Interaction



How a Process Holds Open Files

Process File Management

Each process maintains a **list of all open files** as part of the **process control block (PCB)**.

- Each entry is a **file descriptor** that maps to:
 - A **File Descriptor Table (per process)**
 - The **Kernel File Table (system-wide)**

Example: A Process with Open Files

```
$ cat file.txt > output.log
```

- The process opens **file.txt** for reading and **output.log** for writing.
- These files are tracked in the **process file descriptor table**.

What Happens If Files Are Not Closed Correctly?

Consequences of Not Closing Files

- **Resource Leaks:** Open files consume system resources (file table slots)
- **File Locking Issues:** Some files remain locked and cannot be modified by other processes
- **Data Loss and Corruption:** If a file is **not flushed**, data might be lost
- **Exceeding File Descriptor Limits:** Every process (and the system itself) has a **limit on open files**

Sockets and Other Resources as File Descriptors

💡 Everything is a File

In Unix, **everything is a file** – including sockets, pipes, terminals, and devices. All are accessed via **file descriptors**.

- **Sockets** (network communication)
- **Pipes** (inter-process communication)
- **Terminals and Devices**

💡 Why Use File Descriptors for Sockets?

Allows **uniform handling of files and network resources**. Can use `read()`, `write()`, `close()` on sockets, just like files.

Exercise: File Descriptors After fork()

</> Given Code

```
int main() {
    int fd = open("log.txt",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);
    // fd = 3 (after stdin=0, stdout=1, stderr=2)

    pid_t pid = fork();

    if (pid == 0) {           // Child
        write(fd, "CHILD\n", 6);
        close(fd);
        exit(0);
    } else {                 // Parent
        wait(NULL);
        write(fd, "PARENT\n", 7);
        close(fd);
    }
    return 0;
}
```

Exercise: File Descriptors After `fork()`

Questions

- 1 After `fork()`, how many file descriptors point to `log.txt`? Explain why.
- 2 Do the parent and child share the same **file offset**, or does each have an independent offset?
- 3 What are the **possible contents** of `log.txt` after the program finishes?
- 4 If the child calls `close(fd)` **before** writing, can the parent still write to the file? Why?

Solution: FDs After `fork()` (Q1–Q2)

✓ Solution

Q1: Two FDs (one in each process). `fork()` duplicates the process's file descriptor table. Both parent and child have `fd = 3`, and both entries point to the **same entry in the kernel's open file table**.

✓ Solution

Q2: Shared offset. Since both FDs point to the same kernel file table entry, they share the file offset. When the child writes 6 bytes, the offset advances to 6 for **both** processes.

Solution: FDs After fork() (Q3–Q4)

✓ Solution

Q3: The parent calls `wait(NULL)`, so the order is deterministic – child writes first, then parent. The contents will be:

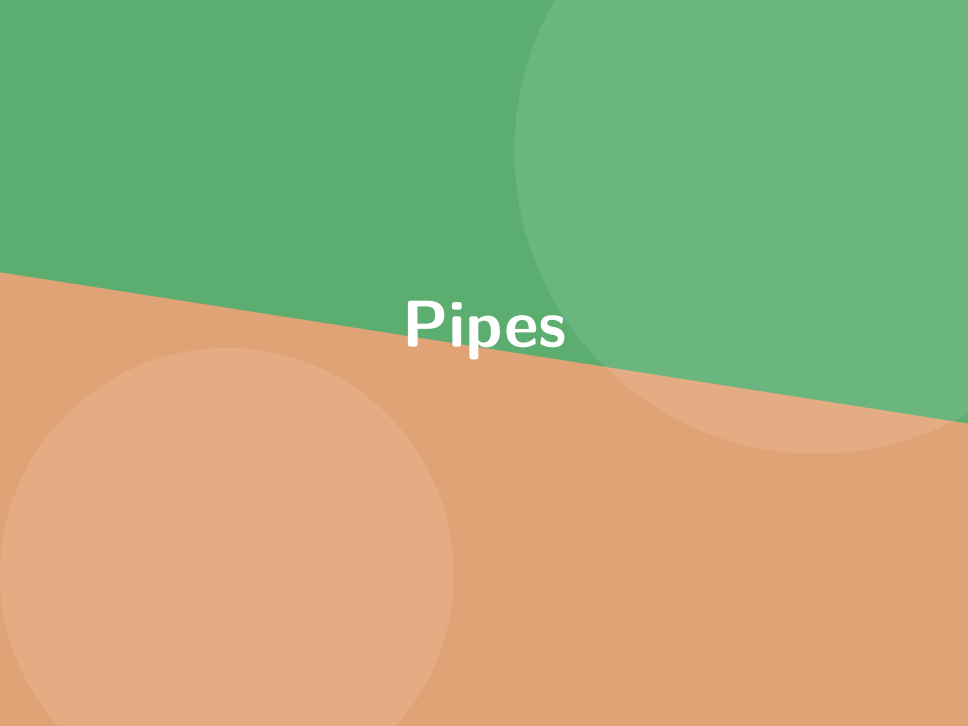
```
CHILD  
PARENT
```

Without `wait()`, both orderings would be possible.

✓ Solution

Q4: Yes. `close(fd)` only closes the child's file descriptor. The kernel file table entry remains open because the parent still references it. The entry is only freed when **all** FDs pointing to it are closed (reference counted).

Pipes

The background is split horizontally into a green upper half and an orange lower half. Large, semi-transparent circles in both colors are overlaid on the background, creating a layered effect.

What Are Pipes and Their Role in IPC?

Pipe

A mechanism for **inter-process communication (IPC)**. Pipes allow **data to flow between processes** via **unidirectional communication**.

Example: Unix Pipe

```
$ ls | grep "file"
```

Key Features

- **Efficient:** Direct memory buffer for process communication
- **Synchronization:** Writer blocks if the pipe is full, reader blocks if empty
- **Two Types:** **Anonymous pipes** and **named pipes (FIFOs)**

Anonymous (Unnamed) Pipes

Anonymous Pipe

A temporary communication channel between **parent and child processes**. Created using **pipe()** and **automatically destroyed** when processes exit.

Anonymous (Unnamed) Pipes

</> Creating a Pipe in C (Part 1)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int fd[2]; // File descriptors for the pipe
    char buffer[6]; // Extra byte for null
                  terminator

    if (pipe(fd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
```

Anonymous (Unnamed) Pipes

⌕ Creating a Pipe in C (Part 2)

```
if (fork() == 0) { // Child process
    close(fd[0]); // Close unused read end
    write(fd[1], "Hello", 5);
    close(fd[1]); // Close write end after
                  writing
    exit(EXIT_SUCCESS);
} else { // Parent process
    close(fd[1]); // Close unused write end
    read(fd[0], buffer, 5);
    buffer[5] = '\0'; // Null-terminate for
                      print
    printf("Parent received: %s\n", buffer);
    close(fd[0]); // Close read end after
                  reading
}
```

Anonymous (Unnamed) Pipes

Limitations

- **Works only between related processes (parent-child)**
- **Cannot communicate between unrelated processes**

Named Pipes (FIFOs)

Named Pipe (FIFO)

A **persistent pipe** that exists in the file system. Allows **unrelated processes** to communicate. Created using `mkfifo`.

Example: Creating and Using a FIFO

```
$ mkfifo mypipe      # Create named pipe
$ echo "Hello" > mypipe # Write to pipe
$ cat < mypipe        # Read from pipe
```

Key Features

- **Exists as a file** – persists after process termination
- **Can be used by any process** – not limited to parent-child
- **Behaves like a regular file** but reads/writes are blocking



RAID

What is RAID?

RAID

Redundant Array of Independent Disks – a method to combine multiple disks for performance, fault tolerance, or both.

- **Performance improvement (striping)**
- **Fault tolerance (redundancy)**
- **Both (hybrid methods)**

Key Insight

Different **RAID levels** offer trade-offs in speed, redundancy, and efficiency.

RAID 0 - Striping for Performance

RAID 0

- > Data is split into **blocks** and spread across multiple disks
- > **No redundancy**, but offers **high performance**

Warning

If one disk fails, **all data is lost**.

RAID 1 - Mirroring for Redundancy



RAID 1

- Data is duplicated on **two disks**
- **High redundancy** but **half storage efficiency**
- Can survive **one disk failure** without data loss

RAID 1+0 (RAID 10) - Mirroring + Striping

RAID 10

- **Mirroring first**, then **striping** for performance
- Can survive **multiple disk failures**, provided that **no mirrored pair is lost**
- Good for **high-performance, high-redundancy setups**

RAID 4 - Dedicated Parity Disk

RAID 4

- **Data is striped** across disks (like RAID 0), with a **dedicated parity disk**
- **Can recover from one disk failure** using the parity disk

Warning

Parity disk can become a bottleneck for writes.

RAID 5 - Distributed Parity for Fault Tolerance

RAID 5

- Data is striped across **multiple disks**
- **Parity is distributed** across disks to prevent bottlenecks
- Can survive **one disk failure** without losing data

RAID 6 - Double Parity for Extra Redundancy

RAID 6

- Like RAID 5 but with **two parity blocks** per stripe
- Can survive **two simultaneous disk failures**
- **Increased reliability** but **higher write overhead**

XOR-Based Data Recovery in RAID 4/5/6

💡 XOR for Parity

RAID 4, 5, and 6 use **XOR operations** to calculate parity. If a disk fails, missing data is **reconstructed using XOR**.

💻 XOR Operations

Calculating Parity: $A1 \oplus A2 \oplus A3 = P$

Recovering a Lost Disk: $A1 \oplus A2 \oplus P = A3$

💻 RAID 6 Enhancements

- Uses **two levels of parity** for dual-disk failure recovery
- Implements **Weighted XOR** to improve reconstruction efficiency

Exercise: RAID Capacity Calculation

Parameters

6 identical disks, each 2 TiB.

Fill in the Table

	RAID 0	RAID 1	RAID 5	RAID 6	RAID 10
Usable capacity	?	?	?	?	?
Max disk failures	?	?	?	?	?

Solution: RAID Usable Capacity

✓ Solution

	RAID 0	RAID 1	RAID 5	RAID 6	RAID 10
Usable capacity	12 TiB	6 TiB	10 TiB	8 TiB	6 TiB
Max disk failures	0	5	1	2	1-3

📁 Capacity Formulas

- **RAID 0:** $N \times S = 6 \times 2 = 12$ TiB (no redundancy)
- **RAID 1:** $\frac{N}{2} \times S = 3 \times 2 = 6$ TiB (full mirror)
- **RAID 5:** $(N - 1) \times S = 5 \times 2 = 10$ TiB (1 disk of parity)
- **RAID 6:** $(N - 2) \times S = 4 \times 2 = 8$ TiB (2 disks of parity)
- **RAID 10:** $\frac{N}{2} \times S = 3 \times 2 = 6$ TiB (mirror + stripe)

Solution: RAID Fault Tolerance

✓ Solution

- **RAID 0: 0 failures** – no redundancy; any single disk failure loses all data
- **RAID 1: 5 failures** – 3 mirror pairs; can lose all but one disk in each pair. In the best case, 5 of 6 disks can fail (one survivor per pair is enough)
- **RAID 5: 1 failure** – one disk's worth of distributed parity allows reconstructing any single failed disk via XOR
- **RAID 6: 2 failures** – two independent parity blocks per stripe allow recovering from any two simultaneous disk failures
- **RAID 10: 1–3 failures** – guaranteed to survive 1 failure; can survive up to 3 if no mirror pair loses both disks

Exercise: XOR Parity & Recovery

RAID 5 Stripe Data

A RAID 5 array with 4 disks. One stripe contains the following 8-bit data blocks:

Disk 1	Disk 2	Disk 3	Disk 4 (Parity)
10110011	01101010	11010001	?

Questions

- 1 **Calculate** the parity block stored on Disk 4.
- 2 Disk 2 **fails**. Reconstruct its data using the remaining disks.

Solution: XOR Parity Calculation (Q1)

✓ Q1: Calculate Parity

XOR all data disks together:

$$\begin{array}{rcl} & 10110011 & \text{(Disk 1)} \\ \oplus & 01101010 & \text{(Disk 2)} \\ \hline = & 11011001 & \\ \oplus & 11010001 & \text{(Disk 3)} \\ \hline = & 00001000 & \text{(Parity} \rightarrow \text{Disk 4)} \end{array}$$

Solution: XOR Disk Recovery (Q2)

✓ Q2: Reconstruct Disk 2

XOR all surviving disks (including parity):

$$\begin{array}{rcl} & 10110011 & \text{(Disk 1)} \\ \oplus & 11010001 & \text{(Disk 3)} \\ \hline = & 01100010 & \\ \oplus & 00001000 & \text{(Disk 4 --- parity)} \\ \hline = & 01101010 & \text{(Recovered Disk 2)} \end{array}$$

💡 Why This Works

XOR has two key properties: $A \oplus A = 0$ and $A \oplus 0 = A$. When we XOR all surviving disks including parity, the failed disk's data is the only value that "remains" – everything else cancels out.

See You Next Week!

Questions?

Recitation 7

Virtual Machines & Virtualization

Daniel Karalnik Liam Tal

Spring Semester 2026



Table of Contents

- 1 A Brief History of Virtualization
- 2 What is Virtualization?
- 3 Hypervisor Types
- 4 CPU Virtualization

Table of Contents (cont.)

- 5 Memory Virtualization
- 6 Containers vs Virtual Machines
- 7 Virtualization & Security
- 8 The Cloud Abstraction Ladder

A Brief History of Virtualization

The Origins (1960s–1990s)

It Started with IBM

IBM CP-40 (1966) and CP/CMS (1967) were the first systems to fully virtualize a mainframe — letting multiple users each run their own isolated “virtual machine.”

- **1966 — IBM CP-40:** First true virtual machine implementation
- **1974 — Popek & Goldberg** publish formal virtualization requirements
- **1998 — VMware founded:** Brings virtualization to commodity x86 hardware
 - x86 was considered “non-virtualizable” at the time because some sensitive instructions didn’t trap

The Modern Era (2000s–Today)

- **2003** — **Xen** hypervisor released (paravirtualization approach)
- **2005–2006** — **Intel VT-x / AMD-V**: Hardware-assisted virtualization finally makes x86 efficiently virtualizable
- **2006** — **AWS EC2 launches**: Virtualization becomes the foundation of cloud computing
- **2013** — **Docker**: Containers bring OS-level virtualization to the mainstream
- **2014** — **Kubernetes**: Container orchestration at scale
- **Today**: Billions of VMs and containers running simultaneously worldwide

Key Insight

Every time you use a cloud service, stream a video, or open a web app — virtualization is involved somewhere in the stack.

What is Virtualization?

Why Virtualization?

💡 From University to Industry

You already understand processes as isolated execution environments — virtualization takes this idea further by isolating **entire operating systems**.

- **Server consolidation** — run multiple workloads on one physical machine
- **Isolation & security** — separate **tenants** (independent users/organizations) on shared hardware
- **Development & testing** — reproduce environments reliably
- **Cloud computing** — the foundation of AWS, Azure, GCP

The Idea Behind a Virtual Machine

💡 A Computer Inside a Computer

Imagine you could take a **physical computer** — with its own CPU, memory, disk, and network — and recreate it entirely in **software**. A program running inside this fake computer wouldn't know the difference. That's a virtual machine.

- You already know this idea at a smaller scale:
 - A **process** thinks it has its own CPU (scheduling) and its own memory (virtual address space)
 - A **virtual machine** takes this further — an entire OS thinks it has its own hardware
- The key trick: a software layer **intercepts** hardware access and **fakes** the responses, so the OS running inside never realizes it's not on real hardware

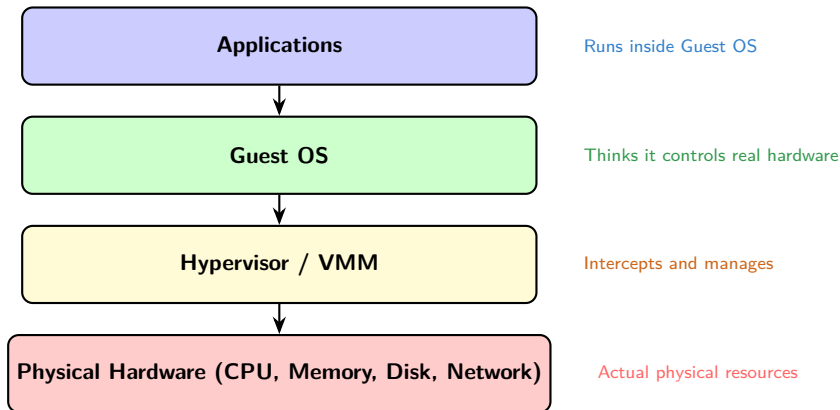
What is a Virtual Machine? — Formal Definition

Virtual Machine

A **software emulation** of a complete computer system that runs its own operating system (the **guest**) on top of a physical machine (the **host**), managed by a **Virtual Machine Monitor (VMM)** or **hypervisor**.

- **Host OS / Host machine** — the physical hardware + its OS
- **Guest OS** — the OS running inside the VM
- **Hypervisor / VMM** — the software layer that creates and manages VMs

Virtualization Abstraction Layers



Hypervisor Types

What is a Hypervisor?

The OS of Operating Systems

You know that the **OS manages processes** — it decides which process runs, how much memory it gets, and keeps processes from interfering with each other. A **hypervisor** does the same thing, but for **entire operating systems**.

Hypervisor / VMM

A software (or firmware) layer that **creates, runs, and manages virtual machines**. It sits between the VMs and the physical hardware, controlling access to CPU, memory, disk, and network.

Hypervisor Roles

- A Hypervisor, also called a **Virtual Machine Monitor (VMM)**
- Responsible for:
 - **Multiplexing** hardware across multiple guest OSes
 - **Isolating** guests from each other and from the host
 - **Emulating** or mediating hardware access for each guest

Type 1 Hypervisor (Bare-Metal)

Type 1 — Bare-Metal

Runs **directly on hardware** — no host OS. The hypervisor **is** the OS.

- Examples: **VMware ESXi, Microsoft Hyper-V, Xen, KVM** (Linux kernel module)
- Used in **data centers and cloud providers** (For example, AWS's EC2 runs on a type 1 hypervisor)
- Lower overhead, better performance

KVM — The Linux Approach

KVM (Kernel-based Virtual Machine)

A **Linux kernel module** that turns the Linux kernel itself into a **Type 1 hypervisor** (although lines become a bit blurred). Instead of replacing the OS, KVM **extends** it — Linux keeps running normally, but gains the ability to host VMs.

- KVM leverages **hardware-assisted virtualization** (VT-x / AMD-V) — without it, KVM cannot run
- KVM handles **CPU and memory virtualization**; QEMU for everything else (disk, network, display).

Type 2 Hypervisor (Hosted)

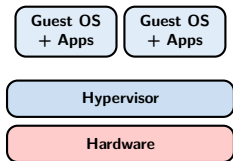
Type 2 — Hosted

Runs **on top of a host operating system** as a regular application.

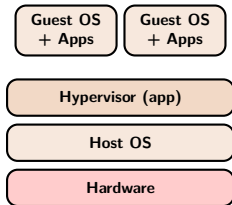
- Examples: **VirtualBox, VMware Workstation, Parallels**
- Common for **development, testing, and education**
- Higher overhead — must go through the host OS

Type 1 vs Type 2 — Comparison

Type 1 (Bare-Metal)



Type 2 (Hosted)



Comparison

Property	Type 1	Type 2
Performance	Near-native	Higher overhead
Use-case	Data centers, cloud	Dev, testing, education
Overhead	Low	Moderate
Examples	ESXi, KVM, Xen	VirtualBox, Parallels

CPU Virtualization

The Core Challenge



The Problem

A guest OS expects to execute **privileged instructions** (e.g., modifying page tables, handling interrupts). But only the **real OS / hypervisor** should be allowed to do this.



Key Insight

This is the same protection problem you already know from **user mode vs kernel mode** — but now applied one level up.

Sensitive vs Privileged Instructions

Two Categories of Instructions

- **Privileged instruction** — an instruction that **traps** (causes a fault) when executed outside kernel mode. You already know these from user/kernel mode.
- **Sensitive instruction** — any instruction whose behavior **depends on the privilege level** or that could **reveal/affect the true state of the machine**. This is a broader category.

Key Insight

All privileged instructions are sensitive, but not all sensitive instructions are privileged. This gap is the core problem for virtualization.

Trap-and-Emulate

Trap-and-Emulate

The guest OS runs in **user mode**. When it attempts a **privileged instruction**, the CPU **traps** to the hypervisor, which **emulates** the effect on behalf of the guest.

- Works naturally when sensitive instructions are also privileged — the CPU traps, and the hypervisor handles them
- Problem: x86 (before VT-x) has instructions that are **sensitive but not privileged** — they don't trap, they silently behave differently

The x86 Problem

Around 17 instructions in the original x86 ISA were sensitive but not privileged. Trap-and-emulate alone **cannot work** on x86 — the hypervisor never gets control when these instructions execute.

Full Emulation (Different CPU Architectures)

Full Emulation

The hypervisor **interprets every guest instruction in software**, translating it to the host's native instruction set. Used when guest and host have **different CPU architectures**.

- **QEMU** — can emulate ARM on x86, MIPS on ARM, etc. Every instruction is decoded and re-executed.

Performance Cost

Full emulation is **orders of magnitude slower** than native execution. Acceptable for development/compatibility, not for production workloads.

Binary Translation (Same CPU, Software Workaround)

Binary Translation

The hypervisor **scans guest code at runtime** and dynamically rewrites problematic instructions into safe equivalents. The host CPU executes the rewritten code natively.

- **VMware's breakthrough (1999):** Made x86 virtualization practical before hardware support existed
- Only **sensitive non-privileged instructions** need translation — safe instructions run at native speed
- Translated blocks are **cached** so repeated code runs fast

Key Insight

This is how VMware solved the “x86 is not virtualizable” problem — by rewriting the few problematic instructions on-the-fly.

Paravirtualization

Paravirtualization

Instead of tricking the guest OS into thinking it runs on real hardware, the guest is **modified to cooperate** with the hypervisor. The guest **knows** it's virtualized and uses special **hypercalls** to communicate directly with the hypervisor.

Hypercalls

A **hypercall** is to the hypervisor what a **system call** is to the OS. Instead of executing a privileged instruction and waiting for a trap, the guest explicitly calls the hypervisor to perform the operation — faster and with no ambiguity.

Paravirtualization - Pros and Cons

- Pioneered by **Xen** (2003) — required modifying the guest Linux kernel
- **Pros:** Better performance than binary translation, no hardware support needed
- **Cons:** Cannot run **unmodified** guest OSES (e.g., Windows couldn't be paravirtualized without Microsoft's cooperation)

Key Insight

Today, paravirtualization lives on in **hybrid form** — modern VMs use hardware-assisted CPU virtualization but paravirtualized I/O drivers (**virtio**) for better disk and network performance.

System Call Translation (WSL1)

System Call Translation

Instead of virtualizing hardware, the host OS **translates guest system calls** into its own native calls. No separate kernel runs.

WSL1 (2016)

Windows translated **Linux system calls** into equivalent Windows NT kernel calls. No Linux kernel running at all — just a translation layer.

System Call Translation (WSL1) - Pros and Cons

- Fast for simple workloads, but **incomplete**
- Not all Linux syscalls had NT equivalents (e.g., `fork()` semantics differ fundamentally)
- Some Linux software simply couldn't run

Fundamental Limitation

Perfect translation requires the host kernel to support every semantic of the guest kernel's API — and Linux and Windows kernels were designed with very different assumptions.

From Translation to VM (WSL2)

WSL2 (2019)

Microsoft gave up on translation and switched to running a **real Linux kernel** inside a **lightweight VM**. Full Linux compatibility, slightly more overhead.

- The VM is managed by **Hyper-V** — Windows' built-in **Type 1 hypervisor**
- **Lightweight VM**: a stripped-down VM with a **minimal kernel**, **no full OS boot**, and **shared resources** with the host (e.g., file system, networking) — starts in ~ 1 second instead of minutes
- Unlike a traditional VM, it doesn't carry a full guest OS with its own desktop, services, and drivers — just the kernel and what's needed to run Linux processes

Hardware-Assisted Virtualization

Hardware-Assisted Virtualization

Modern CPUs (Intel **VT-x**, AMD **AMD-V**) add a new privilege level — sometimes called **Ring -1** or **VMX root mode** — where the hypervisor runs, while the guest OS runs in its normal Ring 0.

Key Insight

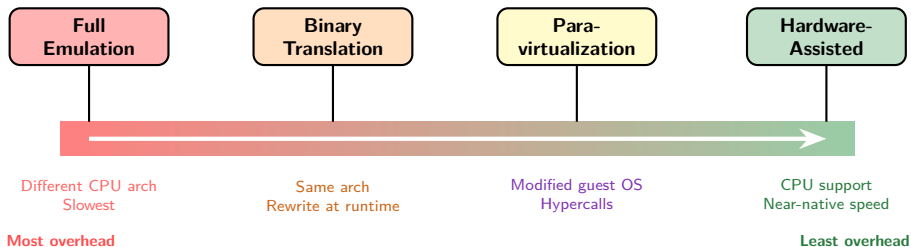
With hardware support, the guest OS can execute privileged instructions natively — the CPU itself knows about virtualization and traps to the hypervisor only when necessary. No binary translation needed.

Why do we even need this at the Hardware Level?

VM Exit

A **VM exit** occurs when the CPU transfers control from the guest back to the hypervisor — for example, when the guest executes a sensitive instruction or accesses a device. The hypervisor handles the event, then resumes the guest (**VM entry**). VM exits are expensive (thousands of cycles), so minimizing them is a key performance goal.

The Virtualization Spectrum



💡 Key Insight

In practice, modern hypervisors combine techniques — e.g., KVM uses hardware-assisted CPU virtualization, paravirtualized I/O (virtio), and emulation for legacy devices.

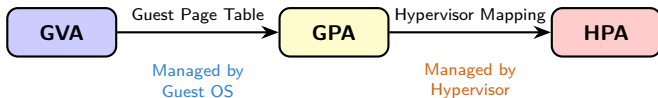
Memory Virtualization

The Double Translation Problem

⚠ Two Levels of Translation

In a VM, each memory access requires **two translations**:

- **Guest Virtual Address (GVA) → Guest Physical Address (GPA)**
— using the guest's page table
- **Guest Physical Address (GPA) → Host Physical Address (HPA)**
— using the hypervisor's mapping



Double Trouble

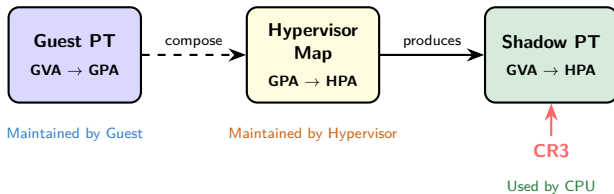
Key Insight

The guest OS thinks it controls physical memory at address 0x1000 — but that's really a *guest physical* address. The hypervisor maps it to some completely different host physical address.

Shadow Page Tables

Shadow Page Tables

The hypervisor maintains a **hidden, combined page table** that maps directly from **GVA** $\rightarrow$ **HPA**, bypassing the two-step translation. The CPU's actual page table register (CR3) points to the **shadow**, not the guest's table.



Shadow Page Table Overhead

Expensive to Maintain

Every time the guest **modifies its page table** (new process, page fault, `mmap`), the hypervisor must **intercept** the change (VM exit / trap) and **update the shadow** to match. Every guest CR3 switch (context switch) may require **switching or rebuilding** the shadow table.

- **Frequent VM exits** — each trapped page table write is expensive (thousands of cycles)
- **Memory overhead** — hypervisor stores a shadow PT for every guest process
- **Complexity** — hypervisor must track all guest PT modifications in real-time

The Shadow Problem

Key Insight

Shadow page tables essentially require the hypervisor to be a “mirror” of the guest’s memory management — any desync causes crashes or security holes.

Extended Page Tables (EPT / NPT)

Extended Page Tables

A **hardware feature** (Intel EPT, AMD NPT) that lets the CPU perform **both translation levels in hardware**. The guest page table maps GVA $\rightarrow$ GPA; a second hardware-managed table maps GPA $\rightarrow$ HPA. The CPU walks both automatically.

- **No shadow tables needed** — the guest manages its own page table freely, no traps required
- **No VM exits** on guest page table modifications — huge performance win
- Trade-off: a **TLB miss** now requires a **2D page walk** (guest levels $\times$ host levels), which can be slow. Mitigated by **large pages** and **EPT caching**.

Shadow PT vs EPT — Comparison

Comparison

Property	Shadow PT	EPT / NPT
Maintained by	Hypervisor (software)	CPU (hardware)
VM exits on PT changes	Yes (frequent)	No
Memory overhead	Shadow PT per process	Single EPT per VM
TLB miss cost	Standard page walk	Nested 2D walk
Hardware required	None	Intel EPT / AMD NPT
Used in practice	Legacy hypervisors	All modern hypervisors

TLB and Virtual Machines

When switching between VMs, the TLB must distinguish entries from different guests. **Tagged TLBs** (Intel **VPID**, AMD **ASID**) assign an ID to each VM — no full flush needed on VM switch. Same idea as PCID for process context switches — but now at the VM level.

Containers vs Virtual Machines

OS-Level Virtualization

Container

A **lightweight, isolated execution environment** that shares the **host kernel** instead of running its own OS. Provides process-level isolation using OS features.

- No separate guest OS — containers are just **isolated processes**
- Much faster startup, lower memory overhead
- Industry standard: **Docker**, orchestrated by **Kubernetes (K8s)**

Linux Building Blocks: Namespaces



Namespaces

The Linux kernel feature that provides **isolation** — each container gets its own view of system resources.

Namespace	What it isolates
PID	Process IDs (container sees its own PID 1)
NET	Network interfaces, IP addresses, ports
MNT	Mount points / file system view
UTS	Hostname
IPC	Inter-process communication
USER	UID/GID mapping

Linux Building Blocks: cgroups

Control Groups (cgroups)

The Linux kernel feature that provides **resource limiting** — controls how much CPU, memory, disk I/O, etc. a container can consume.

- **CPU limits** — max percentage or specific cores
- **Memory limits** — hard cap, OOM killer integration
- **I/O bandwidth** — throttle disk and network

Warning

Without cgroups, a single runaway container could starve the entire host — similar to how without scheduling, a single process could monopolize the CPU.

Container Runtimes

Container Runtime

The software that actually **creates and runs containers** — it talks to the kernel to set up namespaces, cgroups, and the root filesystem for each container. Think of it as the “hypervisor equivalent” for containers.

Our Context

➤ In this recitation we will focus on docker containers (“containerd”).

Docker in Practice: Images and Containers

Docker Image

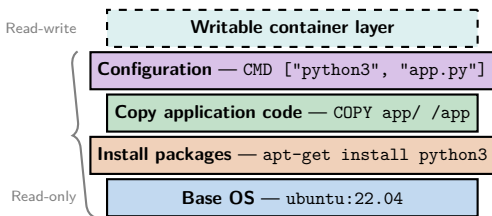
A **read-only template** containing the application code, runtime, libraries, and dependencies. An image is a **blueprint**; a container is a **running instance** of that image.

- **Image** = packaged file system snapshot (like a class in OOP)
- **Container** = running process created from an image (like an instance)
- Images are stored in **registries** (Docker Hub, AWS ECR, GitHub Container Registry)

Docker in Practice: Layers

Image Layers

Docker images are built from **stacked read-only layers**. Each instruction in a Dockerfile creates a new layer. Containers add a thin **writable layer** on top.



Key Insight

Layers are **shared** between images. If 10 containers use `ubuntu:22.04` as a base, that layer is stored only **once** on disk.

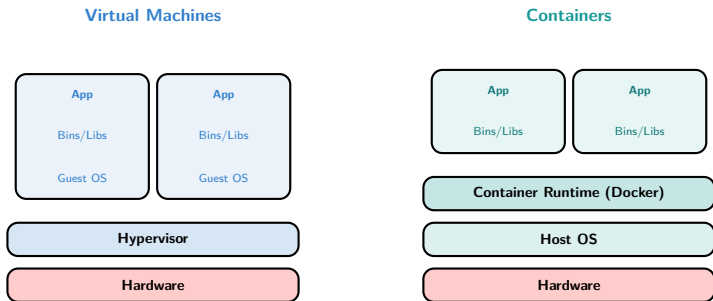
Docker in Practice: Dockerfile Example

</> Example Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "server.py"]
```

- Each line creates a **layer** — ordering matters for **cache efficiency**
- `COPY requirements.txt` before `COPY .` ensures dependency layer is cached unless deps change

VMs vs Containers — Architecture



VMs each carry a full **Guest OS** — containers **share the host kernel**.

VMs vs Containers — Comparison

Full Comparison

Property	Virtual Machines	Containers
Isolation	Full hardware	Process-level (shared kernel)
Startup time	Seconds to minutes	Milliseconds to seconds
Overhead	High (full OS per VM)	Low (shared kernel)
Image size	GBs (includes full OS)	MBs (app + deps only)
OS support	Any OS on any host	Same kernel as host
Security	Hypervisor (strong)	Namespaces (weaker)
Use-case	Multi-tenant, diff. OSes	Microservices, CI/CD

When to Use What?

Industry Rule of Thumb

- **VMs** when you need **strong isolation**, different OS types, or **multi-tenant security** (cloud providers, banks)
- **Containers** when you need **fast deployment, scaling, and microservices** (web apps, CI/CD pipelines)
- **Both together** — the most common production pattern: containers running inside VMs for defense-in-depth (e.g., AWS ECS on EC2, GKE nodes are VMs running containers)

Virtualization & Security

The Security Promise

Isolation as Security

VMs provide a **security boundary** — a compromised guest should not be able to affect the host or other guests.

- Hypervisor is a **small, auditable attack surface** (especially Type 1)
- Each VM has its own **address space, kernel, devices** — full separation
- This is what enables **multi-tenant cloud** — your VM runs on the same physical hardware as a stranger's VM

VM Escape Attacks

VM Escape

An attack where malicious code running **inside a guest VM** exploits a vulnerability in the **hypervisor** to break out and access the host or other VMs.

Key Insight

VM escape is considered one of the most severe vulnerabilities in cloud computing — it breaks the fundamental trust model.

Container Security Considerations

⚠ Weaker Isolation

Containers share the host kernel — a **kernel vulnerability** affects all containers on the host.

- VMs: attacker must escape the hypervisor (small attack surface)
- Containers: attacker must escape namespace/cgroup boundary (larger kernel attack surface)

💡 Defense in Depth

- Containers inside VMs (isolation + efficiency)
- **seccomp** — filter which system calls a container can make
- **AppArmor** / **SELinux** — Security Extensions
- Read-only file systems, minimal base images
- **Rootless containers** — processes run as unprivileged users on the host

MicroVMs — Best of Both Worlds

MicroVM

A **minimal, purpose-built virtual machine** designed for running a **single container or function**. Strips away everything a traditional VM doesn't need — no BIOS, no USB, no display — keeping only a tiny device model for disk and network.

AWS Firecracker

Built by AWS on top of **KVM**. Each Lambda function or Fargate container runs inside its own Firecracker microVM.

- ▶ Boots a Linux kernel in **~125 ms** (vs. seconds for a traditional VM)
- ▶ Memory overhead of **~5 MB** per microVM
- ▶ Runs **thousands of microVMs** on a single host
- ▶ Each microVM gets **its own kernel** — a kernel exploit in one container cannot reach another

Security Comparison Summary

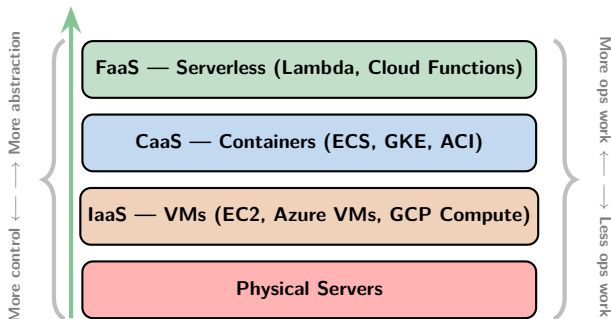


Security Comparison

Aspect	Virtual Machines	Containers
Isolation	Hypervisor + kernel	Namespaces + cgroups
Attack surface	Small (hypervisor)	Large (host kernel)
Escape severity	Critical (rare)	High (more common)
Mitigations	HW support, minimal VMM	seccomp, AppArmor, rootless

The Cloud Abstraction Ladder

From Hardware to Serverless



Good Luck on Your Exams!

It's been a great semester.

Keep exploring — the OS is everywhere.